



Network Security

I 5 Obiettivi Fondamentali della Sicurezza

La sicurezza informatica non è un concetto unico, ma si basa sul raggiungimento e sul mantenimento di specifici obiettivi per proteggere sia i dati che l'intera infrastruttura.

Il "Pentagono" della Sicurezza:

- **Confidentiality (Riservatezza & Privacy):** Garantisce che le informazioni private o confidenziali non vengano lette o scoperte da persone non autorizzate. Include anche la privacy, ovvero il diritto di controllare chi raccoglie i tuoi dati e a chi vengono divulgati.
- **Integrity (Integrità):** Assicura che i dati, i programmi e i sistemi vengano modificati *solo* in modo autorizzato e specifico. L'obiettivo è avere la certezza matematica che il sistema faccia il suo dovere senza subire manipolazioni esterne.
- **Authenticity (Autenticità):** È la garanzia che un oggetto digitale (o un utente) sia effettivamente ciò che dichiara di essere. In una comunicazione, assicura a chi riceve il messaggio che il mittente sia reale e non un truffatore mascherato.
- **Accountability (Responsabilità/Tracciabilità):** È la proprietà che permette di risalire in modo inequivocabile agli autori delle azioni. È il sistema di registrazione (log)

che permette di capire sempre "chi ha fatto cosa", fondamentale per le indagini post-attacco.

-
- **Availability (Disponibilità):** Garantisce che i sistemi funzionino in modo tempestivo e che l'accesso ai servizi non venga mai negato agli utenti legittimi. Un sistema super sicuro ma bloccato (ad esempio per un attacco informatico) fallisce questo obiettivo.
-

Nota:

“come si collegano tra loro?”

l'**Integrità** è il contenitore più grande. Al suo interno troviamo la **Autenticità** (la certezza di chi ha creato il dato) e, unendole insieme, otteniamo il *Non-Repudiation* (l'impossibilità di negare di aver inviato quel dato).

”Un Hash semplice garantisce l'Autenticità del dato?”

la risposta è **NO**. Un Hash garantisce solo che il file non è cambiato (Integrità). Ma siccome non usa alcuna chiave, chiunque può calcolare un nuovo Hash. Per avere l'Autenticità, devi "legare" quel dato a una persona usando una chiave segreta (MAC) o privata (Firma Digitale).

La Differenza: **Utente** vs. **Dato**

Il documento divide l'autenticazione in due categorie specifiche:

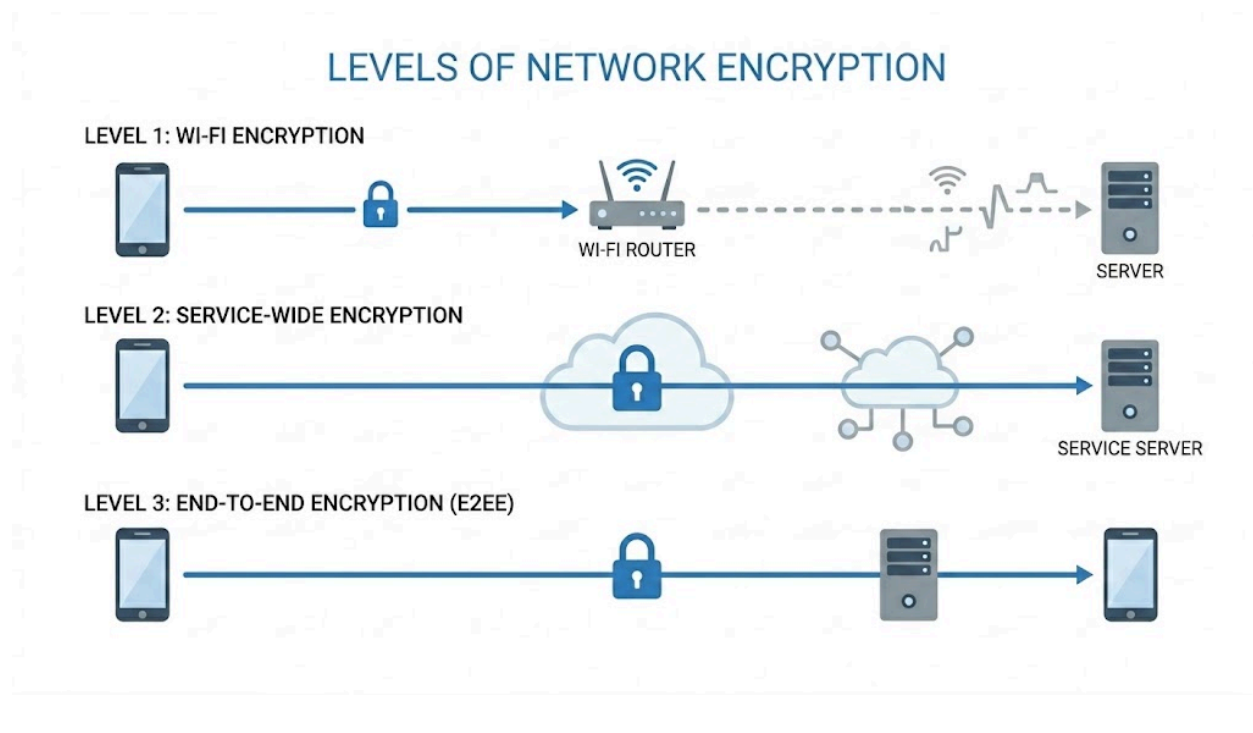
- **Peer Entity Authentication (Autenticità dell'Utente/Entità, con Scambio di autenticazione):** Riguarda una *connessione continua*. Garantisce che l'utente (o il server) dall'altra parte del filo sia davvero chi dice di essere per tutta la durata della sessione.
 - **Data Origin Authentication (Autenticità del Dato, con Mac/Firme Digitali):** Riguarda un *singolo messaggio*. Garantisce in modo inequivocabile la fonte (chi ha creato/inviato) di quel blocco di dati. Si usa per comunicazioni "sganciate" come le email, dove non c'è una sessione live in corso.
-

PLACEMENT OF ENCRYPTION:

l'applicazione della crittografia viene classificata in tre scenari crescenti che definiscono il nostro **perimetro di fiducia**. Si parte dalla base con la cifratura **Link-Level**, poi **End-to-End below Application Layer** e infine **Application-Layer End-to-End**.

Immaginando il percorso dal tuo dispositivo fino al destinatario finale, lo schema si traduce in modo molto intuitivo:

1. **Link-Level:** Il lucchetto protegge solo il primo "salto". Si chiude sul tuo telefono e si apre già al router Wi-Fi. Da lì in poi, i dati viaggiano nudi.
2. **Below Application (Es. HTTPS):** Il lucchetto rimane chiuso per tutto il viaggio attraverso Internet, proteggendoti da spie esterne, ma si apre non appena tocca il server centrale dell'azienda (che quindi può leggere tutto).
3. **Application-Layer (Es. WhatsApp):** Il lucchetto rimane chiuso da schermo a schermo. Nessun cavo, nessun router e nemmeno il server centrale possiede la chiave per aprirlo. Solo tu e il tuo interlocutore finale potete leggere il contenuto.



Message Authentication

Questo argomento introduce un concetto chiave:

l'Autenticazione (chi ha scritto il messaggio?) e la *Confidenzialità* (nascondere il contenuto) sono problemi distinti!

Come visto nella teoria, si utilizza una **PRF** (stateless e riproducibile) per generare il **MAC** (Message Authentication Code), un sigillo di pochi bit. Poi lo si spedisce insieme al messaggio e infine il destinatario ricalcola il MAC sul messaggio ricevuto. Se i due codici corrispondono, il messaggio è *integro e autentico*.

il MAC serve quindi ad aggiungere le proprietà di Integrity e Authentication permettendo al destinatario di verificare che il messaggio non sia stato alterato e provenga effettivamente dal mittente legittimo prima ancora di elaborarlo (protezione da Dos).

Funzioni di Hash e Message Authentication

Un Hash da solo garantisce solo l'integrità (se cambia un bit, cambia l'hash). Per garantire l'**Autenticazione**, l'hash deve essere "blindato" con una chiave. I tre modi:

1. **Cifratura Simmetrica:** Cifro l'hash con una chiave condivisa.
2. **Firma Digitale:** Cifro l'hash con la mia chiave privata.
3. **Secret Value:** Aggiungo un segreto S al messaggio (un segreto "condiviso" con l'altra parte) prima di frullare tutto nell'hash, così da rendere impossibile all'hacker di riprodurre un risultato accettato.

Proprietà funzione HASH:

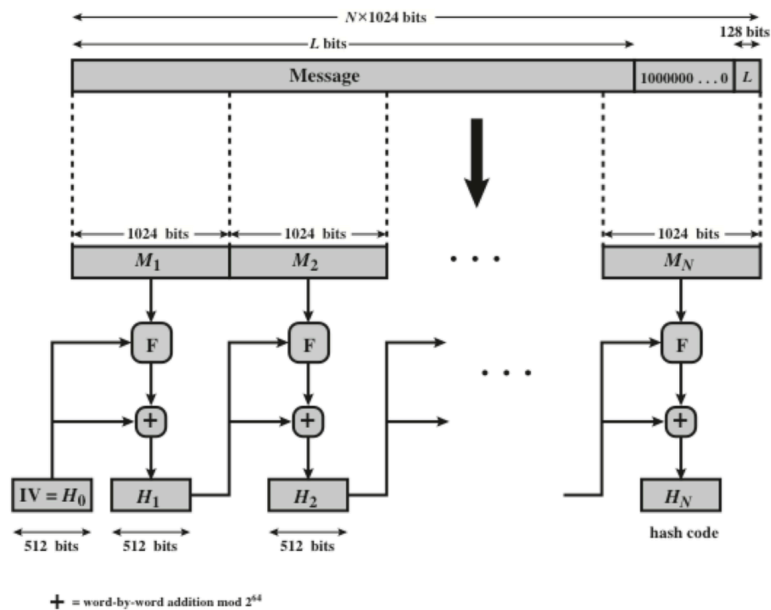
- **Unidirezionalità:** Impossibile tornare dall'hash al messaggio originale.
- **Resistenza seconda Pre-immagine (2^n tentativi):** Dato un messaggio, è impossibile trovarne un altro con lo stesso hash.
- **Resistenza Collisioni ($2^{n/2}$ tentativi):** È impossibile trovare *due messaggi qualsiasi* con lo stesso hash.
Lo sforzo è minore ($2^{n/2}$) per il **Paradosso del Compleanno** (è più facile che due persone a caso compiano gli anni lo stesso giorno piuttosto che uno colpisca

proprio il tuo compleanno). SHA256 è lo standard oggi, Lo SHA-512 invece non è solo "più sicuro", ma è spesso più veloce sui processori moderni a 64 bit rispetto allo SHA-256.

Table 3-1: Comparison of SHA Parameters

	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Message Digest Size	160	224	256	384	512
Message Size	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
Block Size	512	512	512	1024	1024
Word Size	32	32	32	64	64
Number of Steps	80	64	64	80	80

SHA-512



Per capire come lavora lo SHA-512, facciamo un esempio:

- **Messaggio:** "54321" → **Padding:** [54] [32] [10] [05] (l'ultimo blocco indica la lunghezza originale).
- **IV (Seme iniziale):** 100

Il processo iterativo:

1. $100 \text{ (IV)} + 54 = 154$
2. $154 + 32 = 186$
3. $186 + 10 = 196$
4. $196 + 05 = 201 \rightarrow \text{Digest Finale}$

Commenti Finali:

- Se cambi anche solo il primo numero ($54 \rightarrow 55$), l'errore si propaga come un'onda, cambiando completamente il 201 finale.
 - Senza il padding e l'indicazione della lunghezza finale (05), un hacker potrebbe aggiungere degli zeri in fondo al messaggio senza che l'hash cambi (attacco di estensione). Inserire la lunghezza "sigilla" il file.
 - **Dalla Somma alla Sicurezza:** Ovviamente lo SHA-512 reale non fa semplici somme, ma usa operazioni logiche (XOR, rotazioni di bit) estremamente complesse all'interno della funzione F.
-

Sistemi di Autenticazione: HMAC, CMAC e CCM

L'obiettivo di queste sezioni è spiegare come trasformare funzioni nate per altri scopi (come l'Hash o l'AES) in strumenti per garantire l'**integrità** e l'**autenticità** dei messaggi.

1. HMAC (Hash-based Message Authentication Code)

L'HMAC nasce dall'esigenza di creare un codice di autenticazione (MAC) partendo da funzioni di hash esistenti (come lo SHA-1 o lo SHA-256).

- **Perché non usare l'Hash puro?** Le funzioni di hash standard non usano chiavi segrete; chiunque può calcolare l'hash di un messaggio. Per autenticare, serve incorporare una chiave segreta nel calcolo.
- **Perché usare l'Hash invece della cifratura?** Le funzioni di hash sono generalmente più veloci in software rispetto ad algoritmi di cifratura convenzionali come il DES.

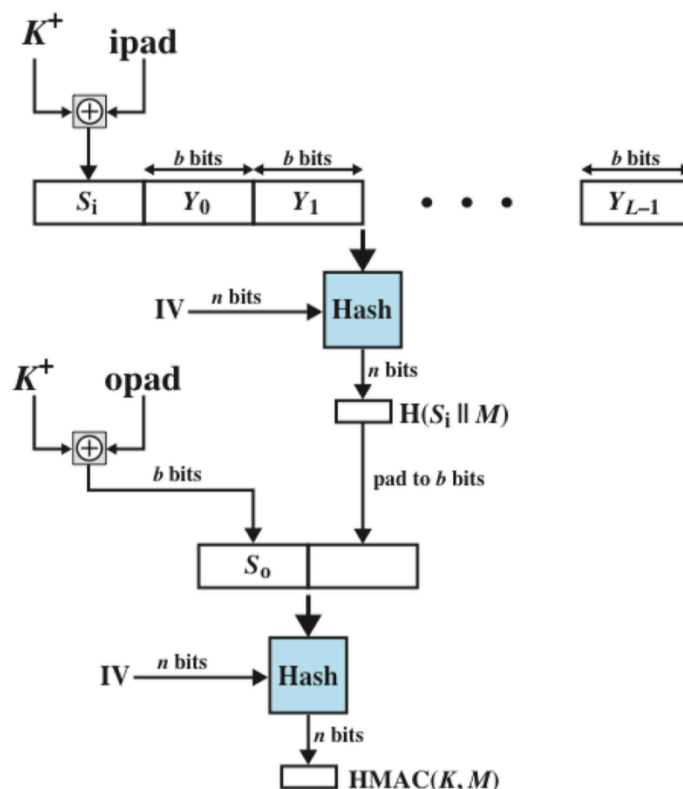
LHMAC non si limita ad aggiungere la chiave al messaggio, ma esegue due passaggi di hash per garantire massima sicurezza:

1. **Passaggio Interno:** La chiave K viene combinata con una stringa fissa chiamata **ipad** (inner pad) tramite XOR e poi concatenata al messaggio M . Di tutto questo si calcola il primo Hash.

2. **Passaggio Esterno:** Il risultato del primo hash viene combinato con la chiave XORata con un'altra stringa, l'**opad** (outer pad). Si calcola l'hash finale.

Formula: $HMAC(K, M) = H[(K^+ \oplus opad) \parallel H[(K^+ \oplus ipad) \parallel [cite_{start} M]]]$

HMAC Structure

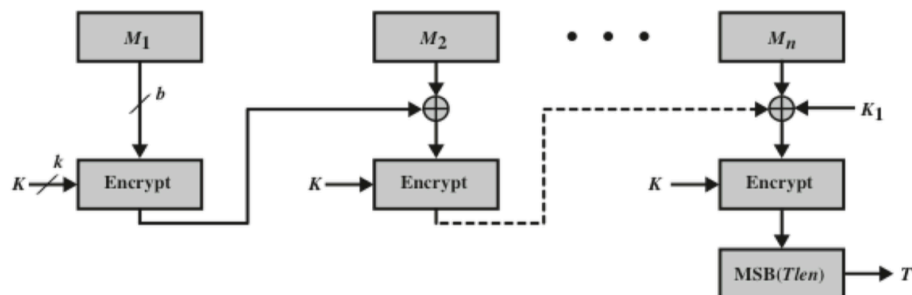


2. CMAC (Cipher-based MAC)

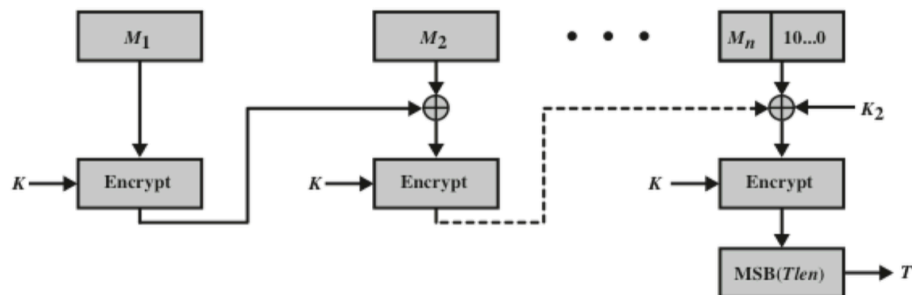
Se non vogliamo usare funzioni di hash, possiamo usare algoritmi di cifratura a blocchi (come AES o **Triple DES**) per generare un MAC.

CMAC: Garantisce esclusivamente **Integrity** e **Authenticity**. Si usa quando la **Confidentiality** non è richiesta.

- **Come funziona:** Il messaggio viene diviso in blocchi. Ogni blocco viene cifrato con la chiave K e il risultato viene combinato (XOR) con il blocco successivo.



(a) Message length is integer multiple of block size



(b) Message length is not integer multiple of block size

3. CCM (Counter with CBC-MAC)

È una modalità di *Authenticated Encryption* che garantisce simultaneamente **Integrity**, **Authenticity** e **Confidentiality**. Utilizza una singola chiave per proteggere tutti e tre gli aspetti, rendendolo ideale per sistemi con risorse limitate.

Unisce tre componenti basati sull'AES:

1. **AES:** Come algoritmo motore di base.

2. **CTR mode**: Per la cifratura del messaggio.

3. **CMAC**: Per l'autenticazione del messaggio.

RIASSUMENDO: HMAC vs CMAC vs CCM:

Tutti e tre servono a garantire che nessuno tocchi i tuoi dati, ma cambia il "motore" che usano.

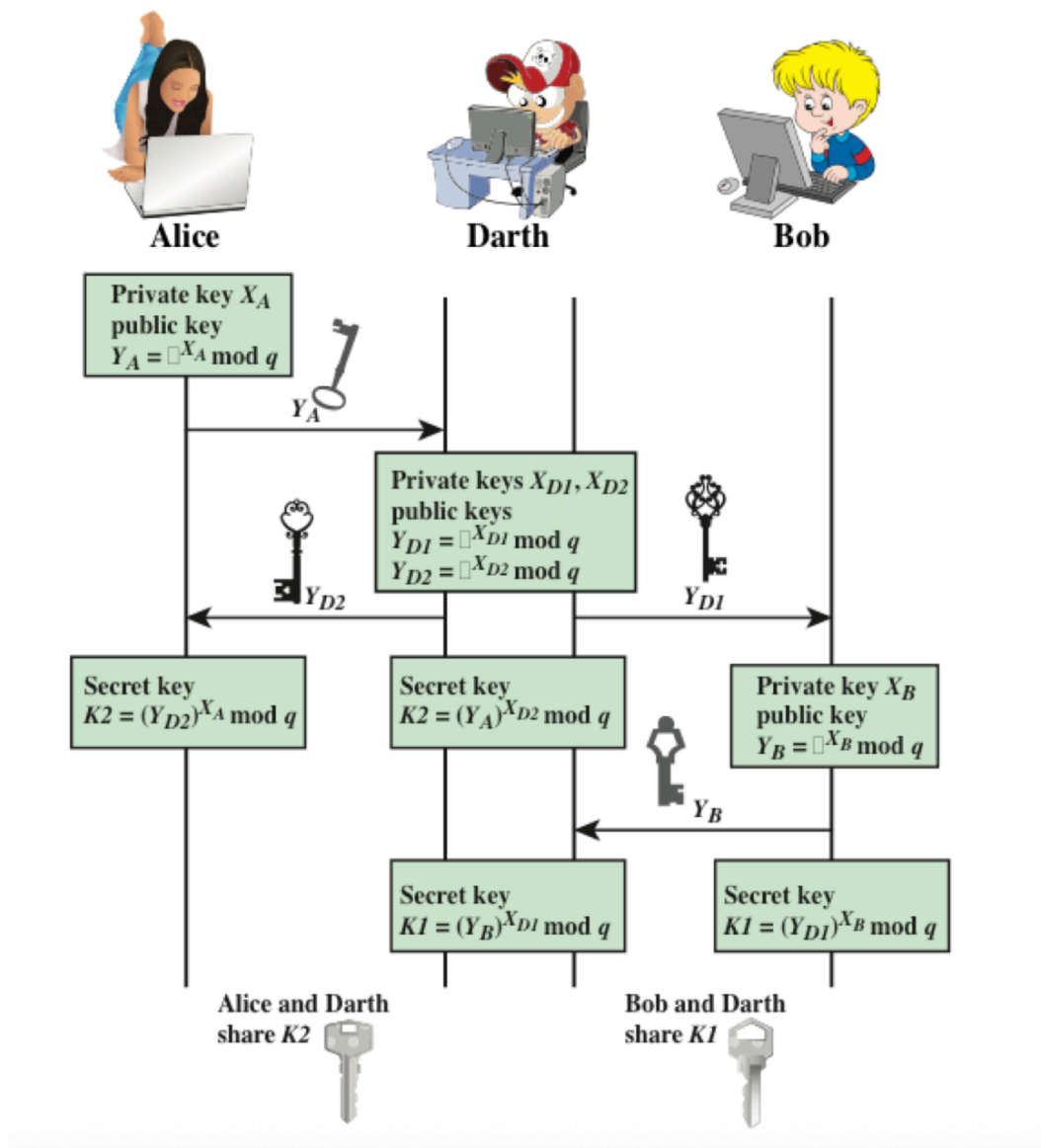
- **HMAC (Hash-based MAC)**: Garantisce **Integrity e Authenticity**.
 - **Confidentiality: NO** (il messaggio è in chiaro).
 - **Differenza**: Usa funzioni di **Hash** (es. SHA-256). Lo usi se la tua ESP32 non ha accelerazione AES o se il server preferisce gli Hash per velocità software.
 - **CMAC (Cipher-based MAC)**: Garantisce **Integrity e Authenticity**.
 - **Confidentiality: NO** (il messaggio resta leggibile).
 - **Differenza**: Usa un algoritmo di **Cifratura a blocchi** (es. AES) ma "solo per creare il sigillo".
 - **CCM (Counter with CBC-MAC)**: Garantisce **Integrity, Authenticity E Confidentiality**.
 - **Differenza**: È l'unico dei tre che **nasconde anche il dato**. Usa una singola chiave per fare tutto, unendo la modalità CTR (per cifrare) e il CMAC (per autenticare).
-

Applicazioni per sistemi di crittografia asimmetrica:

Algorithm	Encryption/Decryption	Digital Signature	Key Exchange
RSA	Yes	Yes	Yes
Diffie-Hellman	No	No	Yes
DSS	No	Yes	No
Elliptic Curve	Yes	Yes	Yes

Problema di sicurezza di Diffie-Hellman:

il protocollo permette di scambiare in modo sicuro una chiave su internet, ma ha un problema di autenticazione. Se io scambio la chiave senza avere la certezza di chi ho dall'altra parte, mi espongo ad un Man in the Middle Attack:



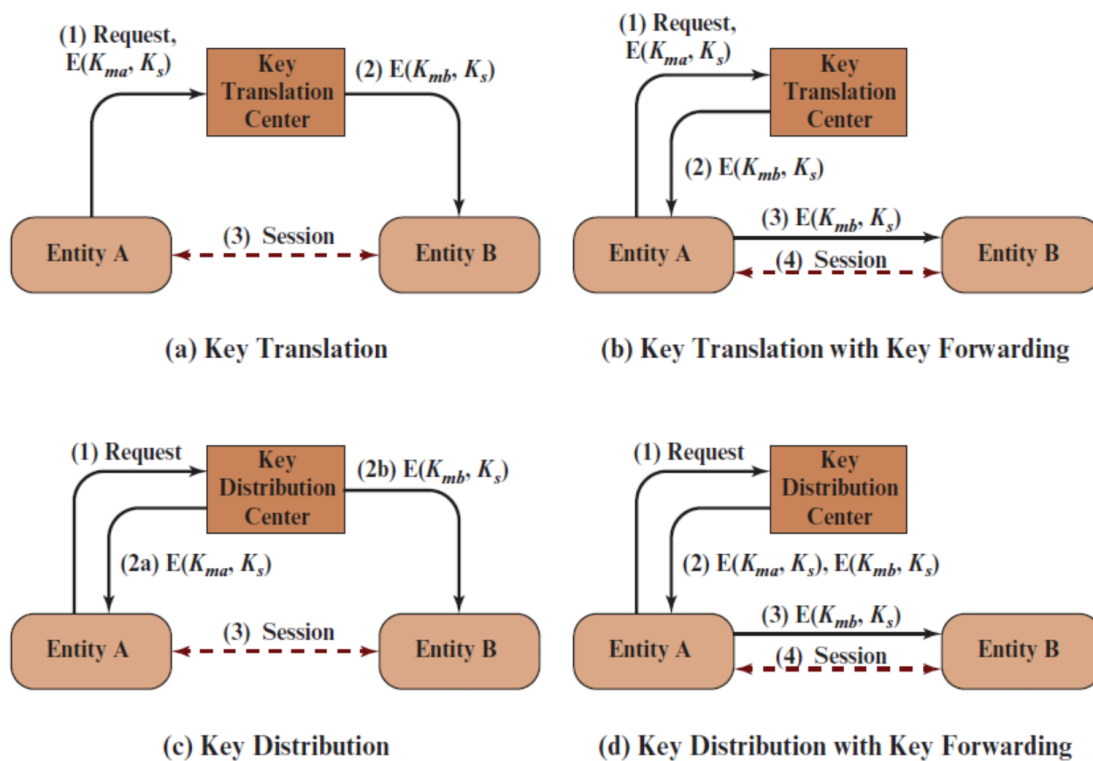
*Il problema fondamentale è che Alice e Bob **pensano** di parlare tra loro, ma in realtà stanno entrambi parlando con Darth senza saperlo.*

Gestione e Distribuzione delle Chiavi Crittografiche

Il “*Cryptographic key management*” è il processo completo dalla generazione, alla custodia e monitoraggio accessi delle chiavi.

Symmetric Key Distribution

per la distribuzione tramite chiavi simmetriche abbiamo 4 modi:

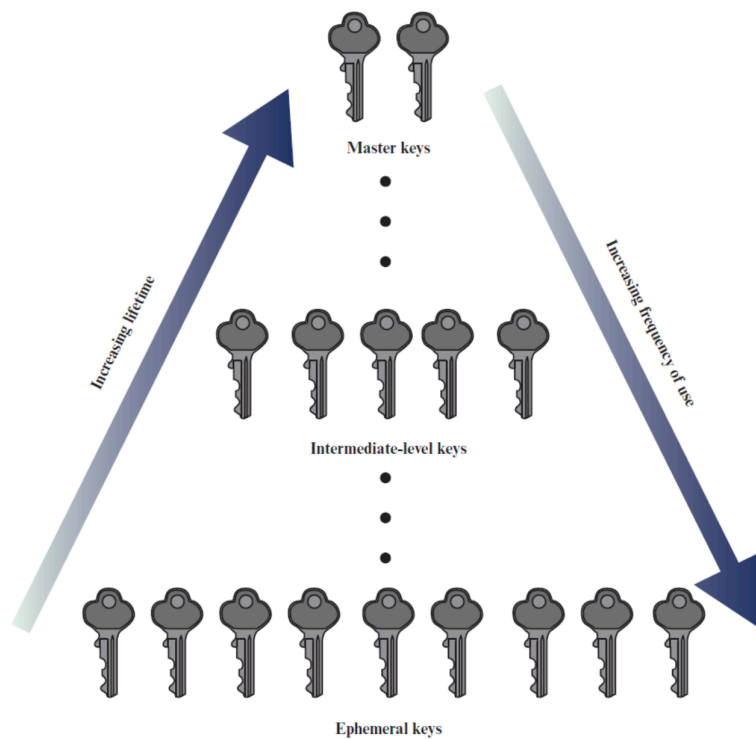


Gerarchia delle Chiavi Simmetriche

- **Master keys:** chiavi a lungo termine, servono a cifrare e proteggere le chiavi inferiori
- **Intermediate-level keys:** Servono a **compartimentare il rischio**. Invece di usare una sola Master Key per tutto, si usano chiavi intermedie per proteggere gruppi

specifici di chiavi di sessione.

- **Ephemeral Keys:** Sono le chiavi "operaie". Il loro unico compito è cifrare i dati effettivi (il payload). Sono chiamate "effimere" perché vengono generate per una singola sessione e distrutte subito dopo, riducendo al minimo la quantità di dati che un attaccante potrebbe decifrare se riuscisse a rubarne una.



Nonce e Timestamp

- **NONCE (Unicità):** Numero casuale "usa e getta" che rende ogni messaggio matematicamente **unico**; serve a bloccare i **Replay Attack** impedendo a un hacker di rimettere in circolo vecchi comandi intercettati.
- **TIMESTAMP (Scadenza):** Record temporale che garantisce la **freschezza** del dato; permette al server di scartare messaggi "vecchi", limitando il tempo in cui un messaggio intercettato rimane valido.

- **NONCE IBRIDO (Combo):** Unisce Tempo + Numero casuale per ottenere scadenza certa (grazie al tempo) e l'unicità assoluta (grazie al numero), eliminando il rischio di collisioni se invii più pacchetti nello stesso istante.
-

In breve: Il **Nonce** dice "questo è l'unico", il **Timestamp** dice "questo è fresco". Usati insieme per non dover salvare miliardi di numeri in memoria sul server (dopo 5 secondi il server li cancella e "pulisce" la cache).

DISTRIBUZIONE CHIAVI con NEEDHAM SCHROEDER

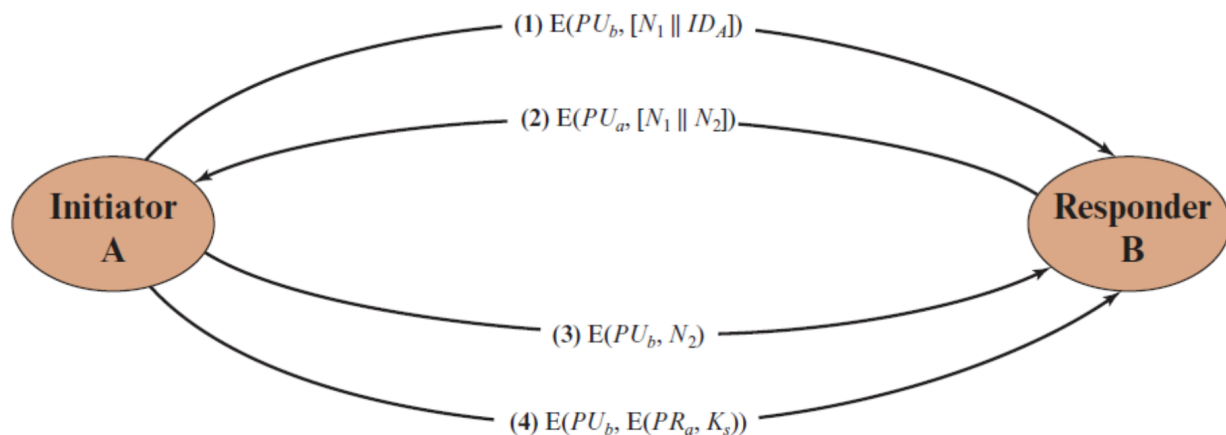
Needham-schroeder è una famiglia di protocolli pensati per autenticare e scambiarsi una chiave simmetrica in modo sicuro tra 2 persone.

- **SIMMETRICO (Chiave Segreta):** Alice e Bob usano una chiave uguale ("breve"). Per trovarsi hanno bisogno di un mediatore (il **KDC**)("completo").
- **ASIMMETRICO (Chiave Pubblica):** Alice e Bob usano coppie di chiavi (pubblica/privata). Hanno bisogno di un notaio che garantisca l'identità (la **CA** o **PKA**)("completo").

TUTTE ATTACCABILI DA MAN IN THE MIDDLE (Va aggiunto il FIX di LOWE)!!

Needham-Schroeder Public Key (NSPK) (asimmetrico "breve")

per la distribuzione tramite chiavi asimmetriche, proposto nel 1978, è il protocollo a chiave pubblica di Needham Schoreder. Serve a verificare che "siamo davvero noi due" e a scambiare una chiave segreta per parlare in modo protetto.

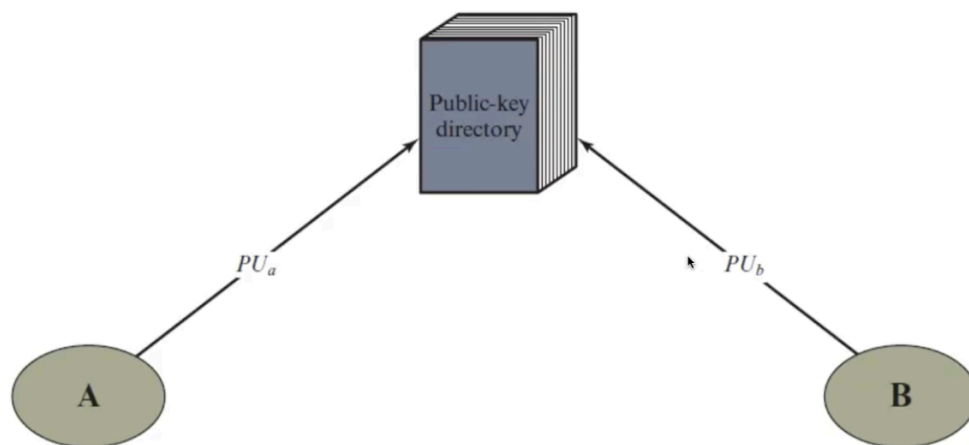


non è più utilizzato direttamente nelle applicazioni moderne a causa di una vulnerabilità all'attacco man-in-the-middle, identificata successivamente.

Quindi il problema che sorge spontaneo da tutti questi attacchi man-in-the-middle è il seguente:

"come facciamo con tutte queste chiavi pubbliche in giro a fidarci che siano vere?"

Certification Authority (CA)

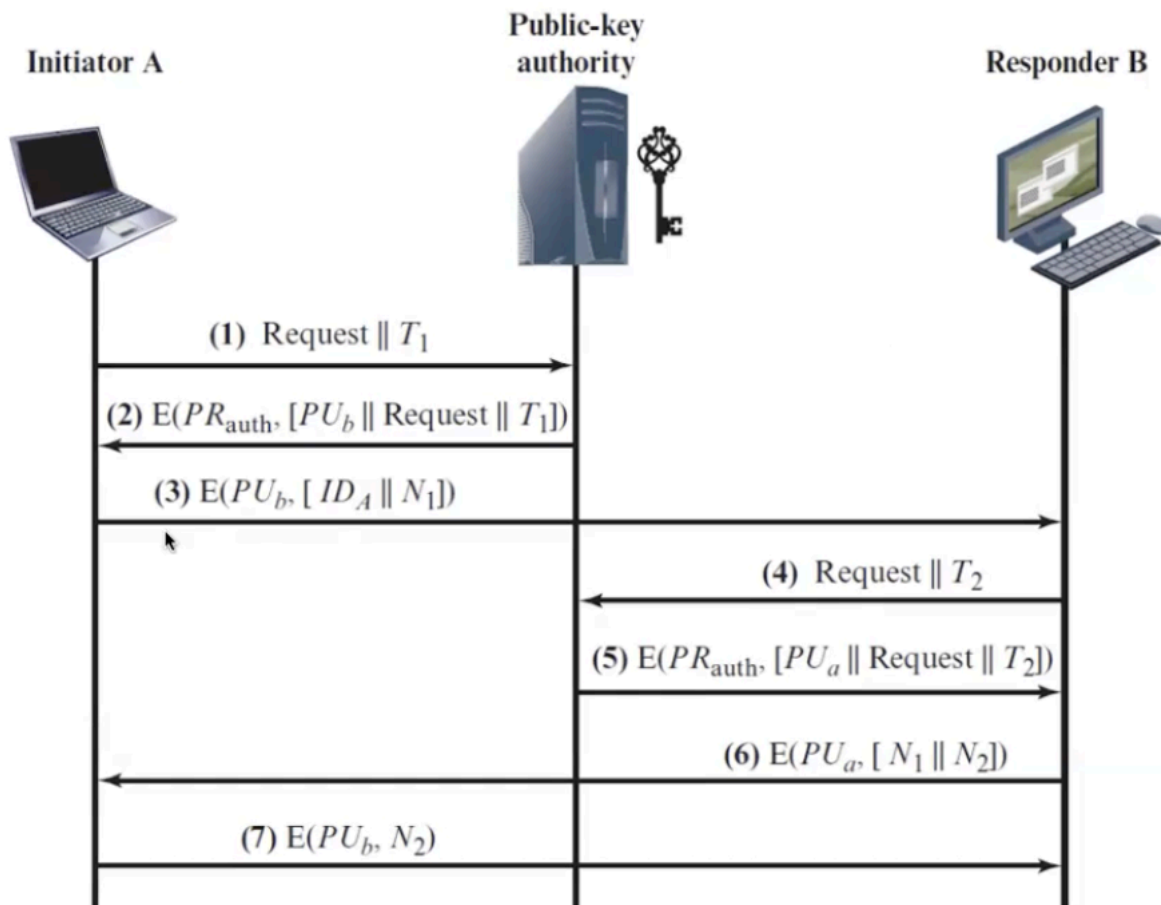


L'idea è quella di affidare a delle strutture pubbliche fidate, la gestione dei registri delle chiavi pubbliche (public-key directory). Ovviamente il loro "fallimento" è possibile e in quel caso scattano delle contromisure di revoca di tutti i certificati emessi da quella CA.

Needham-Schroeder Public Key (NSPK) - con CA (**asimmetrico** **"completo"**)

Il protocollo assume semplicemente che Alice conosca la chiave pubblica di Bob e viceversa. **Come** le ottengano (se tramite una CA, un database pubblico o uno scambio a mano) non è parte del protocollo Needham-Schroeder in sé.

Questo esempio mostra l'utilizzo della CA per ottenere le identità delle due parti.

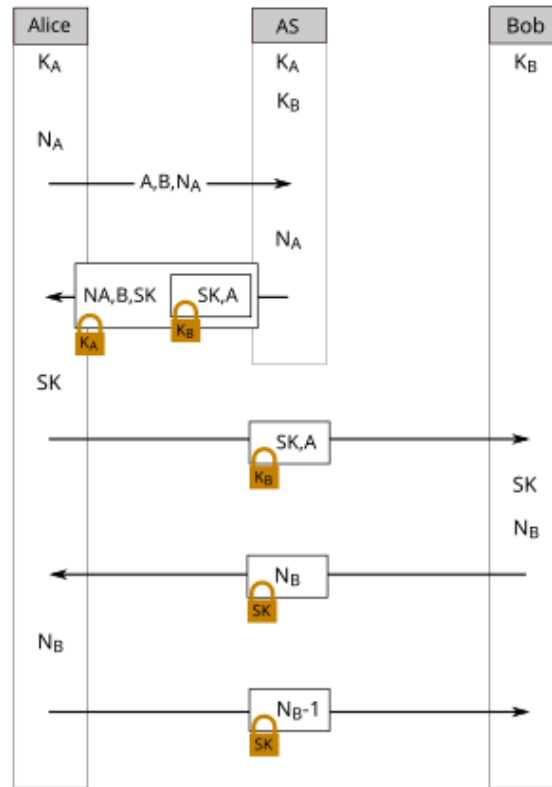


NEEDHAM-SCHROEDER a Chiavi SIMMETRICHE (**"simmetrico** **"completo"**)

Il *protocollo di Needham-Schroeder a chiave segreta* è basato sulla crittografia simmetrica ed è alla base del protocollo di rete Kerberos.

È un protocollo teorico che permette a due utenti (Alice e Bob) di autenticarsi a vicenda

e scambiarsi una chiave di sessione in modo sicuro, utilizzando esclusivamente la crittografia simmetrica e un server di fiducia chiamato **KDC (Key Distribution Center)**.



K_A = chiave condivisa tra Server fidato e Alice

K_B = chiave condivisa tra Server fidato e Bob

SK = chiave simmetrica generata per Alice e Bob (K_{AB})

N_A

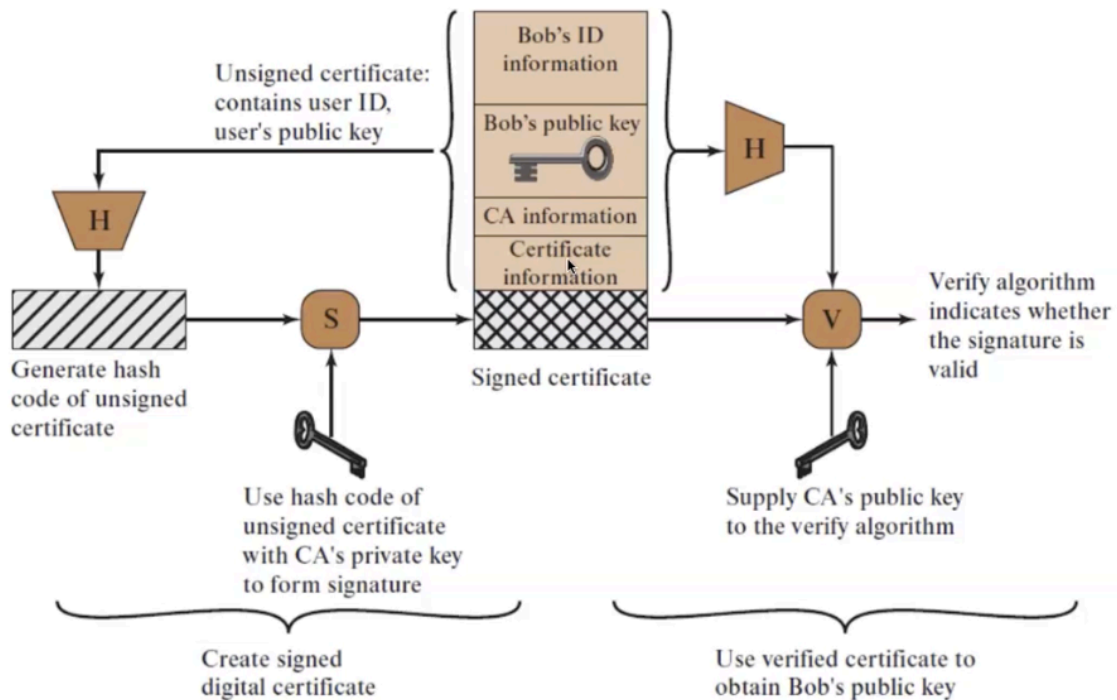
LISTA ALGORITMI A CHIAVE ASIMMETRICA:

- **RSA**: È l'algoritmo raccomandato per l'uso all'interno del framework dei certificati X.509.
- **Diffie-Hellman**: Utilizzato per lo scambio sicuro di chiavi su canali non protetti.
- **DSS (Digital Signature Standard)**: Documento standard per le firme digitali.

- Needham-Schroeder: sia a chiavi simmetriche che asimmetriche.

X.509 CERTIFICATES

Certificate Use



Il certificato viaggia come un pacchetto composto dai **dati in chiaro** e dall'**hash firmato** dalla CA. Il ricevente non deve fare altro che ricalcolare l'hash dei dati e confrontarlo con quello estratto dalla firma usando la **chiave pubblica della CA**.

Se i due hash sono identici, hai la prova matematica che:

- **Integrità**: Nessuno ha cambiato un singolo bit dei dati.
- **Autenticità**: Solo la CA può aver generato quella firma con la sua chiave privata.

$$\text{Verifica} \implies H(\text{DatiRicevuti}) \stackrel{?}{=} K_{PU_{CA}}(\text{Firma})$$

versione esaustiva:

$$K_{PU_{CA}}(K_{PR_{CA}}(H(Dati))) \stackrel{?}{=} H(DatiRicevuti)$$

dove $K_{PU_{CA}}$ è la chiave pubblica della CA e $K_{PR_{CA}}$ la chiave privata.

Contenuto Certificati

- Version
- Serial number
- Signature algorithm identifier
- Issuer name
- Period of validity
- Subject name
- Subject's public-key information
- Issuer unique identifier
- Subject unique identifier
- Extensions
- Signature

note:

- **Version:** Indica lo standard del formato (es. v3), definendo quali campi possono essere presenti.
 - **Serial number:** Numero univoco assegnato dalla CA per identificare e, se necessario, revocare quel singolo certificato.
 - **Signature algorithm identifier:** Specifica la "ricetta" usata (es. SHA-256 con RSA) per creare la firma digitale.
 - **Issuer name:** Il nome ufficiale dell'Autorità di Certificazione (CA) che ha emesso e garantito il documento.
 - **Period of validity:** Le date di "inizio" e "fine" oltre le quali il certificato non è più considerato affidabile.
 - **Subject name:** L'identità del proprietario del certificato (persona, server o azienda).
 - **Subject's public-key information:** La chiave pubblica del proprietario e l'algoritmo necessario per utilizzarla correttamente.
 - **Issuer Unique Identifier:** Bit string opzionale utilizzata per risolvere ambiguità nel caso in cui diverse CA condividano lo stesso nome (X.500 Distinguished Name).
 - **Subject Unique Identifier:** Bit string opzionale impiegata per identificare univocamente il titolare della chiave qualora esistano più soggetti con nomi identici nel sistema
 - **Extensions:** Campi aggiuntivi che specificano permessi extra, come l'uso della chiave per la cifratura o per le email.
 - **Signature:** Il sigillo finale della CA, creato cifrando l'hash di tutti i campi precedenti con la propria chiave privata.
-

Versione 3 di X.509 Certificates

La v3 si distingue specificatamente per l'introduzione e l'uso massiccio del campo Extensions, permette di aggiungere informazioni personalizzate senza dover cambiare lo standard ogni volta.

Le estensioni della v3 servono a coprire tre aree principali che mancavano nelle versioni precedenti:

1. **Informazioni sulla chiave e policy:** Ad esempio, specifica se la chiave serve per la firma digitale o per la cifratura (Key Usage).
2. **Attributi del soggetto e dell'issuer:** Permette di inserire nomi alternativi (come indirizzi email o DNS) oltre al nome standard.
3. **Path Constraints:** Serve a limitare i tipi di certificati che una CA subordinata può emettere.

Indicatori di criticità: Ogni estensione nella v3 include un flag di "criticality". Se un'estensione è segnata come critica e il software che legge il certificato non la riconosce, deve rifiutare l'intero certificato per sicurezza.

Certification Path Constraints

Queste estensioni sono fondamentali quando una CA ne certifica un'altra (gerarchia), perché permettono di stabilire cosa può o non può fare la CA subordinata.

- **Basic Constraints (Vincoli di base):** È il campo più importante; specifica se il soggetto del certificato è una **CA** (e quindi può emettere altri certificati) o un **utente finale** (che non può firmare nulla).
- **Name Constraints (Vincoli sul nome):** Permette di limitare lo "spazio dei nomi" per cui una CA subordinata è valida. Ad esempio, una CA aziendale può essere autorizzata a firmare solo certificati che contengono il dominio `@azienda.it`.
- **Policy Constraints (Vincoli sulle policy):** Impone che certi passaggi della catena di fiducia seguano regole specifiche, come l'identificazione esplicita delle policy di certificazione.

Quando si riceve un certificato, non basta controllare che la firma sia valida e che non sia scaduto. Bisogna anche verificare che non sia presente nella CRL (Certificate Revocation List)

REVOCA DEI CERTIFICATI

La revoca dei certificati è il processo necessario per annullare la validità di un certificato prima della sua scadenza naturale.

Perché si revoca un certificato?

Anche se ogni certificato ha un suo **periodo di validità** predefinito , a volte è necessario "disattivarlo" prima del tempo per motivi di sicurezza:

- **Chiave compromessa:** Si sospetta che la chiave privata dell'utente sia stata rubata o scoperta.
 - **Fine del rapporto:** L'utente non è più certificato da quella specifica CA (ad esempio, ha lasciato l'azienda).
 - **CA compromessa:** Se la chiave della stessa Autorità di Certificazione è compromessa, tutti i certificati da lei emessi perdono affidabilità.
-

CRL (Certificate Revocation List)

Per gestire queste emergenze, ogni CA deve mantenere una **CRL**, ovvero una "lista di proscrizione" che contiene tutti i certificati revocati ma non ancora scaduti.

ontiene informazioni cruciali:

- **Issuer Name:** Il nome della CA che ha emesso la lista.
 - **This update date / Next update date:** Indicano quando è stata pubblicata la lista attuale e quando verrà rilasciato il prossimo aggiornamento.
 - **Revoked Certificate:** Per ogni certificato revocato viene riportato il **Serial Number** (numero di serie univoco) e la **data di revoca**.
 - **Signature:** La lista intera è firmata digitalmente dalla CA per garantirne l'autenticità.
-

TRUST MODELS

Il Modello di Trust (Fiducia)

Il **Trust** non è un algoritmo, ma la base politica e organizzativa della sicurezza. Rappresenta la disponibilità di un'organizzazione a essere vulnerabile verso un'altra parte (es. un fornitore Cloud o un dipendente), con l'aspettativa che questa si comporti secondo le policy stabilite, anche senza un controllo costante.

I 3 Pilastri del Trust

1. **Affidabilità** (**Trustworthiness**): Il grado in cui un sistema o un individuo merita fiducia.
 2. **Propensione al Trust**: Il livello base di fiducia che un'organizzazione decide di concedere a priori.
 3. **Rischio**: Il calcolo tra l'impatto di un fallimento della sicurezza e la sua probabilità.
-

Trust nelle Persone e nei Sistemi

- **Individuale**: Si stabilisce tramite policy HR e programmi di formazione continua.
 - **Sistemico**: Un sistema è considerato "trustworthy" se garantisce i requisiti di **Confidentiality, Integrity e Availability** contro ogni minaccia.
 - **Relazioni Esterne**: La fiducia tra organizzazioni viene formalizzata tramite contratti, accordi di livello di servizio (SLA) o garanzie fornite da terze parti (es. certificazioni).
-

"Perché serve il Trust se abbiamo la crittografia?", la risposta è che la crittografia protegge i dati, ma il Trust gestisce il **rischio umano e organizzativo** che la tecnologia da sola non può risolvere.

Modelli di Trust (Gerarchia dei Certificati)

Il problema principale è: come faccio a fidarmi di un certificato se non conosco direttamente la CA che lo ha emesso? Si risolve con una catena di fiducia.

i modelli di trust principali sono 3:

1) Direct Trust (Fiducia Diretta)

- **Concetto:** Si verifica quando tutti gli utenti sono iscritti alla **stessa CA**.
- **Funzionamento:** Poiché esiste una CA comune di cui tutti si fidano, si stabilisce una fiducia reciproca attraverso di essa.
- **Distribuzione:** I certificati possono essere messi in una directory pubblica accessibile a tutti o inviati direttamente da utente a utente.

2) Hierarchical Trust (Fiducia Gerarchica)

- **Concetto:** La fiducia è strutturata ad **albero**. Non serve conoscere ogni singola CA: basta fidarsi della **Root CA** (il vertice) che garantisce per tutte le altre sotto di lei.
- **Funzionamento:** La **Root CA** (ancora di fiducia) certifica le **CA Subordinate**, che possono a loro volta certificare altri livelli (CA intermedie) fino all'utente finale (foglia). La fiducia passa "dall'alto verso il basso".
- **Verifica:** Per convalidare un certificato, il sistema ripercorre la **catena di certificazione** all'indietro (risalendo i rami dell'albero) finché non trova una Root CA già installata e fidata nel computer.
- **Cross-Certification:** È un "ponte" tra alberi diversi. Due Root CA si firmano i certificati a vicenda, permettendo ai rispettivi utenti di fidarsi l'uno dell'altro anche se appartengono a gerarchie differenti.

Questo è il formato usato da X.509, di seguito un esempio usando la notazione $X \ll Y \gg$:

scenario di esempio: Alice verifica Bob

Alice è un'utente della gerarchia che fa capo alla **CA X** (la sua "ancora di fiducia"), mentre Bob è un utente certificato dalla **CA Z**. Alice non conosce direttamente la CA di Bob, ma può verificarlo risalendo una catena di fiducia attraverso dei "ponti" (cross-certification) tra le autorità.

La Catena di Certificazione

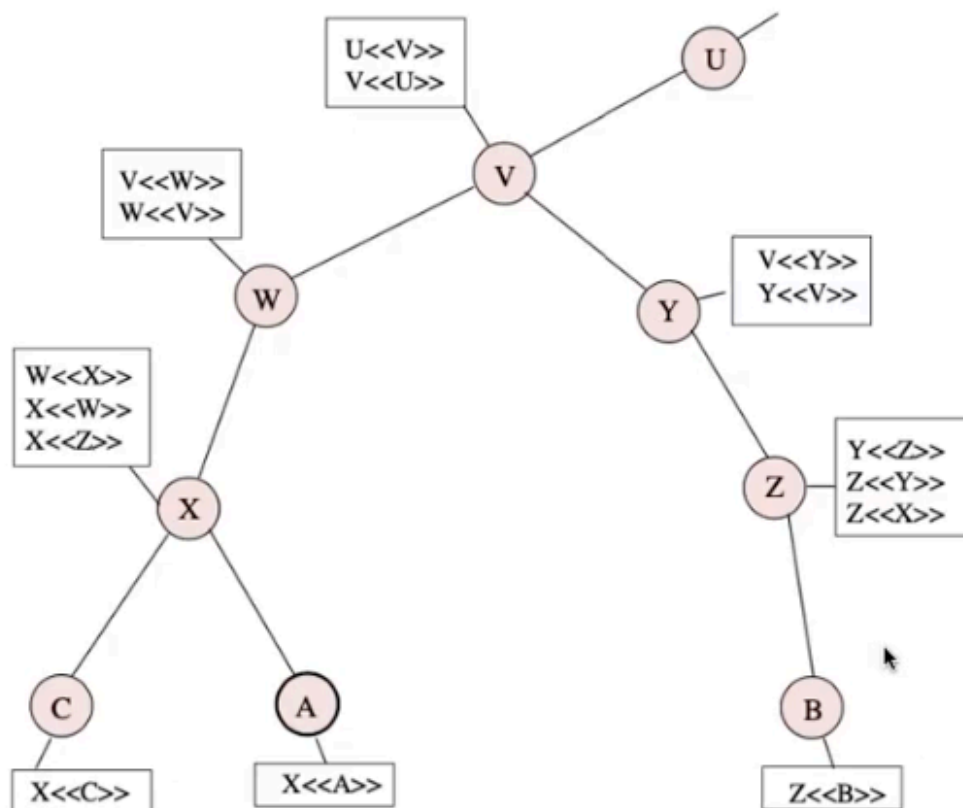
Per ottenere la chiave pubblica di Bob, Alice deve "srotolare" questa sequenza di certificati:

$$X \ll W \gg \quad W \ll V \gg \quad V \ll Y \gg \quad Y \ll Z \gg \quad Z \ll B \gg$$

Spiegazione dei passaggi:

- $X \ll W \gg$: La CA X (fidata da Alice) emette un certificato per la CA W . Questo è un **Reverse Certificate** (certificato di un'altra CA generato da X).
- $W \ll V \gg$: La CA W certifica la CA V . Alice ora si fida di V perché si fida di W .
- $V \ll Y \gg$ **Cross-Certification** fondamentale. Anche se sono su rami diversi, le CA hanno creato certificati l'una per l'altra (spesso scambiandosi le chiavi pubbliche in una directory).
- $Y \ll Z \gg$: La CA Y certifica la CA Z (quella che ha emesso il certificato di Bob).
- $Z \ll B \gg$: Infine, la CA Z certifica l'utente finale **Bob**.

Alice deve **risalire** l'albero verso la Root usando i **Reverse**, Poi deve **scendere** verso Bob usando i **Forward**.



Conclusione:

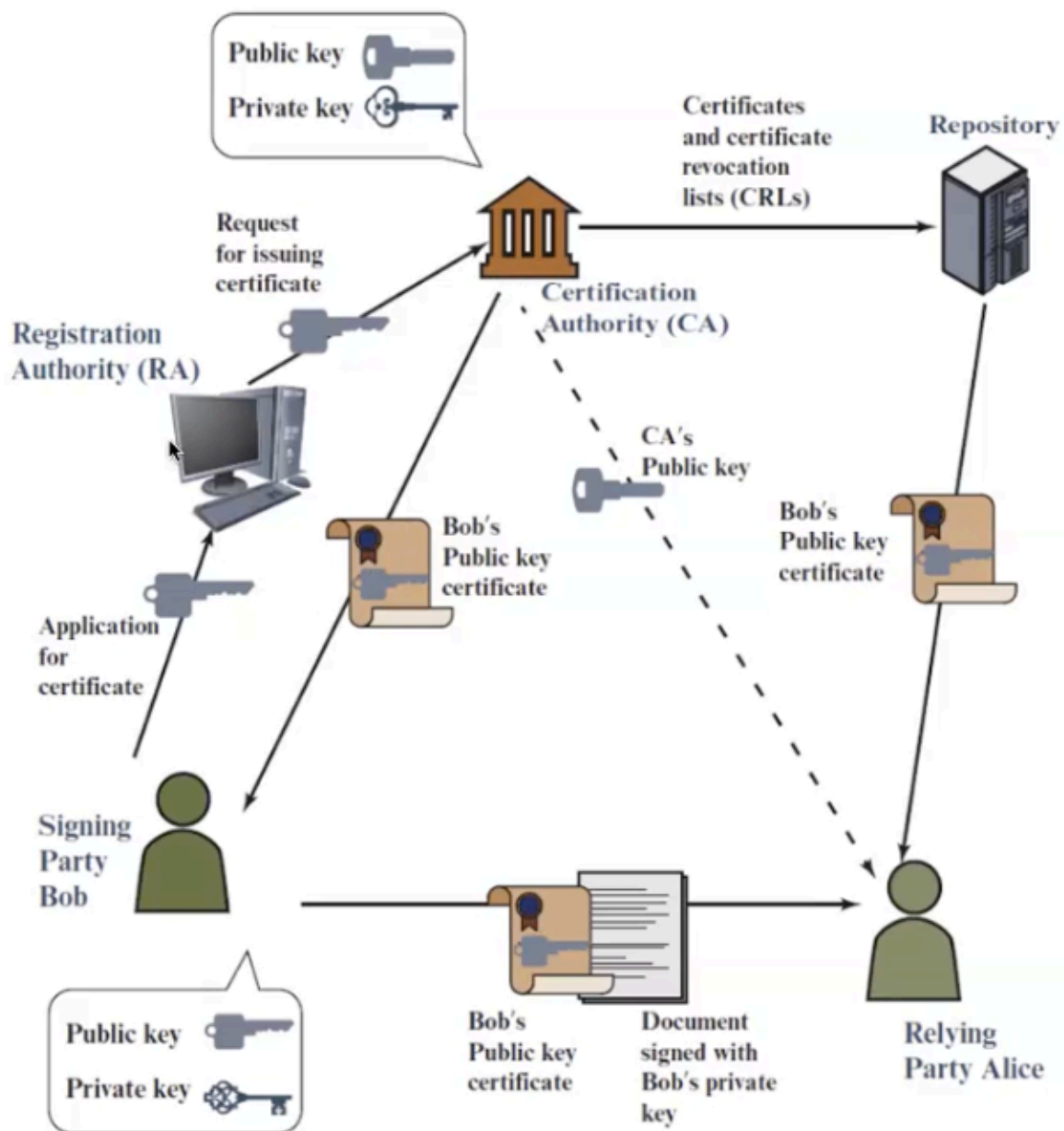
Grazie a questa catena, Alice può verificare Bob partendo dalla sua Root (X) anche se

non appartengono allo stesso albero. Allo stesso modo, Bob potrebbe verificare Alice seguendo la catena inversa:

$$Z \ll Y \gg \quad Y \ll V \gg \quad V \ll W \gg \quad W \ll X \gg \quad X \ll A \gg$$

PKI (Public-Key Infrastructure)

ovvero l'insieme di componenti e procedure che permettono di gestire in modo sicuro le chiavi pubbliche e l'identità degli utenti.



Ecco i protagonisti e i passaggi fondamentali del diagramma:

1. I Protagonisti

- **Bob (Signing Party):** L'utente che vuole ottenere un certificato per la sua chiave pubblica.
 - **Registration Authority (RA):** L'intermediario che riceve la richiesta di Bob, ne verifica l'identità e la gira alla CA.
 - **Certification Authority (CA):** L'ente fidato che emette e firma digitalmente il certificato di Bob.
 - **Repository:** Un database pubblico dove vengono conservati i certificati e le liste di revoca (CRL).
 - **Alice (Relying Party):** L'utente che riceve un documento da Bob e deve verificarne l'autenticità usando il certificato.
-

2. Il Ciclo di Vita (I Passaggi)

1. **Richiesta:** Bob invia una domanda di certificazione alla **RA**.
2. **Emissione:** Una volta approvata, la **CA** crea il certificato e lo invia a Bob.
3. **Pubblicazione:** La CA invia il certificato (e le liste di revoca) al **Repository** (**per la struttura ad albero di prima, non serve arrivare al Repository**).
4. **Utilizzo:** Bob firma un documento con la sua **chiave privata** e lo invia ad Alice insieme al suo certificato.
5. **Verifica:** Alice riceve il pacchetto e usa la **chiave pubblica della CA** (che già possiede) per verificare che il certificato di Bob sia autentico. Inoltre, può consultare il Repository per assicurarsi che il certificato non sia stato revocato.

In sintesi per gli appunti:

È il "ecosistema" che permette ad Alice di fidarsi di Bob anche se non lo ha mai incontrato, basandosi sulla fiducia comune verso l'Autorità di Certificazione (CA).

3) Web of Trust (Rete di Fiducia)

- **Concetto:** Tipico di **PGP** che rappresenta un altro meccanismo di certificazione di chiavi che ha una struttura diversa rispetto a **X.509**, unisce elementi degli altri modelli ma aggiunge l'idea che la fiducia sia soggettiva ("negli occhi di chi guarda").
- **Funzionamento:** Non serve un'infrastruttura centrale (No CA). Ogni utente può agire come autorità di certificazione, firmando le chiavi delle persone che conosce.
- **Livelli di Trust:** Ogni firma porta con sé un livello di fiducia (es. parziale, totale, sconosciuto) che indica quanto l'utente si fida del fatto che il proprietario di quella chiave possa a sua volta certificare altri
Tutto però si basa sul fatto che, ad ogni utente, saranno validi solo certificati emessi da un altro utente di cui si riconosce almeno un validatore.
(Io mi fido di Bob solo se qualcuno che conosco si fida di Bob)

Esempio di un certificato PGP:

Alice vuole inviare un messaggio a Bob ma non ha una CA che garantisca per lui. Inizia quindi a "investigare" tra i firmatari della chiave di Bob (Ellen, Fred e Giselle).

Notazione: Ellen, Fred, Giselle, Bob $\ll$ Bob $\gg$.

Inizialmente segue la pista di **Giselle**, firmata a sua volta da **Henry**. Tuttavia, Alice conosce Henry solo vagamente e nota che il suo livello di fiducia è "**casual**" (superficiale); non ritenendo questa testimonianza abbastanza forte, si ferma.

La svolta avviene analizzando **Ellen**: Alice scopre che è garantita da **Jack**, suo cugino, di cui si fida ciecamente (livello "**positive**"). Grazie a questo legame familiare, la fiducia si trasmette: Alice si fida di Jack, Jack garantisce per Ellen e Ellen garantisce per Bob. Solo ora Alice può finalmente accettare la chiave di Bob come autentica.

User Authentication & Access Control

Finora abbiamo visto come la crittografia (Hash, MAC, Certificati X.509, PKI) ci permetta di fidarci *dei dati* e di scambiare chiavi in sicurezza tra due entità.

Ora il focus si sposta: come facciamo ad avere la certezza assoluta che la persona in

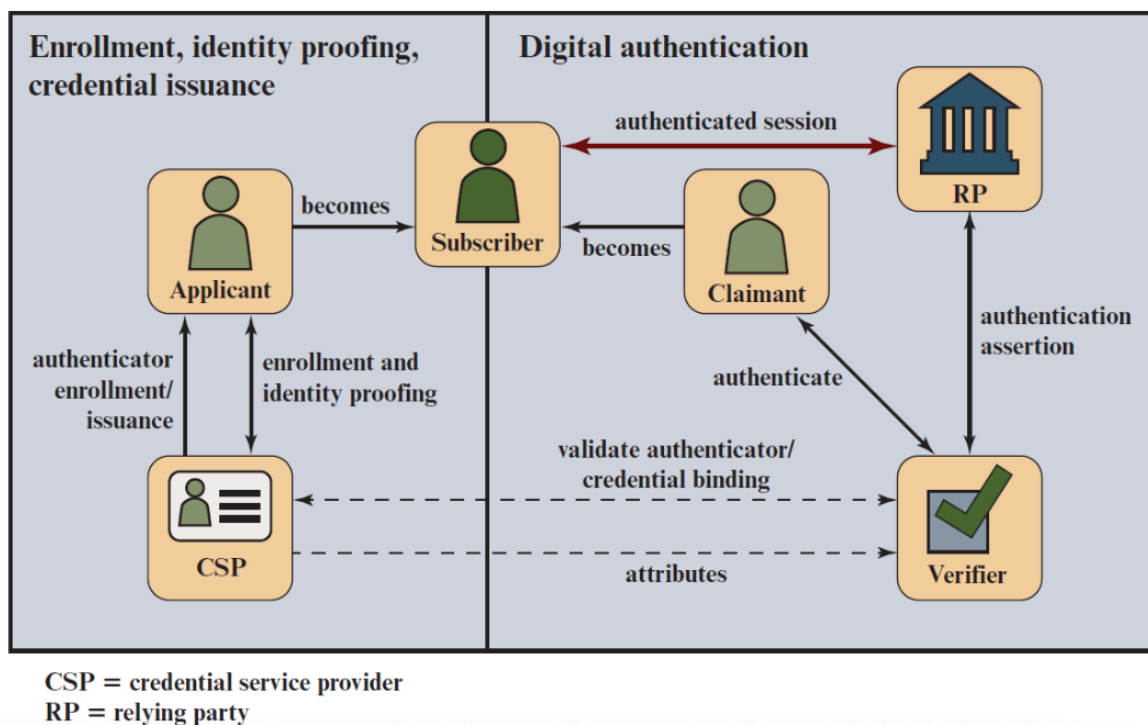
carne ed ossa (o il software) che sta tentando di accedere al nostro sistema sia davvero il proprietario legittimo di quelle credenziali?

Questa è la vera e propria **Autenticazione dell'Utente**, che si differenzia nettamente dall'Autenticazione del Messaggio vista in precedenza.

L'autenticazione del messaggio verifica che il contenuto non sia stato alterato e che la fonte sia autentica, mentre l'autenticazione dell'utente fornisce il controllo degli accessi al sistema verificando le credenziali contro un database.

3 Principi dell'autenticazione:

1. Digital Identity = un set di attributi che rende univoco un soggetto in uno specifico contesto.
2. Identity Proofing = processo di raccolta e validazione delle informazioni che rendono univo un soggetto nel contesto (registrazioni/sign up, la prima volta)
3. Digital Authentication = processo di verifica di possesso di quelle informazioni raccolte nell'identity proofing.



Per autenticare un utente si utilizzano 3 categorie di fattori, usati singolarmente o combinati (Multifactor Authentication):

- Qualcosa che l'utente SA
- Qualcosa che l'utente HA
- Qualcosa che l'utente È o FA

Factor	Examples	Properties
Knowledge	User ID Password PIN	Can be shared Many passwords easy to guess Can be forgotten
Possession	Smart Card Electronic Badge Electronic Key	Can be shared Can be duplicated (cloned) Can be lost or stolen
Inherence	Fingerprint Face Iris Voice print	Not possible to share False positives and false Negatives possible Forging difficult

Mutual Authentication & Replay Attacks

Nei sistemi distribuiti, le due parti devono autenticarsi a vicenda (Mutual Authentication) e scambiarsi le chiavi di sessione.

Questo richiede due proprietà fondamentali: la **Confidenzialità** (i dati devono viaggiare cifrati) e la **Tempestività** (Timeliness), cruciale per difendersi dagli attacchi di tipo Replay (incluso anche DoS).

Un **Replay Attack** avviene quando un attaccante intercetta un messaggio valido e lo riutilizza. Ne esistono 4 varianti:

- L'attaccante Copia e Rispedisce il messaggio in seguito.
- L'attaccante Copia e Rispedisce il messaggio subito finché ancora valido (valid timestamps).

- L'attaccante Copia e Blocca il messaggio originale e Spedisce la sua copia ancora valido.
- L'attaccante rimanda il primo messaggio indietro al mittente originale (magari in un'altra sessione), per ottenere da lui la risposta a quel messaggio (non funziona sempre).

Contromisure a Replay Attack

- Numeri di sequenza (pesante)
 - Timestamps (dipendente dagli orologi sincronizzati)
 - Nonce
-

il Needham-Schroeder simmetrico è il "padre teorico" di Kerberos. Nasce proprio per risolvere il problema della *Mutual Authentication* tramite un server centrale, ma avendo un difetto legato ai *Replay Attack*, ha reso necessaria l'invenzione di Kerberos (che aggiunge i Timestamp per risolvere il problema).

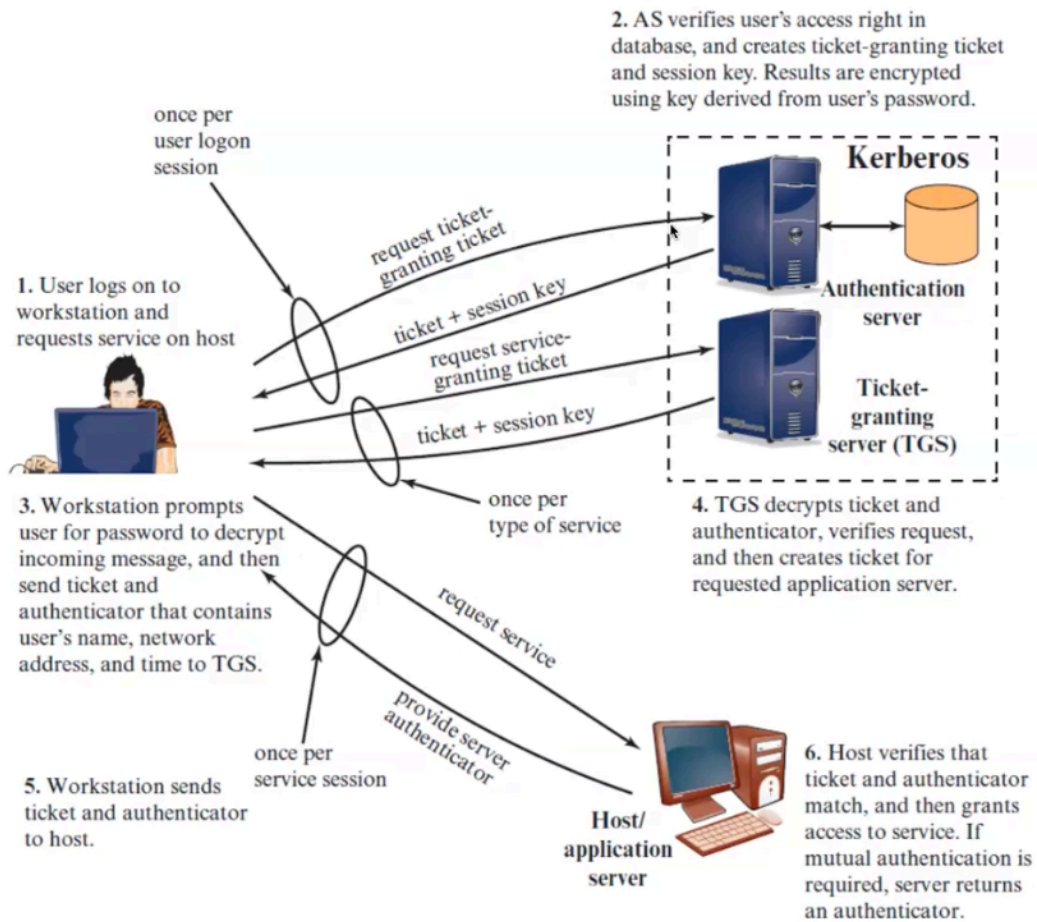
KERBEROS

Kerberos è un **servizio di autenticazione centralizzato** (sviluppato originariamente dal MIT) progettato per reti distribuite non sicure. Il suo scopo primario è fornire la *Mutual Authentication* (autenticazione reciproca) tra utenti e server, utilizzando esclusivamente la **crittografia simmetrica** e una terza parte fidata chiamata **KDC** (Key Distribution Center).

L'architettura è pensata per soddisfare quattro requisiti fondamentali: essere **Sicura** (nessuna password viaggia mai in chiaro), **Affidabile** (supporta server di backup), **Trasparente** (l'utente inserisce la password una sola volta) e **Scalabile**.

tip: Kerberos è il motore di autenticazione predefinito utilizzato da Active Directory Domain Services (AD DS). Active Directory è l'intera infrastruttura di Microsoft (l'elenco di utenti, computer, permessi e server di un'azienda), mentre Kerberos è il meccanismo crittografico nascosto al suo interno che gestisce i login.

Overview



Complete Message Exchanges (version 4)

(1) $C \rightarrow AS \quad ID_C \parallel ID_{TGS} \parallel TS_1$

(2) $AS \rightarrow C \quad E(K_{c, tgs}, [K_{c, tgs} \parallel ID_{TGS} \parallel TS_2 \parallel Lifetime_2 \parallel Ticket_{tgs}])$

$$Ticket_{tgs} = E(K_{tgs}, [K_{c, tgs} \parallel ID_C \parallel AD_C \parallel ID_{TGS} \parallel TS_2 \parallel Lifetime_2])$$

(a) Authentication Service Exchange to obtain ticket-granting ticket

(3) $C \rightarrow TGS \quad ID_V \parallel Ticket_{tgs} \parallel Authenticator_c$

(4) $TGS \rightarrow C \quad E(K_{c, tgs}, [K_{c, v} \parallel ID_V \parallel TS_4 \parallel Ticket_v])$

$$Ticket_{tgs} = E(K_{tgs}, [K_{c, tgs} \parallel ID_C \parallel AD_C \parallel ID_{TGS} \parallel TS_2 \parallel Lifetime_2])$$

$$Ticket_v = E(K_v, [K_{c, v} \parallel ID_C \parallel AD_C \parallel ID_V \parallel TS_4 \parallel Lifetime_4])$$

$$Authenticator_c = E(K_{c, tgs}, [ID_C \parallel AD_C \parallel TS_3])$$

(b) Ticket-Granting Service Exchange to obtain service-granting ticket

(5) $C \rightarrow V \quad Ticket_v \parallel Authenticator_c$

(6) $V \rightarrow C \quad E(K_{c, v}, [TS_5 + 1])$ (for mutual authentication)

$$Ticket_v = E(K_v, [K_{c, v} \parallel ID_C \parallel AD_C \parallel ID_V \parallel TS_4 \parallel Lifetime_4])$$

$$Authenticator_c = E(K_{c, v}, [ID_C \parallel AD_C \parallel TS_5])$$

(c) Client/Server Authentication Exchange to obtain service

- *Timestamp* = anti Replay Attack
- *ID* = anti Binding Attack
- *K_c* = chiave simmetrica condivisa tra client e AS, MA COME FA AD AVERLA???

risposta: (Quando ti assumono - IDENTITY PROOFING)

Il giorno in cui sei arrivato in azienda, l'amministratore di sistema ha creato il tuo account "nome_tuo" e ti ha assegnato una password iniziale.

In quel preciso istante, il Server Kerberos (l'AS) ha preso quella password, ne ha fatto l'hash (creando così la famosa chiave *K_c*) e se l'è salvata a tempo indeterminato nel suo database interno sicuro.

Quando OGGI tu fai il login, il pc manda un messaggio:

"Ciao, sono l'utente x, voglio entrare" → (messaggio 1 del protocollo kerberos)

Il server **cerca se ti trova nel suo database**, trova la tua chiave *K_c* e ti spedisce il messaggio 2 del protocollo.

Il pacchetto arriva al tuo PC illeggibile (perché cifrato con *K_c*), tu scrivi la password, ne viene fatto l'hash locale (**ottenendo K_c**) e la utilizzi per decifrare tutto il pacchetto.

L'AS è SSO - Single Sign On → ci parlo solo una volta al “mattino” per ottenere il ticketTGS, e poi per ogni esigenza tornare direttamente al TGS ogni volta.

- Kc, tgs e le altre chiavi simmetriche = create invece al momento con entropia → PRG → PRF
 - ADc = Address (indirizzo di rete)
 - IDv = nome della risorsa che il Client vorrebbe usare
 - *Authenticato* = ha un TS_n che deve essere compreso tra TS_{n-1} e $Lifetime_{n-1}$.
-

Scalabilità (Cross-Realm Authentication)

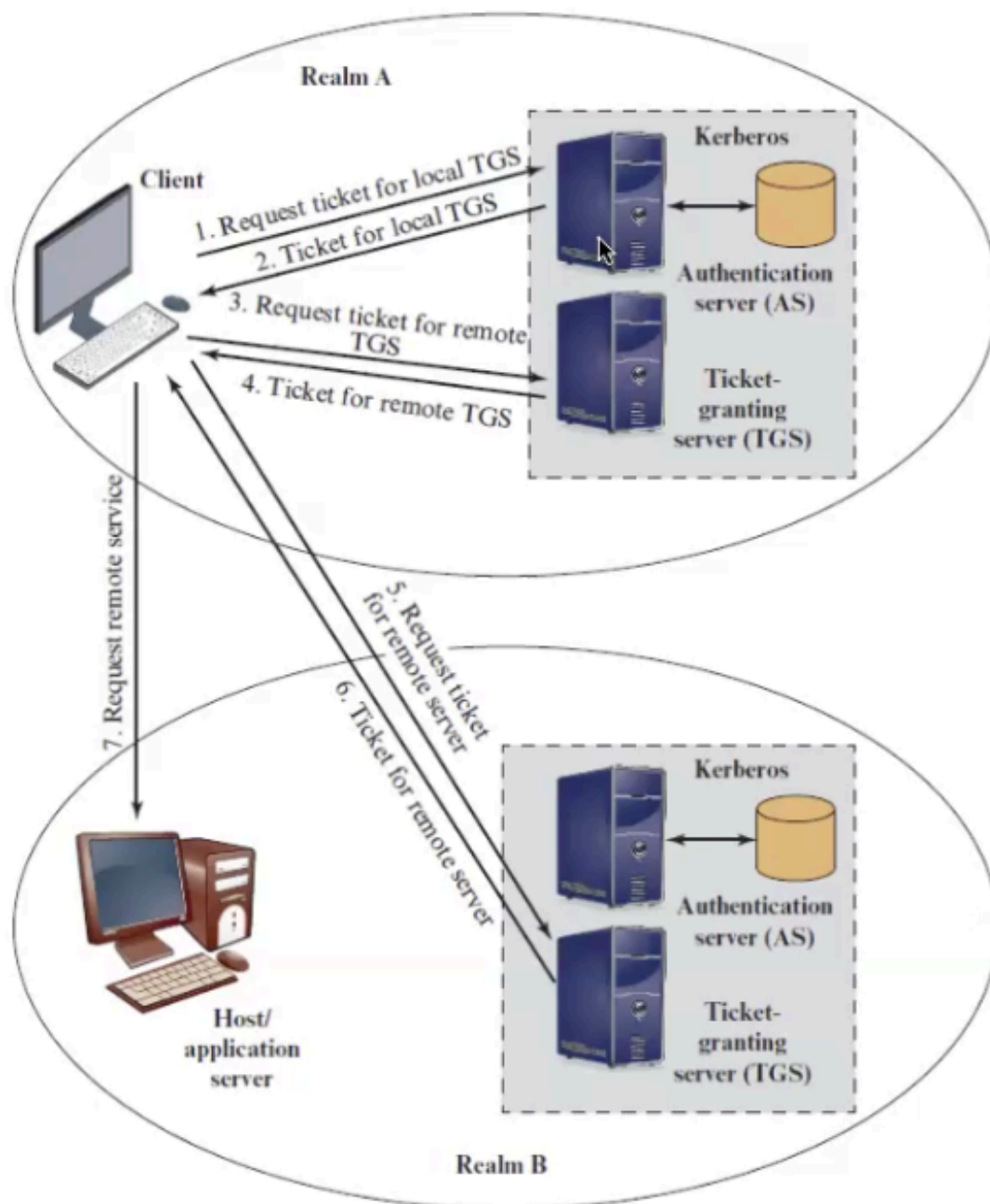
Una caratteristica che ha reso così utilizzato Kerberos, è la sua scalabilità (complessità $O(n^2)$).

Se un'azienda avesse 100.000 computer e un solo KDC centrale, quel server esploderebbe per le troppe richieste. La soluzione alla scalabilità è **dividere la rete in tanti "Realms"** (quelli che in Windows Active Directory si chiamano *Domini*). Ogni Realm ha il suo KDC, che gestisce solo i suoi utenti locali.

Ma cosa succede se tu (nel Realm A) devi accedere a un server aziendale che si trova a New York (nel Realm B)?

Non puoi usare il tuo TGT (il pass-giornaliero) nel Realm B, perché il KDC di New York non ti conosce e non ha la tua password.

La soluzione è la **Relazione di Trust (Trust Relationship)**: gli amministratori fanno in modo che il KDC del Realm A e il KDC del Realm B condividano una **chiave crittografica segreta tra di loro**.



L'autenticazione Cross-Realm in 4 Fasi

Immagina i due Realm come due entità separate che hanno stretto un accordo diplomatico (la **Relazione di Trust**, ovvero una chiave segreta condivisa tra i due KDC). Tu sei un utente del **Realm A** e vuoi accedere a un server nel **Realm B**.

- 1. Passaggi 1 e 2 (Il Passaporto Locale):** Ti rivolgi all'**AS locale** (nel Realm A) per ottenere il tuo documento base (il tuo **TGT locale**). L'AS ha la tua chiave nel suo

database, ti autentica e ti rilascia il TGT.

2. **Passaggi 3 e 4 (Il Visto Internazionale):** Vai al **TGS locale** (sempre nel Realm A) e richiedi l'accesso a una risorsa che si trova fuori dal tuo dominio. Il TGS locale, sfruttando la relazione di Trust, ti rilascia un **Cross-Realm TGT** (un "Visto d'uscita"). Questo pacchetto significa: *"Il KDC del Realm A garantisce ufficialmente l'identità di questo utente"*.
 3. **Passaggi 5 e 6 (L'Ambasciata Estera):** Ora contatti direttamente il **TGS remoto** (nel Realm B) e gli presenti il tuo Cross-Realm TGT. Il TGS remoto riconosce la crittografia dell'accordo di Trust, si fida della garanzia data dal Realm A, e ti genera il **Ticket di Servizio** finale e specifico per il server di destinazione.
 4. **Passaggio 7 (L'Accesso):** Consegni il Ticket di Servizio al **Target Server** (nel Realm B). Il server lo verifica e ti fa entrare.
-

Differenze tra Version 4 e Version 5

La versione 4 è ormai un reperto storico (nata negli anni '80 e piena di difetti), mentre la versione 5 (nata negli anni '90 e costantemente aggiornata) è lo standard robusto che usiamo oggi ovunque.

1. La Crittografia

- **Versione 4:** Era costruita "su misura" per un solo algoritmo di cifratura: il **DES**. Oggi il DES è considerato debolissimo e facilmente bucabile.
- **Versione 5:** È stata riscritta per essere "modulare". Non è più bloccata su un singolo algoritmo, ma supporta qualsiasi sistema crittografico moderno, come **AES**.

2. La Scadenza dei Ticket

- **Versione 4:** Per un limite tecnico su come contava il tempo (usava un contatore troppo piccolo), un ticket non poteva durare fisicamente più di **21 ore circa**. Se avevi un programma che doveva lavorare sul server in background per tre giorni, il sistema ti buttava fuori.
- **Versione 5:** Rimuove questo limite e, soprattutto, inventa i **Ticket Rinnovabili**. Un ticket può avere una scadenza breve (es. 8 ore), ma il tuo PC può chiederne

automaticamente il rinnovo al KDC prima che scada, permettendo l'esecuzione di processi lunghi giorni interi senza chiederti di rimettere la password.

3. La Dipendenza dagli Indirizzi IP

- **Versione 4:** Era legata a doppio filo agli indirizzi di rete IPv4. Il sistema inseriva letteralmente il tuo indirizzo IP dentro il ticket. Questo creava grossi problemi di mobilità e non era compatibile con le reti moderne.
- **Versione 5:** È **indipendente dal protocollo di rete**. I ticket possono trasportare qualsiasi tipo di indirizzo (compreso il moderno IPv6) o addirittura funzionare in reti dove gli indirizzi IP non vengono usati affatto.

4. La Delega (I Ticket Forwardable)

Questa è una novità cruciale della Versione 5.

- Immagina di accedere a un Server A e chiedergli di elaborare un file che però si trova su un Server B.
 - **La Versione 5** permette la **Delega** (Forwarding). Tu puoi dare al Server A un ticket speciale che lo autorizza ad andare dal Server B "a nome tuo", proprio come se tu gli firmassi una delega cartacea per ritirare un pacco in posta. La versione 4 non sapeva gestire questa operazione in modo sicuro.
-

Mutual Authentication, Denning Protocol

A differenza degli algoritmi base (come Diffie-Hellman o RSA) che si limitano a scambiare le chiavi alla cieca, i protocolli di **Denning** (tramite orologi) e **Woo-Lam** (tramite numeri casuali) introducono la **Mutua Autenticazione**, garantendo l'identità certa di entrambe le parti oltre che lo scambio della chiave.

- **Denning** protocol using timestamps
 - Uses an authentication server (AS) to provide public-key certificates
 - Requires the synchronization of clocks
 1. $A \rightarrow AS: ID_A \parallel ID_B$
 2. $AS \rightarrow A: E(PR_{as}, [ID_A \parallel PU_a \parallel T]) \parallel E(PR_{as}, [ID_B \parallel PU_b \parallel T])$
 3. $A \rightarrow B: E(PR_{as}, [ID_A \parallel PU_a \parallel T]) \parallel E(PR_{as}, [ID_B \parallel PU_b \parallel T]) \parallel E(PU_b, E(PR_a, [K_s \parallel T]))$
- Woo and Lam makes use of nonces
 - Care needed to ensure no protocol flaws

TUTTI i modi VISTI FIN'ORA per lo SCAMBIO AUTENTICATO DI CHIAVI

Precedentemente avevamo visto una lista protocolli in cui scambiare in maniera autenticata una chiave tramite crittografia ASIMMETRICA e SIMMETRICA, ma solo alcuni fanno Mutual Authentication.

Ecco un recap:

Famiglia Crittografica	Protocollo	Mutual Auth	Il Concetto in Sintesi
1. SIMMETRICA <i>(Richiede un Server Centrale di fiducia. Usa una chiave segreta condivisa).</i>	Needham-Schroeder (Simm.)	Sì	Il capostipite. Un KDC genera la chiave di sessione e la invia chiusa nei segreti di A e B.
	Kerberos (v4 & v5)	Sì	L'evoluzione pratica. Aggiunge Ticket (TGT), Timestamp e divisione AS/TGS per il Single Sign-On.

2. ASIMMETRICA (Usa Chiavi Pubbliche e Private. I segreti non vanno in rete).	Diffie-Hellman	No	Magia matematica. A e B generano insieme la chiave "al volo", senza trasmetterla. Rischio Man-in-the-Middle.
	RSA (Key Exchange)	No	Il lucchetto pubblico. A crea la chiave e la spedisce a B cifrandola con la chiave pubblica di B. B non sa chi gliel'ha mandata.
	Denning	Sì	Basato sul Tempo. Usa un Server per i certificati pubblici. Si protegge e autentica con i <i>Timestamp</i> (orologi).
	Woo-Lam	Sì	Basato sulla Casualità. Alternativa a Denning. Si protegge e autentica scambiandosi i <i>Nonce</i> (numeri casuali).
	Needham-Schroeder (Asimm.)	Sì	Basato su Sfide. Usa un Server per i certificati e si autentica sfidandosi a vicenda tramite <i>Nonce</i> .

One Way Authentication

IL VERO PROBLEMA (Perché serve questo approccio?):

In tutti i protocolli visti finora (Kerberos, Denning, ecc.), Alice e Bob sono **entrambi online contemporaneamente**. Possono dialogare in tempo reale ("botta e risposta") per sfidarsi, verificare la reciproca identità e accordarsi su una chiave simmetrica *prima* di scambiarsi i dati.

Nell'Autenticazione a Senso Unico (es. **E-mail**), **il destinatario è offline**. Non c'è alcun dialogo possibile. Il mittente deve fare tutto da solo: deve creare una chiave simmetrica, cifrare il messaggio e spedire tutto in un unico pacchetto "alla cieca", senza aver prima potuto stabilire un contatto con l'altra parte.

SOLUZIONE:

Dato che il destinatario è offline, l'unico punto di partenza che il mittente ha a disposizione è la **Chiave Pubblica** del destinatario. Per inviare il messaggio in sicurezza senza potersi prima parlare, si usa un incastro in due passaggi:

1. **Per il messaggio (Velocità):** Il mittente genera sul momento una *chiave simmetrica usa-e-getta* e la usa per cifrare l'intero testo del messaggio (la crittografia simmetrica è fulminea).
2. **Per la chiave (Sicurezza senza dialogo):** Il mittente deve far avere questa chiave usa-e-getta al destinatario offline, senza che nessun altro possa intercettarla. Prende quindi la *chiave pubblica del destinatario* e la usa come una busta blindata per sigillare soltanto la piccola chiave simmetrica.

Quando il destinatario si collegherà (anche ore dopo):

1. Userà la propria **chiave privata** per aprire la "busta blindata" e recuperare la chiave simmetrica usa-e-getta.
2. Userà la chiave simmetrica appena recuperata per decifrare e leggere il testo del messaggio.

ATTACCHI WEB

	Threats	Consequences	Countermeasures
Integrity	• Modification of user data • Trojan horse browser • Modification of memory • Modification of message traffic in transit	• Loss of information • Compromise of machine • Vulnerability to all other threats	Cryptographic checksums
Confidentiality	• Eavesdropping on the net • Theft of info from server • Theft of data from client • Info about network configuration • Info about which client talks to server	• Loss of information • Loss of privacy	Encryption, Web proxies
Denial of Service	• Killing of user threads • Flooding machine with bogus requests • Filling up disk or memory • Isolating machine by DNS attacks	• Disruptive • Annoying • Prevent user from getting work done	Difficult to prevent
Authentication	• Impersonation of legitimate users • Data forgery	• Misrepresentation of user • Belief that false information is valid	Cryptographic techniques

LA POSIZIONE DELLA SICUREZZA

La sicurezza si può mettere a vari livelli dello stack di rete : a livello **Network** (IPSec, trasparente a tutto), a livello **Transport** (SSL/TLS, su cui ci focalizziamo ora), o a livello **Application** (S/MIME o Kerberos, specifici per singole app).

1. Livello di Rete: IPSec

- **Cos'è:** Un protocollo che cifra e autentica i pacchetti dati a livello IP (è il motore delle VPN).
- **Parola chiave (Trasparenza):** Protegge in automatico tutto il traffico. Le applicazioni non devono essere modificate e non si accorgono di nulla.

2. Livello di Trasporto: SSL/TLS

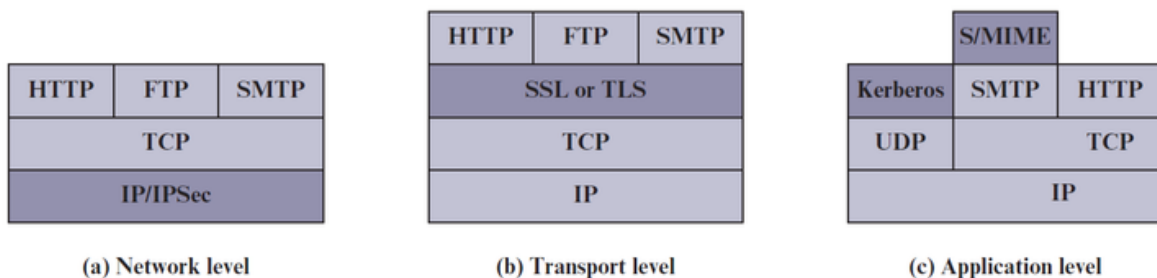
- **Cos'è:** Un protocollo che crea un canale di comunicazione sicuro per una specifica connessione (è quello che trasforma HTTP in HTTPS). (socket autenticati)
- **Parola chiave (Flessibilità):** Sicurezza end-to-end tra due programmi. Non è trasparente: le app (come il browser) devono essere scritte per usarlo.

3. Livello Applicativo: S/MIME

- **Cos'è:** Uno standard di crittografia integrato nelle app di posta per cifrare e firmare le email.
- **Parola chiave (Dato al sicuro):** Protegge il messaggio stesso. L'email è illeggibile per chiunque non sia il destinatario, sia mentre viaggia sia quando è salvata sul server.

4. Livello Applicativo: Kerberos

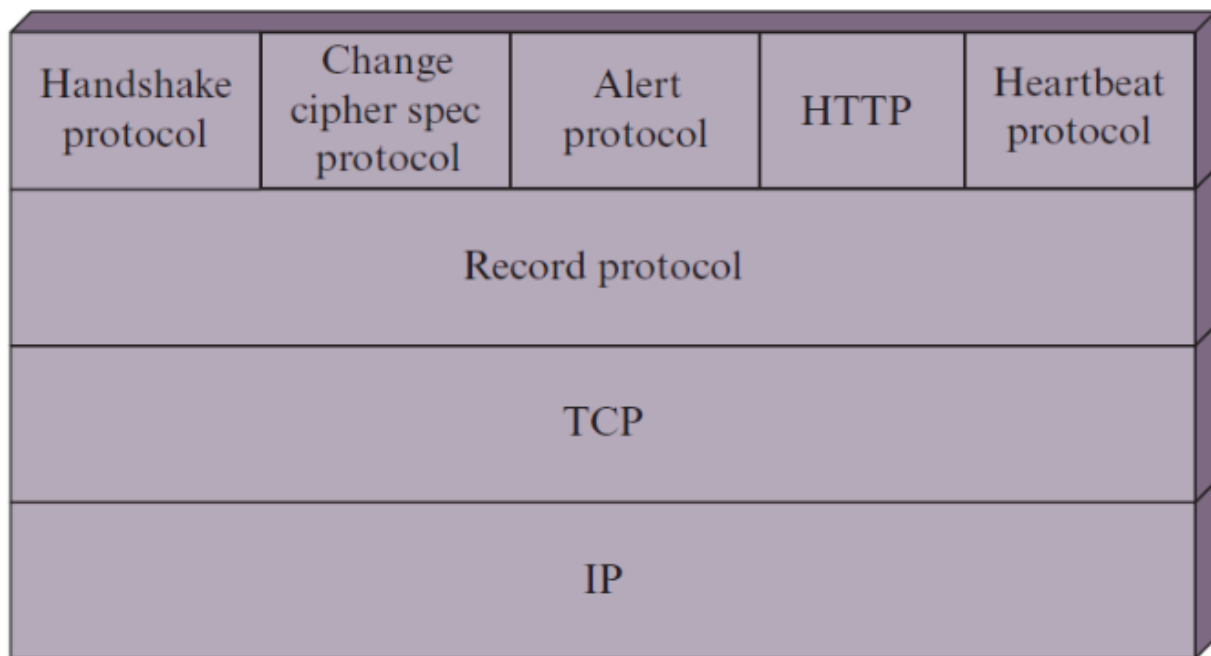
- **Cos'è:** Un sistema di autenticazione di rete che usa dei "ticket" digitali al posto delle password.
- **Parola chiave (Single Sign-On):** Ti autentichi una volta sola in modo sicuro (senza far viaggiare la password in chiaro sulla rete) e il ticket ti dà accesso a tutti i servizi consentiti.



TLS PROTOCOL STACK (Confidentiality, Integrity e Authenticity)

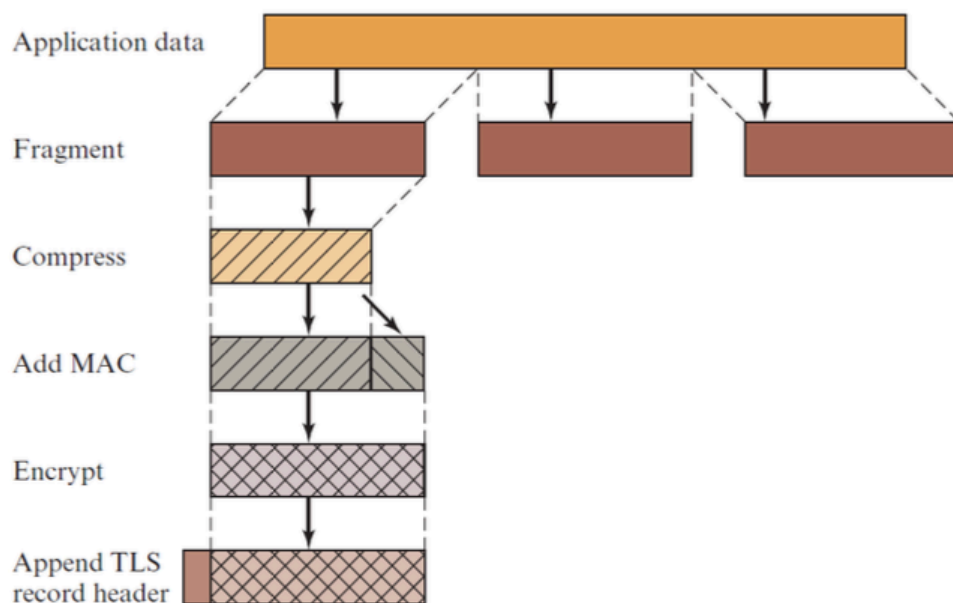
Il protocollo TLS non è un'unica entità monolitica, ma una **suite di protocolli** impilati l'uno sull'altro. Si appoggia su una connessione affidabile fornita dal livello di trasporto

sottostante (il **TCP**) e si divide logicamente in due strati (o layer) principali:

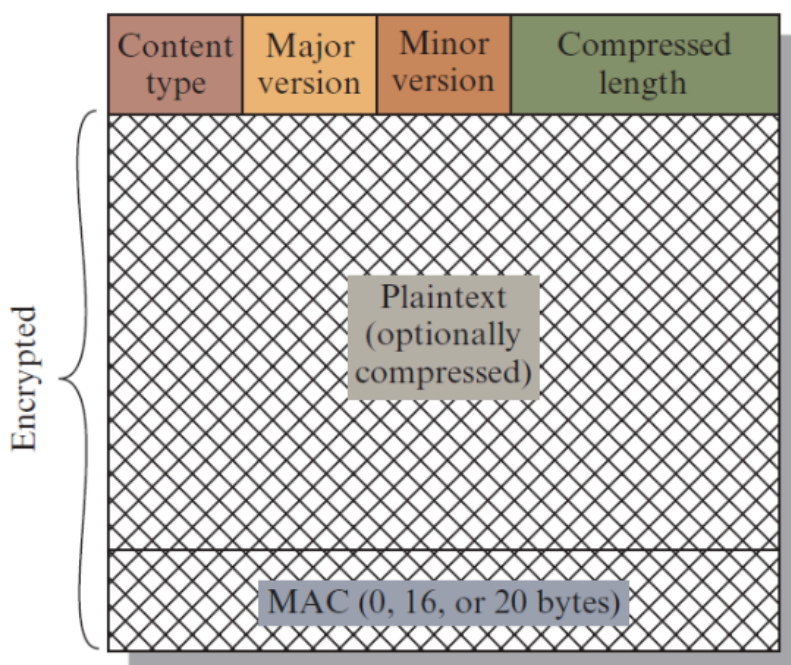


Lo Strato Inferiore:

- **TLS Record Protocol:** È il vero e proprio "trasportatore blindato". Prende i messaggi dai protocolli superiori, li frammenta in blocchi gestibili, li comprime (opzionale, spesso disabilitato oggi), applica un codice MAC per l'integrità e infine li cifra. Qualsiasi cosa passi nello stack TLS, alla fine finisce dentro un pacchetto del *Record Protocol* prima di essere consegnata a TCP.



Otteniamo il pacchetto finale (header in alto, messaggio, mac):



il campo Content Type può essere

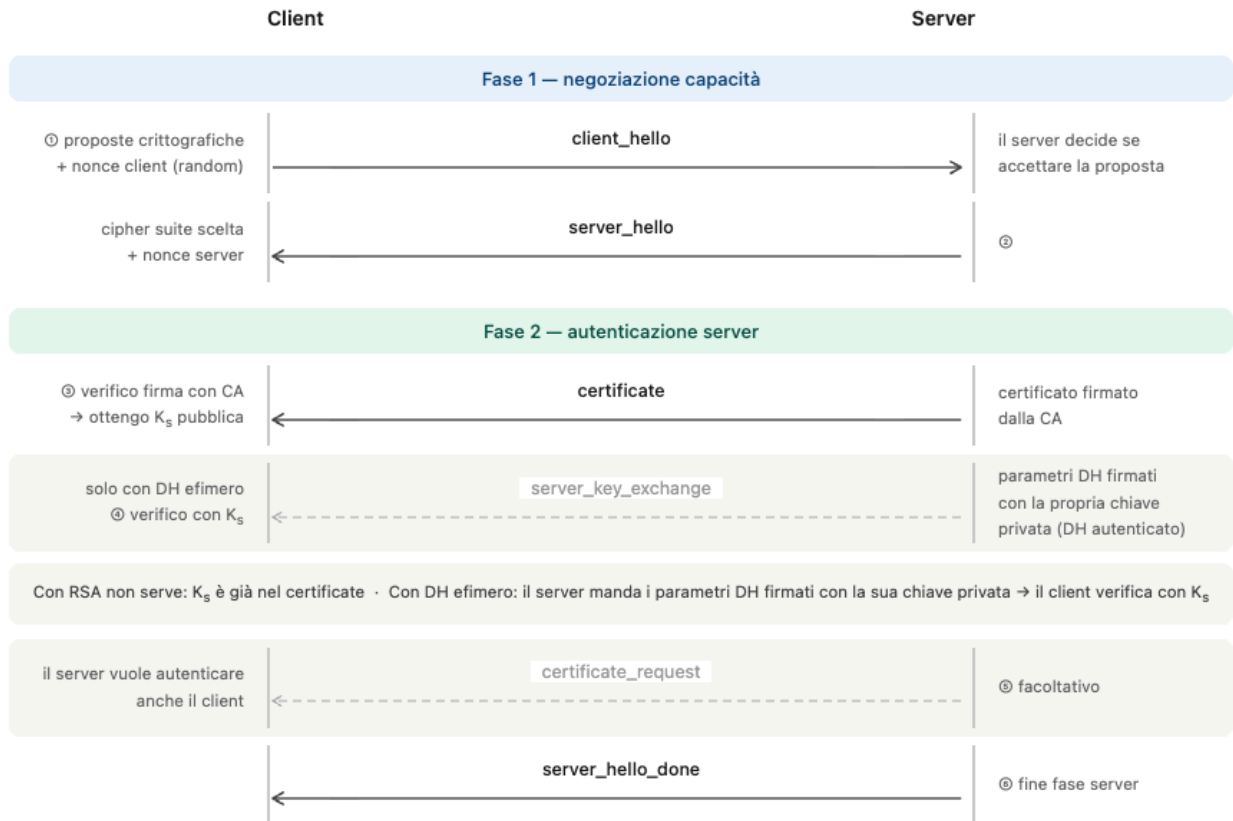
Lo Strato Superiore:

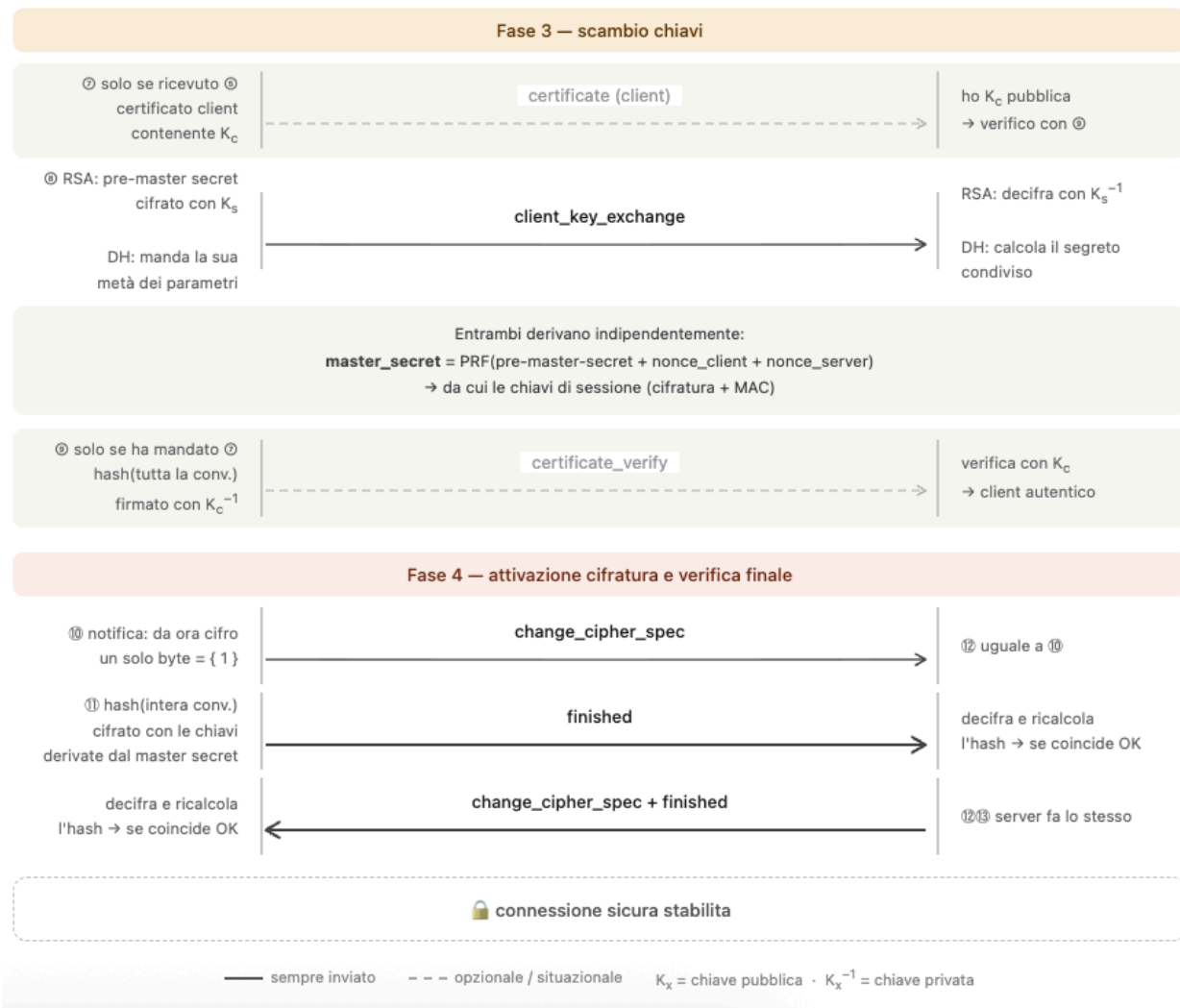
Sopra il Record Protocol girano diversi protocolli paralleli, ognuno con un compito ben

specifico. Si dividono in protocolli di gestione della sicurezza e protocolli per i dati veri e propri:

- **Change Cipher Spec Protocol:** È il protocollo più semplice (un solo byte). È l'interruttore che dice: *"Da questo momento in poi, tutto ciò che invio sarà cifrato con le chiavi e gli algoritmi che abbiamo appena concordato nell'Handshake"*.
 - **Alert Protocol:** È il sistema di messaggistica di emergenza e servizio. Viene usato per comunicare errori (es. "Certificato non valido", "Firma errata") o per segnalare la chiusura pulita della connessione.
 - **Heartbeat Protocol:** Serve per inviare piccoli messaggi di "ping" crittografati per mantenere viva la connessione tra Client e Server anche se non c'è traffico HTTP, ed evitare che i firewall taglino la connessione.
 - **HTTP (o altri protocolli applicativi):** Sono i dati veri e propri della tua applicazione (es. richieste web, traffico email). Per il livello TLS, questi sono semplicemente dati opachi da proteggere e spedire.
 - **Handshake Protocol:** È il protocollo più complesso. Serve all'inizio della connessione per far negoziare a Client e Server quali algoritmi crittografici usare, per autenticarsi (tramite certificati X.509) e per scambiarsi in sicurezza la chiave master.
-

TLS Handshake Protocol





Da pre-master a Master secreta a Chiavi Derivate:

Il **pre_master_secret** viene passato alla PRF con label "**master_secret**" e i due nonce per produrre 48 byte di **master_secret**. Il **master_secret** viene poi ripassato alla stessa PRF con label "**key_expansion**" per espandere 128+ byte di materiale da cui si tagliano le chiavi di sessione: **client_write_key**, **server_write_key**, **client_write_MAC**, **server_write_MAC**. La PRF è costruita su HMAC-SHA256 applicato due volte per blocco — una volta per far avanzare la catena $A(i)$, una volta per produrre l'output. Le chiavi MAC derivate vengono poi usate anch'esse con HMAC per autenticare ogni record TLS trasmesso durante la sessione.

una cosa FONDAMENTALE è il concetto di PERFECT FORWARD SECRECY

Perfect Forward Secrecy (PFS)

Definizione:

Se un attaccante registra tutto il traffico cifrato oggi, e in futuro riesce a ottenere la chiave privata del server, con PFS non può decifrare nulla — perché le chiavi di sessione passate non sono ricostruibili.

Perché RSA non ce l'ha

Con RSA, il *pre_master_secret* viene cifrato con la chiave pubblica del server e mandato direttamente. Chi ha la chiave privata del server può decifrarlo, e da lì ricostruire tutto il *master_secret* e tutte le chiavi di sessione — anche anni dopo.

registra traffico oggi → ottiene $K^{-1}s$ domani → decifra tutto = ✓

Perché DH efimero ce l'ha

Con DHE (Diffie-Hellman **efimero**), il server genera una coppia di chiavi DH **nuova per ogni sessione** e la butta via alla fine. La chiave privata a lungo termine serve solo per **firmare** i parametri DH, non per cifrare il segreto.

*registra traffico oggi → ottiene $K^{-1}s$ domani →
può verificare la firma ma non ricostruire il segreto DH = ✗*

Il segreto condiviso è stato calcolato da chiavi effimere che non esistono più.

SESSIONI E CONNESSIONI TLS

Il problema principale di TLS è che la fase iniziale di autenticazione e scambio delle chiavi (crittografia asimmetrica) richiede un'enorme quantità di potenza di calcolo (CPU) e tempo di rete. Un sito web moderno richiede di scaricare decine di risorse diverse (HTML, immagini, script). Se il browser dovesse eseguire un Handshake crittografico completo per ogni singola risorsa, i server collasserebbero sotto il carico computazionale e la navigazione avrebbe latenze inaccettabili.

La soluzione architetturale di TLS è separare nettamente la **creazione dell'accordo crittografico** dal **trasferimento pratico dei dati**.

Si crea **una sola Sessione** (operazione lenta e costosa) per stabilire il contesto di sicurezza globale, all'interno della quale vengono aperte **molteplici Connessioni**

parallele e sequenziali (operazioni fulminee) che ereditano la sicurezza dalla Sessione madre.

quanto dura una sessione? dipende, lo decide l'amministratore del server. Abbastanza da beneficiarne ma non troppo da avere rischi di sicurezza.

PARAMETRI SESSIONE

Questi sono i parametri pesanti che vengono negoziati all'inizio e mantenuti in memoria per poter riaprire rapidamente nuove connessioni in futuro.

- **Session Identifier (ID di Sessione):** Un codice univoco scelto dal server per identificare quella specifica sessione.
- **Peer Certificate:** Il certificato di sicurezza (in formato X509.v3) che dimostra crittograficamente l'identità del server (o, a volte, del client).
- **Compression Method:** L'algoritmo scelto per comprimere i dati prima di cifrarli.
- **Cipher Spec:** Definisce esplicitamente quali algoritmi usare, ad esempio: *"Useremo AES per cifrare i dati in massa e SHA per calcolare le firme di integrità (MAC)"*.
- **Master Secret:** È una super-chiave segreta di 48 byte condivisa esclusivamente tra client e server. Da questo segreto centrale verranno generate tutte le chiavette temporanee.
- **Is Resumable:** Un interruttore (flag) che indica se questa sessione è ancora valida per generare nuove connessioni future.

PARAMETRI CONNESSIONE

Questi sono i parametri generati "al volo" ogni singola volta che si apre un nuovo tubo di comunicazione. Le chiavi sono sempre sdoppiate (una per chi invia, una per chi riceve) per evitare che il traffico in entrata e in uscita vada in conflitto.

- **Server and Client Random:** Nounce usati per la generazioni delle chiavi impedendo che qualcuno le riutilizzi nel futuro prossimo.
- **MAC Secret (Server e Client):** Le chiavi segrete usate per calcolare il codice di autenticazione del messaggio (MAC) e garantirne l'integrità.
- **Write Key (Server e Client):** Le chiavi simmetriche vere e proprie (es. quelle dell'algoritmo AES scelto prima) usate per cifrare e decifrare i pacchetti di dati.

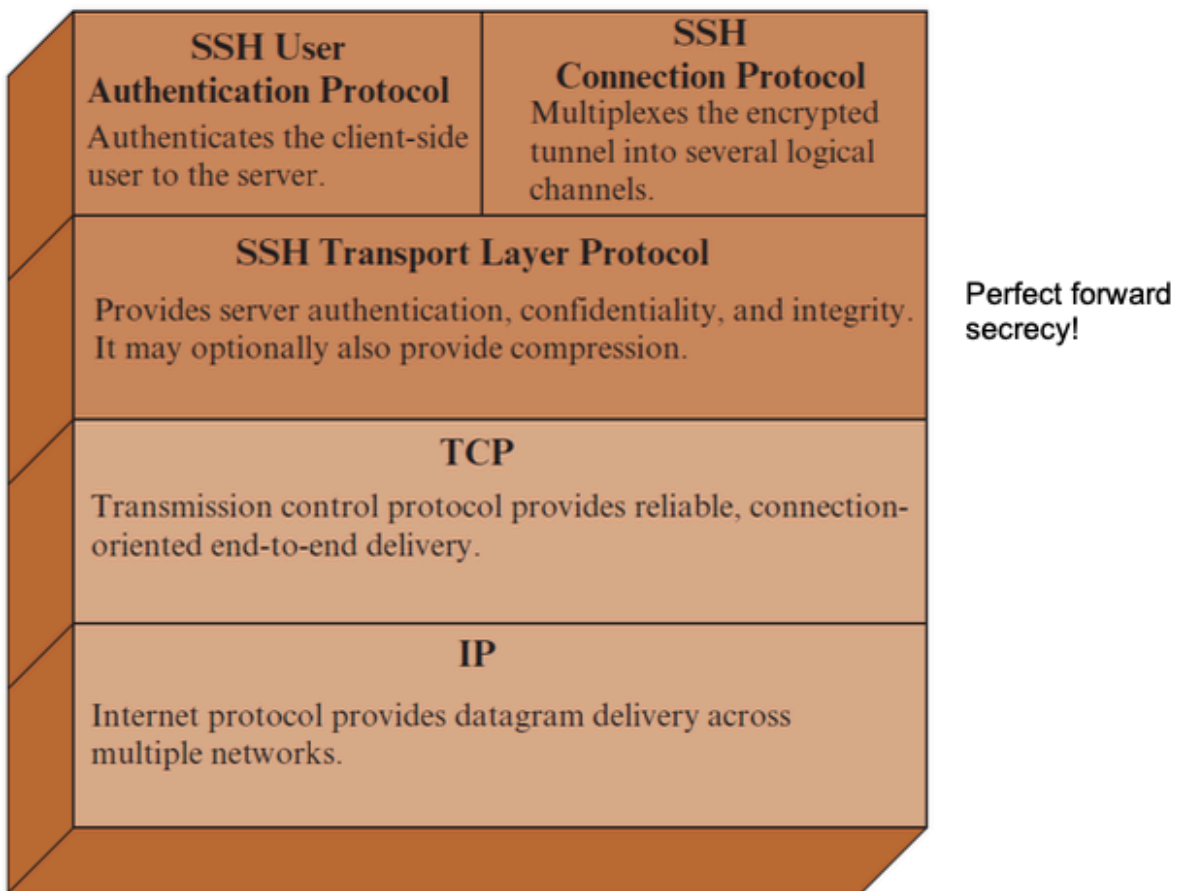
- **Initialization Vectors (IV):** Numeri di partenza (Vettori di Inizializzazione) necessari se l'algoritmo di cifratura lavora "a blocchi concatenati" (modalità CBC).
 - **Sequence Numbers:** contatore dei messaggi inviati e ricevuti. È vitale contro gli attacchi di *Replay*: impedendo a un hacker di catturare un tuo pacchetto e reinviarlo istantaneamente per farti ripetere involontariamente un'azione all'interno della stessa connessione.
-

SSH

SSH (Secure Shell) è un protocollo per comunicazioni di rete sicure, nato come sostituto sicuro di Telnet e altri strumenti di accesso remoto che trasmettevano tutto in chiaro. A differenza di HTTPS — che aggiunge TLS sopra HTTP — SSH è un protocollo autonomo che integra al suo interno autenticazione, cifratura e integrità.

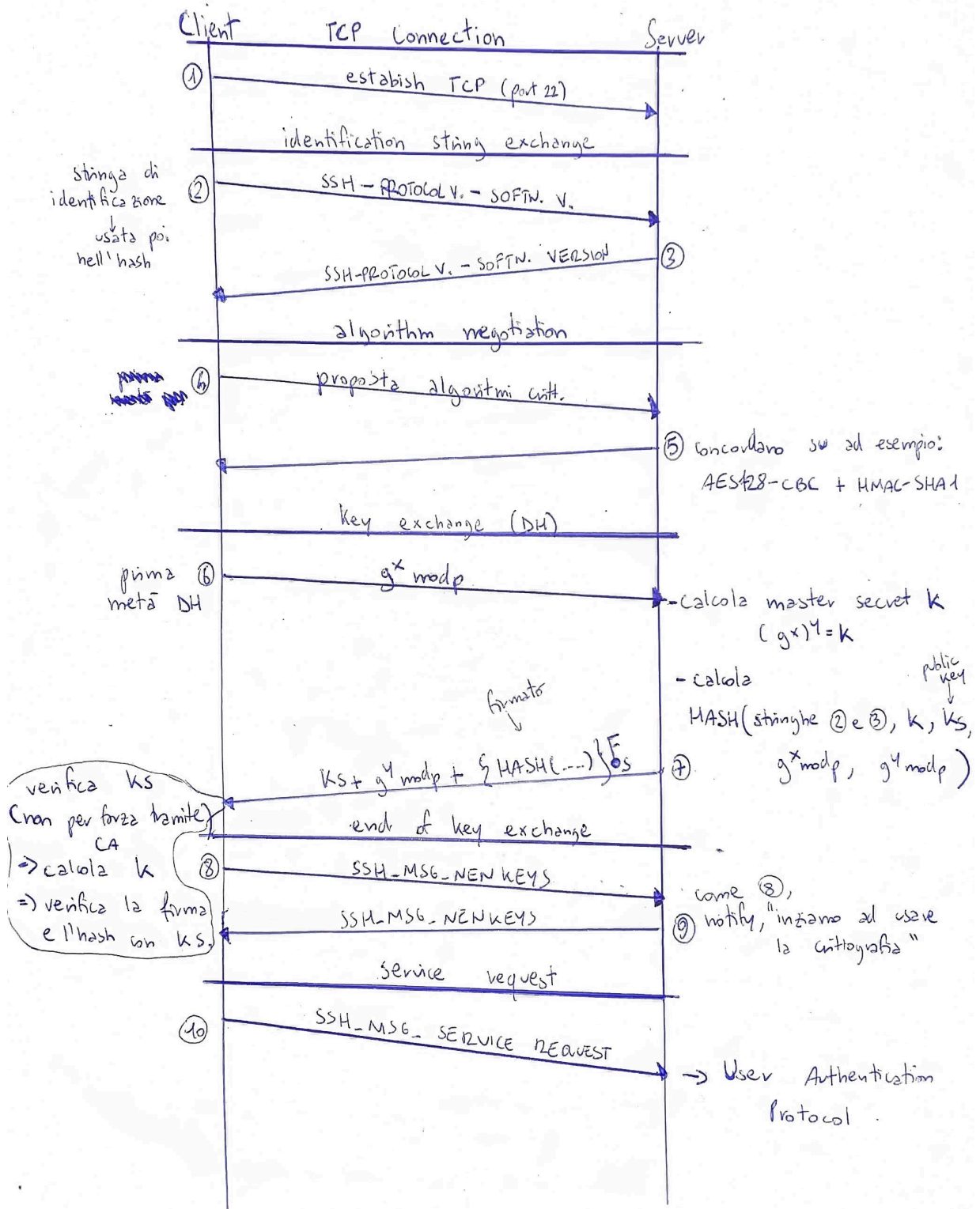
Si compone di tre livelli sovrapposti: il **Transport Layer Protocol**, che gestisce l'autenticazione del server tramite DH efimero (con perfect forward secrecy), la cifratura e l'integrità della connessione; l'**User Authentication Protocol**, che autentica il client al server tramite chiave pubblica, password o host-based; e il **Connection Protocol**, che moltiplica la connessione sicura in più canali logici indipendenti, abilitando funzionalità come shell remote, file transfer e port forwarding.

Una differenza chiave rispetto a TLS è il modello di fiducia: SSH non richiede necessariamente una CA — il client può verificare il server tramite un repository locale di chiavi pubbliche note (modello TOFU, trust on first use).



TRANSPORT LAYER PROTOCOL

SSH TRANSPORT LAYER PROTOCOL



USER AUTHENTICATION PROTOCOL

Il client manda `SSH_MSG_USERAUTH_REQUEST` sostanzialmente chiede "quali metodi accetti?". Il server risponde con `SSH_MSG_USERAUTH_FAILURE` contenente la lista dei metodi disponibili. Da lì il client sceglie un metodo e ci prova.

I tre metodi principali:

Password — il più semplice. Il client manda username + password in chiaro dentro il pacchetto SSH (che però è già cifrato dal transport layer, quindi la password viaggia protetta). Il server verifica e risponde successo o fallimento.

Public key — il client dice "voglio usare questa chiave pubblica". Il server controlla se quella chiave è nel suo `~/.ssh/authorized_keys`. Se sì, il client firma un messaggio con la sua **chiave privata** e lo manda al server. Il server verifica la firma con la chiave pubblica — se torna, l'identità è provata senza che la chiave privata abbia mai lasciato il client.

Host-based — l'autenticazione non è sul singolo utente ma sull'intera macchina client. Il server si fida del host del client, che firma con la propria chiave privata host. Meno usato, perché se la macchina client è compromessa si compromette tutto.

Se un metodo fallisce il server rimanda `SSH_MSG_USERAUTH_FAILURE` con i metodi ancora disponibili, e il client può riprovare con un altro. Quando tutto va a buon fine il server manda `SSH_MSG_USERAUTH_SUCCESS` e si passa al Connection Protocol.

CONNECTION PROTOCOL

Lo **SSH Connection Protocol** gira sopra il User Auth — quindi il canale è cifrato, il server autenticato, e l'utente autenticato. Il suo scopo è **multiplexare** tutto questo su canali logici separati.

I tipi di canale:

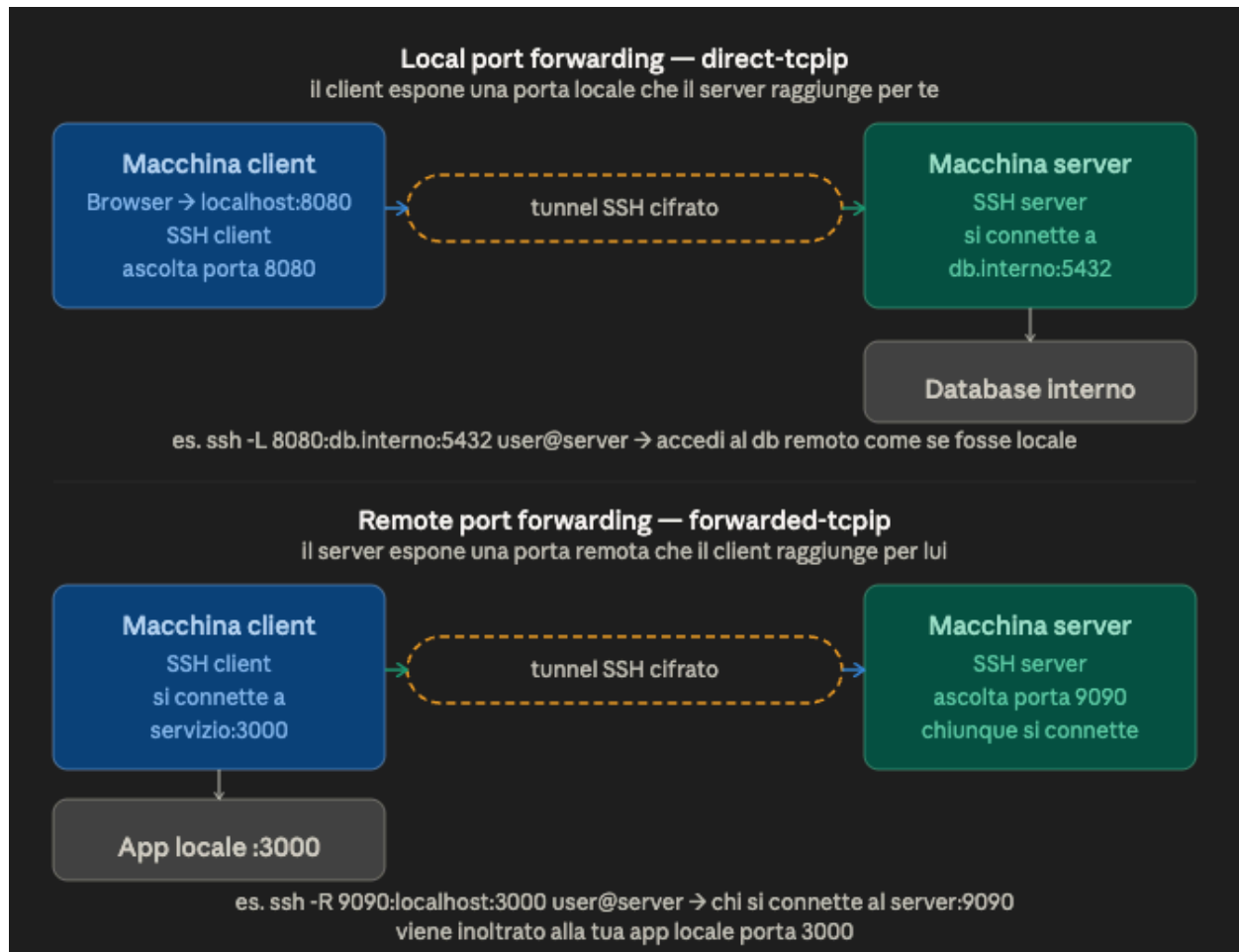
Session — il più comune. Apre una sessione remota: può essere una shell interattiva, l'esecuzione di un singolo comando, o un trasferimento file.

X11 — forwarding del sistema grafico X Window. L'applicazione gira sul server ma la GUI viene mostrata sul desktop locale.

Direct-tcpip (Local Port Forwarding) — Il client chiede al server di aprire una connessione TCP verso un host:porta remoto, e il traffico passa attraverso il tunnel SSH

cifrato.

Forwarded-tcpip (Remote Port Forwarding) — Il server ascolta su una porta e rimanda il traffico al client, che lo consegna a un host locale.



IPsec

IPsec (Internet Protocol Security) è una suite di protocolli e standard progettata per fornire sicurezza crittografica alle comunicazioni direttamente a livello di rete (Livello 3 del modello OSI/Pila TCP-IP).

Perché nasce: Il protocollo IP originale (IPv4) è stato storicamente concepito per instradare pacchetti nel modo più efficiente possibile, ma è privo di qualsiasi meccanismo di sicurezza nativo. Questo lascia il traffico esposto a vulnerabilità critiche, quali

- **Analisi del traffico (Traffic Analysis):** Anche se il pacchetto è protetto da TLS, l'header IP è in chiaro. L'attaccante sa **con chi** stai parlando, **quando** ci parli e **quanti dati** vi scambiate. In certi contesti (es. spionaggio aziendale), sapere che il CEO dell'Azienda A comunica intensamente con l'Ufficio Acquisizioni dell'Azienda B è già un'informazione preziosa, anche senza leggerne i messaggi in chiaro. IPsec (in *Tunnel Mode*) cifra anche l'header IP originale, nascondendo chi sta parlando con chi.
- **Replay Attack (a livello IP):** Anche senza capire o modificare il pacchetto TLS, l'attaccante può registrarlo e reinviarlo (re-play) mille volte. Se il livello applicativo non è progettato per gestire duplicati, questo può causare blocchi o comportamenti anomali (es. sovraccarico dei server). IPsec blocca questo attacco usando un numero di sequenza a livello IP che il ricevitore controlla, scartando i duplicati.
- **Spoofing (falsificazione dell'IP):** L'attaccante crea un pacchetto *nuovo* (non il tuo intercettato) e falsifica l'indirizzo IP mittente per far credere al server che la richiesta arrivi da te. Se il server basa la sicurezza solo sull'IP sorgente (ad esempio per l'accesso a una intranet), l'attacco ha successo. IPsec (con AH o ESP autenticato) impedisce questo verificando matematicamente l'identità del nodo.

A cosa serve: Il suo scopo è proteggere il traffico IP end-to-end (tra due host) o tra due reti (tra due gateway/router), fornendo quattro garanzie fondamentali:

CONFIDENTIALITY, INTEGRITY, AUTHENTICATION (message auth), ACCESS CONTROL e ANTI-REPLAY.

Essendo implementato a livello di rete, il suo grande vantaggio è che protegge automaticamente tutti i protocolli di livello superiore (TCP, UDP) operando in modo del tutto trasparente per le applicazioni e gli utenti finali.

Oltre a creare un canale sicuro, IPsec funge da filtro avanzato per il traffico (comportandosi come un firewall a livello IP). Può essere implementato in due scenari:

1. **Sistemi End-System (Host/OS):** Installato sui singoli computer per proteggere la comunicazione diretta esclusivamente tra due macchine specifiche (end-to-end).

2. **Security Gateway (Router/Firewall):** Installato sugli apparati di confine per fare da "scudo" a un'intera rete locale. In questo caso protegge in blocco tutto il traffico in entrata e in uscita dalla sede (è il meccanismo base per creare le reti VPN aziendali).
-

STRUTTURA GENERALE IPSEC

IPsec non è un singolo protocollo, ma una suite composta da **quattro pilastri principali**:

1. **Architettura (Architecture):** È il "manuale generale". Copre i concetti di base, i requisiti di sicurezza e i meccanismi che definiscono la tecnologia IPsec nel suo complesso. (*Specifica attuale: RFC 4301*).
 2. **AH (Authentication Header):** È il protocollo che si occupa di fornire l'autenticazione del messaggio. In parole povere, garantisce che il mittente sia reale e che il pacchetto non sia stato modificato durante il tragitto (Integrità), ma **non** cifra il contenuto. È come un sigillo di ceralacca su una busta trasparente. (*Specifica attuale: RFC 4302*).
 3. **ESP (Encapsulating Security Payload):** È il protocollo che fa il lavoro pesante. Viene usato per fornire la cifratura (Confidentiality) o una combinazione di cifratura e autenticazione. Rende i dati completamente illeggibili a chiunque provi a intercettarli. È come mettere il tuo messaggio dentro una cassaforte blindata. (*Specifica attuale: RFC 4303*).
 4. **IKE (Internet Key Exchange):** È l'insieme di documenti che descrive gli schemi di gestione delle chiavi. Affinché AH ed ESP possano funzionare, i due nodi che comunicano devono prima mettersi d'accordo su quali chiavi segrete usare. L'IKE è il vigile che organizza questo scambio iniziale di chiavi in modo totalmente sicuro. (*Specifica principale: RFC 7296*).
-

LE POLICY DI IPsec: SA, SAD e SPD

IPsec gestisce il traffico attraverso un sistema di policy che decide, pacchetto per pacchetto, se e come applicare la sicurezza. Per farlo si basa su tre concetti

fondamentali che lavorano insieme: la **SA**, il **SAD** e lo **SPD**.

Security Association (SA)

Il primo mattone è la SA. Quando due host vogliono comunicare in modo sicuro, devono stabilire un accordo su come proteggere il traffico: questo accordo si chiama **Security Association**. È una **connessione logica unidirezionale** — poiché è unidirezionale, una comunicazione bidirezionale ($A \leftrightarrow B$) richiede due SA distinte. È B a scegliere i parametri per la SA in direzione $A \rightarrow B$.

Ogni SA è identificata univocamente da tre parametri:

SPI (Security Parameters Index): un intero a 32 bit che identifica la SA; ha significato solo locale ed è trasportato nell'header IPsec (AH o ESP).

IP Destination Address: l'indirizzo del destinatario — dove finisce il tunnel crittografato.

Security Protocol Identifier: indica se la SA usa AH o ESP (header esterno, in chiaro).

SAD (Security Association Database)

Tutte le SA attive devono essere memorizzate da qualche parte: è questo il ruolo del **SAD**. Quando arriva o parte un pacchetto IPsec, il sistema consulta il SAD per sapere come elaborarlo. I campi principali di ogni entry sono:

- **Security Parameters Index** — identifica univocamente la SA.
- **Sequence Number Counter** — contatore incrementato a ogni pacchetto inviato, serve per l'anti-replay.
- **Sequence Counter Overflow** — se il contatore esaurisce i numeri, blocca tutto per evitare di riusare lo stesso numero due volte.
- **Anti-Replay Window** — finestra scorrevole lato ricevitore per rilevare pacchetti duplicati o replay.
- **AH Information** — algoritmo di autenticazione, chiavi e parametri usati con AH.
- **ESP Information** — algoritmi di cifratura/autenticazione, chiavi e parametri usati con ESP.

- **Lifetime della SA** — dopo quanto tempo o quanti MB la chiave scade e ne serve una nuova.
 - **IPsec Protocol Mode** — tunnel, transport o wildcard.
 - **Path MTU** — dimensione massima del pacchetto che può transitare senza essere frammentato.
-

SPD (Security Policy Database)

Il SAD sa *come* proteggere, ma prima bisogna decidere *se* proteggere. Questo è il compito dello **SPD**: definisce le regole di filtraggio e risponde alla domanda "questo pacchetto va protetto, lasciato passare, o bloccato?". Quando arriva un pacchetto, lo SPD lo analizza tramite dei **selettori**:

- **Remote IP Address** — il destinatario; singolo indirizzo, lista, range o wildcard.
- **Local IP Address** — il mittente; stessa flessibilità.
- **Next Layer Protocol** — il protocollo di trasporto: TCP, UDP, ecc.
- **Name** — l'utente loggato nel sistema operativo (non è nell'header IP, disponibile solo se IPsec gira sullo stesso OS).
- **Local and Remote Ports** — le porte TCP/UDP da proteggere o meno; singoli valori, liste o wildcard.

In base ai selettori, la policy può avere tre esiti: **BYPASS** (passa in chiaro), **DISCARD** (bloccato), **PROTECT** (processato via IPsec — il sistema cerca una SA nel SAD, o ne crea una nuova tramite IKE).

Table 20.1 Host SPD Example

- SPD on a host system at 1.2.3.101; basic corporate network on IPs 1.2.3.0/24 and secure LAN DMZ on IPs 1.2.4.0/24

Protocol	Local IP	Port	Remote IP	Port	Action	Comment
UDP	1.2.3.101	500	*	500	BYPASS	IKE
ICMP	1.2.3.101	*	*	*	BYPASS	Error messages
*	1.2.3.101	*	1.2.3.0/24	*	PROTECT: ESP intransport-mode	Encrypt intranet traffic
TCP	1.2.3.101	*	1.2.4.10	80	PROTECT: ESP intransport-mode	Encrypt to server
TCP	1.2.3.101	*	1.2.4.10	443	BYPASS	TLS: avoid double encryption
*	1.2.3.101	*	1.2.4.0/24	*	DISCARD	Others in DMZ
*	1.2.3.101	*	*	*	BYPASS	Internet

Elaborazione dei pacchetti: Outbound e Inbound

Lo SPD è il punto di controllo obbligatorio per tutto il traffico IP: ogni pacchetto, in entrata o in uscita, viene confrontato con le sue regole e ottiene una risposta — passare in chiaro, essere bloccato, o essere protetto da IPsec.

SAD e SPD non sono solo database statici — vengono consultati attivamente ogni volta che un pacchetto entra o esce dal sistema. Il flusso di elaborazione è diverso a seconda della direzione.

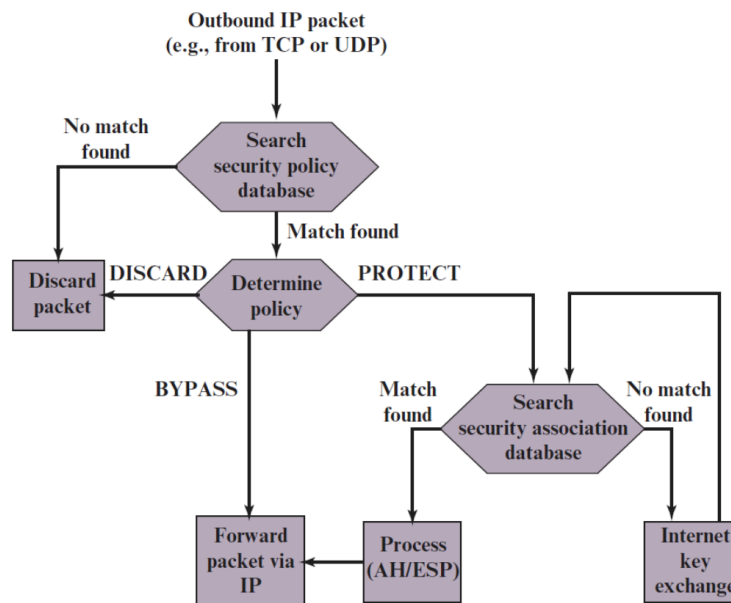
Outbound (pacchetto in uscita):

Il pacchetto parte, IPsec lo intercetta e segue questi step:

1. Cerca nell'**SPD** una regola che corrisponda a quel pacchetto — se non trova nulla lo scarta.
2. Se trova una regola, legge l'azione (**Determine Policy**):
 - **BYPASS** → manda il pacchetto in chiaro
 - **DISCARD** → scarta

- **PROTECT** → cerca la SA nel **SAD**. Se la trova processa con AH/ESP, se non la trova chiama **IKE** per crearne una nuova.

Outbound Packets

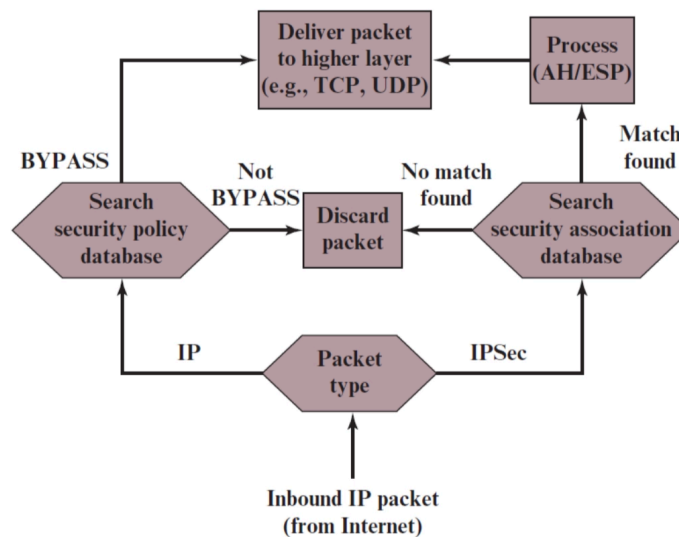


Inbound (pacchetto in arrivo):

Arriva un pacchetto, IPsec guarda prima il tipo:

- **Pacchetto IP normale** → cerca nell'**SPD**, se dice BYPASS lo consegna al livello superiore, altrimenti scarta.
- **Pacchetto IPsec** (ha header AH o ESP) → cerca nel **SAD** tramite l'SPI. Se trova la SA processa (decifra/verifica) e consegna, se non la trova scarta.

Inbound Packets

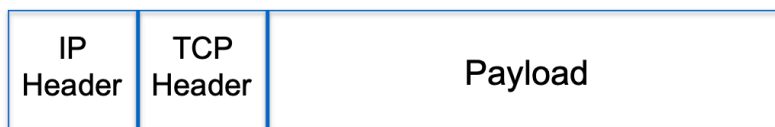


TUNNEL VS TRANSPORT MODE

IPsec opera in due modalità: in **Transport Mode** l'header IP originale resta intatto e viene solo inserito un header IPsec tra IP e TCP — il routing non cambia, il pacchetto è solo leggermente più grande, usato tipicamente per connessioni end-to-end tra due host. In **Tunnel Mode** invece l'intero pacchetto originale viene cifrato e incapsulato in un nuovo pacchetto con un nuovo header IP — il routing cambia, nessun router intermedio vede il pacchetto originale, usato tipicamente per VPN site-to-site tra gateway.

Transport Mode lo usi quando vuoi proteggere il contenuto ma non hai bisogno di nascondere chi parla con chi — tipicamente due host che si conoscono già e comunicano direttamente. Oggi TLS copre già la maggior parte di questi casi, quindi transport mode ha senso principalmente per protocolli legacy senza cifratura propria o in contesti dove non puoi toccare le applicazioni ma vuoi comunque garantire sicurezza a livello di rete.

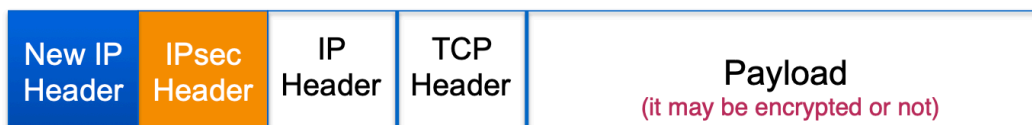
Tunnel Mode lo usi quando vuoi nascondere completamente il traffico interno, inclusi gli indirizzi reali — tipicamente due gateway che proteggono la comunicazione per conto di intere reti, senza che i singoli host debbano fare nulla.



Without IPsec



Transport Mode
IPsec



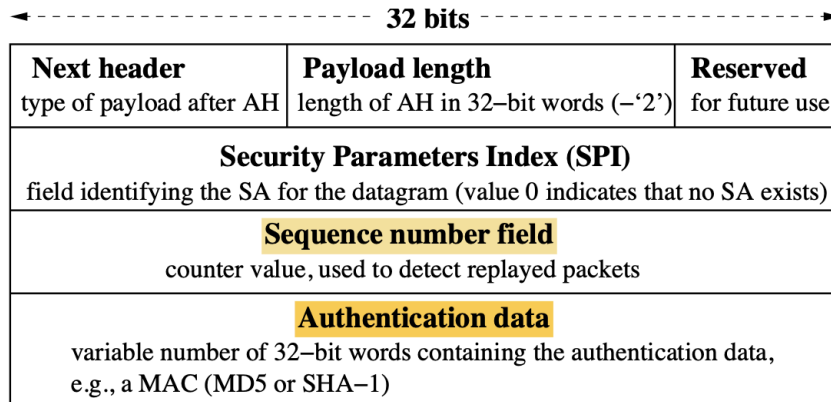
Tunnel Mode
IPsec

Il Protocollo AH (Authentication Header)

AH è uno dei due protocolli operativi di IPsec (l'altro è ESP). Il suo scopo è fornire **Integrità, Autenticazione** e protezione **Anti-Replay**.

- **Regola d'oro di AH: NON cifra i dati** (non offre Confidentiality). Il contenuto del pacchetto viaggia in chiaro ed è leggibile da chiunque lo intercetti. AH aggiunge semplicemente un "sigillo" crittografico per garantire che nessuno lo abbia modificato e che il mittente sia autentico.
- **Posizionamento:** Viene inserito come un header extra ("in mezzo") tra l'header IP (Livello 3) e l'header TCP/UDP (Livello 4).

Com'è fatto l'Header AH (I 3 campi fondamentali):

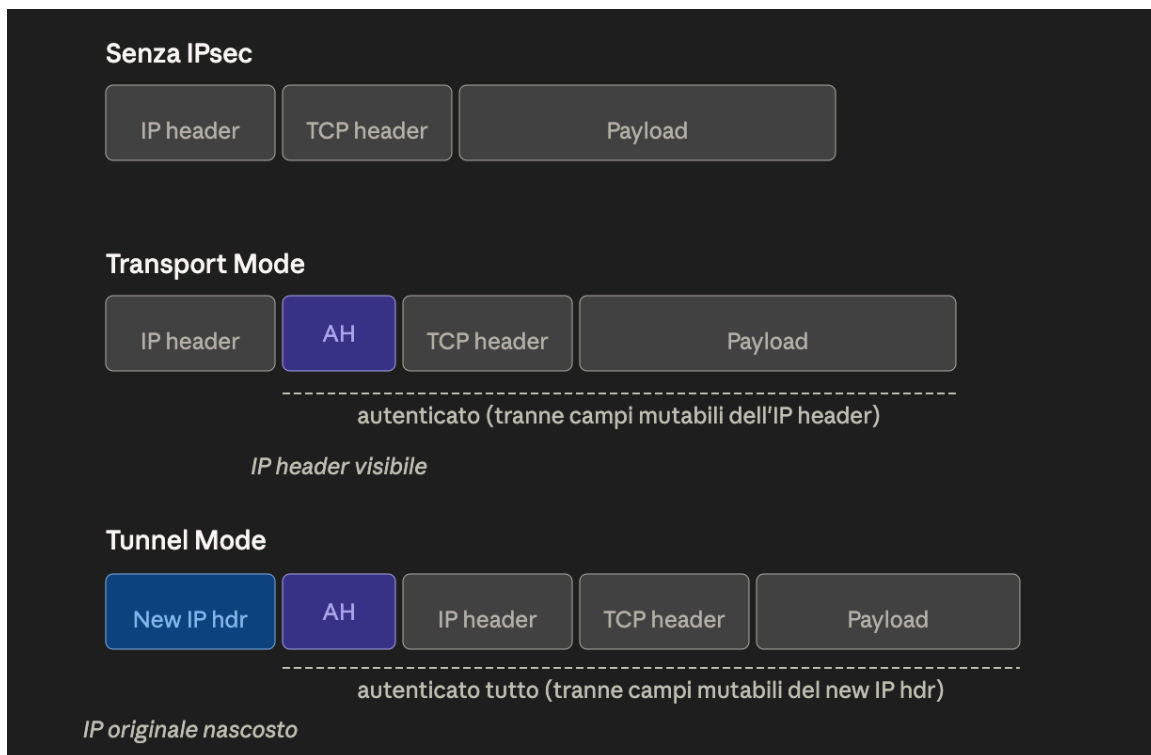


1. **SPI (Security Parameters Index):** È un numero identificativo. Dice al computer ricevente quale "contratto" (la Security Association nel database SAD) deve andare a leggere per sapere quali chiavi e algoritmi usare per verificare quel pacchetto.
2. **Sequence Number (Numero di sequenza):** È un contatore che parte da zero e viene incrementato di 1 dal mittente per ogni nuovo pacchetto spedito.
 - *Meccanismo Anti-Replay:* Poiché il protocollo IP standard non garantisce l'ordine di arrivo dei pacchetti, il ricevitore usa questo numero insieme a una **finestra mobile (sliding window)**, tipicamente grande 64 pacchetti. Se arriva un pacchetto con un numero di sequenza troppo vecchio o già visto nella finestra, viene scartato immediatamente, bloccando i *Replay Attack*.
3. **Authentication Data (MAC):** È la vera e propria "firma" matematica del pacchetto (calcolata usando algoritmi come MD5 o SHA-1).

La particolarità di AH: A differenza di altri sistemi, il calcolo della firma di AH (il MAC) copre non solo i dati trasportati (payload), ma **protegge anche le parti "fisse" (immutabili) dell'header IP originale**, come ad esempio gli indirizzi IP sorgente e destinazione. Questo garantisce in modo assoluto che nessuno possa falsificare l'IP del mittente durante il transito.

AH MODES (la parte di IPsec Header)

A seconda della modalità scelta, l'header AH viene posizionato in punti diversi del pacchetto e calcola la sua "firma" (MAC) su porzioni diverse.



1. AH in Transport Mode (Modalità Trasporto)

- **Dove si posiziona:** Viene infilato esattamente in mezzo, dopo l'header IP originale e prima dei dati veri e propri (il payload).
- **Cosa autentica:** Prende l'impronta (MAC) di **tutto** il pacchetto. Esclude solo i "campi mutabili" dell'header IP, ovvero quei valori che cambiano per forza mentre il pacchetto viaggia fisicamente sui router (come il parametro TTL).
- **Scopo:** Fornire protezione diretta end-to-end tra due sistemi.

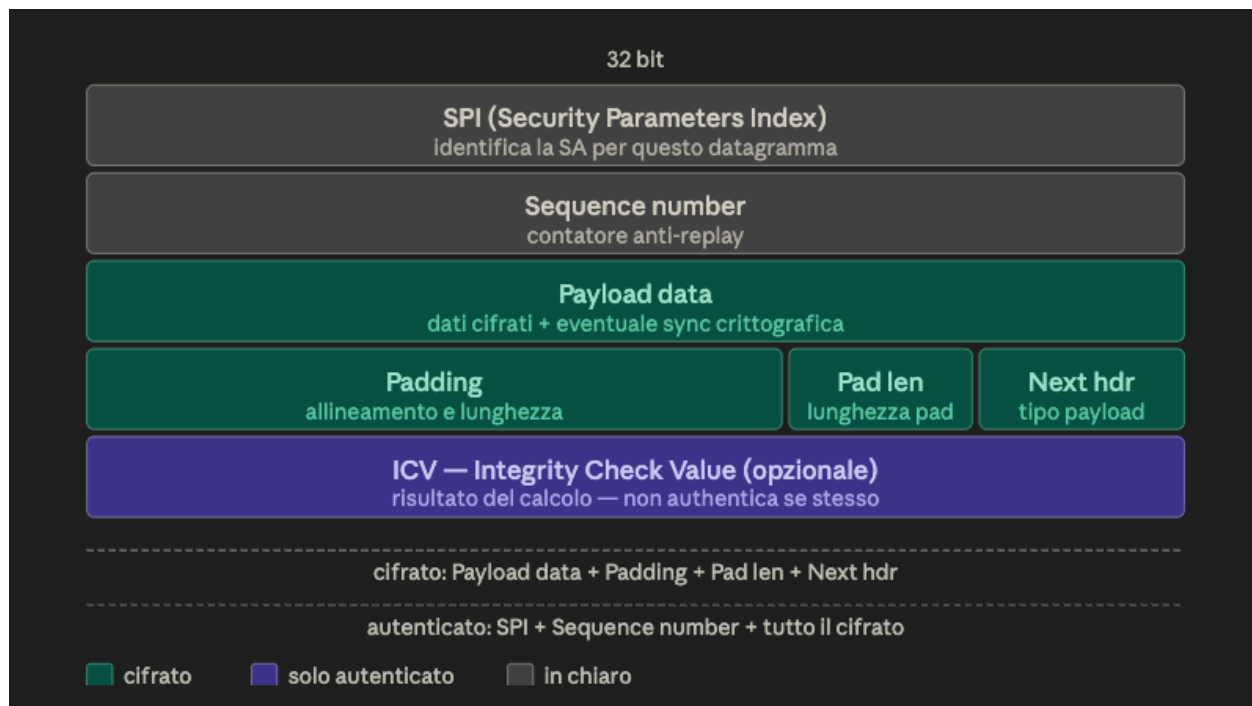
2. AH in Tunnel Mode (Modalità Tunnel)

- **Dove si posiziona:** Viene creato un **nuovo** header IP esterno. L'header AH viene inserito subito dopo questo nuovo header esterno, proteggendo e incapsulando il pacchetto IP originale.
- **Cosa autentica:** Autentica assolutamente **tutto**: l'intero pacchetto interno originale e anche il nuovo header IP esterno (sempre escludendo i campi mutabili del nuovo header).
- **Scopo:** L'header interno mantiene gli indirizzi IP originali della sorgente e della destinazione finale. Il nuovo header esterno, invece, contiene gli indirizzi IP dei

dispositivi intermedi, come firewall o gateway di sicurezza.

Il protocollo ESP (Encapsulating Security Payload)

Il protocollo **ESP**, la cui specifica attuale è definita nel documento RFC 4303, è progettato per fornire primariamente il servizio di **confidenzialità (cifratura)** e, in modo opzionale, **i servizi di integrità e autenticazione dei dati**. A differenza di **AH**, che si occupa di **autenticazione e integrità**, ESP avvolge il pacchetto in una struttura composta da un'intestazione (Header) e una coda (Trailer), crittografando il carico utile.



Come illustrato nello schema strutturale, i campi operativi di ESP si dividono in tre categorie di visibilità: in chiaro, cifrati e solo autenticati.

1. Struttura del Datagramma ESP e Analisi dei Campi

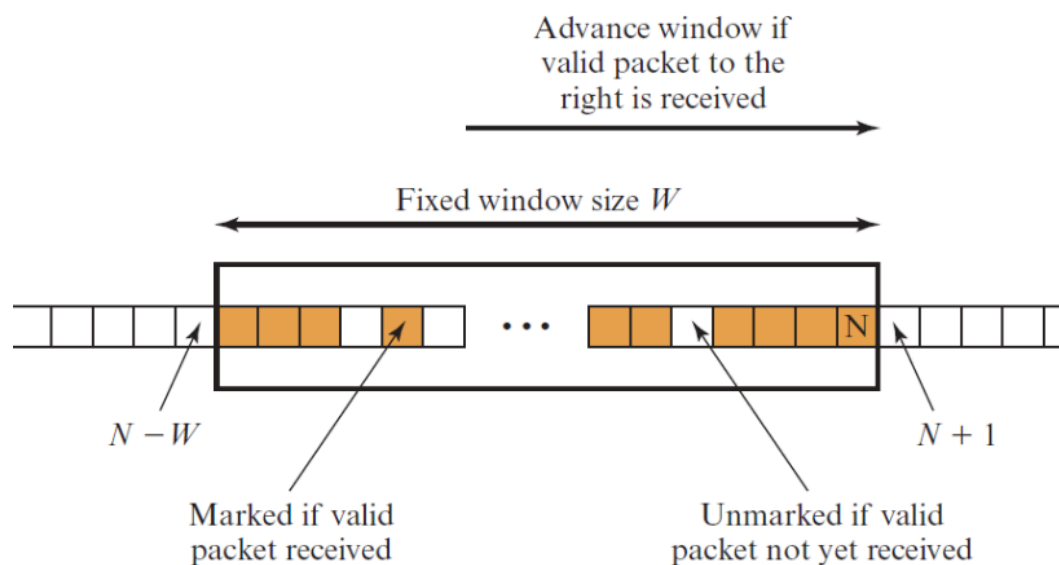
- **SPI (Security Parameters Index):** Campo a 32 bit trasmesso in chiaro. Identifica in modo univoco la Security Association (SA) di riferimento per elaborare correttamente il datagramma.

- **Sequence Number:** Campo a 32 bit in chiaro. Contiene un valore contatore essenziale per l'implementazione del meccanismo anti-replay.
 - **Payload Data (Dati trasportati):** Contiene i dati originali protetti. Se l'algoritmo impiegato lo richiede, eventuali dati di sincronizzazione crittografica vengono inseriti esplicitamente all'inizio di questo campo.
 - **ESP Trailer (Padding, Pad Length, Next Header):** Questa sezione terminale del blocco cifrato svolge funzioni strutturali critiche:
 - **Padding:** Viene inserito per tre scopi precisi:
 1. *Espansione del plaintext:* Se un algoritmo di cifratura richiede che il testo in chiaro sia un multiplo esatto di un determinato numero di byte, il padding lo porta alla dimensione richiesta.
 2. *Allineamento:* Assicura il corretto posizionamento in memoria dei campi Pad Length e Next Header.
 3. *Partial traffic-flow confidentiality:* Può essere aggiunto padding addizionale per occultare la lunghezza effettiva del payload, mitigando i tentativi di analisi del traffico.
 - **Pad Length:** Specifica la lunghezza esatta del campo padding appena inserito.
 - **Next Header:** Indica il tipo di protocollo incapsulato nel payload (es. TCP).
 - **ICV (Integrity Check Value):** Campo opzionale, presente esclusivamente se viene richiesto il servizio di integrità.
 - *Caratteristica fondamentale:* L'ICV viene calcolato matematicamente **dopo** che è stata eseguita la cifratura sui campi precedenti. Questo ordine di elaborazione è stato progettato per facilitare la mitigazione degli attacchi DoS (Denial of Service).
 - Poiché non è protetto da cifratura, deve essere calcolato utilizzando un algoritmo di integrità con chiave (keyed integrity algorithm).
-

2. Il Meccanismo Anti-Replay

Poiché il protocollo di rete IP è *connectionless* (senza connessione), non vi è garanzia che i pacchetti vengano consegnati e, soprattutto, che vengano consegnati in ordine. La ricezione di pacchetti IP duplicati e autenticati potrebbe compromettere il sistema. Per sventare questi attacchi, ESP implementa un controllo sul Sequence Number basato su una "finestra mobile" di dimensione fissa W :

1. Alla creazione di una nuova SA, il mittente inizializza il contatore a 0 e lo incrementa per ogni nuovo pacchetto inviato.
2. Il ricevitore mantiene una finestra di validità. Quando riceve un pacchetto, valuta la posizione del suo Sequence Number:
 - **Dentro la finestra:** Il pacchetto viene verificato crittograficamente (MAC) e, se valido, il suo posizionamento nella finestra viene marcato.
 - **A destra della finestra:** Il pacchetto è nuovo. Viene verificato e, in caso di successo, la finestra trasla (shifta) verso destra.
 - **A sinistra della finestra:** Il pacchetto è considerato vecchio o duplicato. Viene immediatamente scartato. Qualsiasi pacchetto la cui autenticazione fallisca viene parimenti scartato.

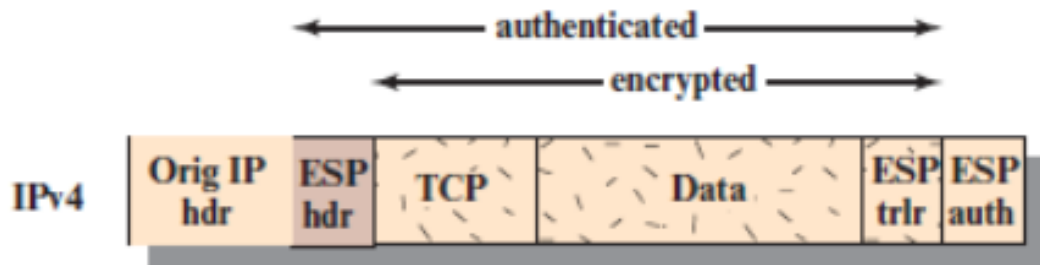
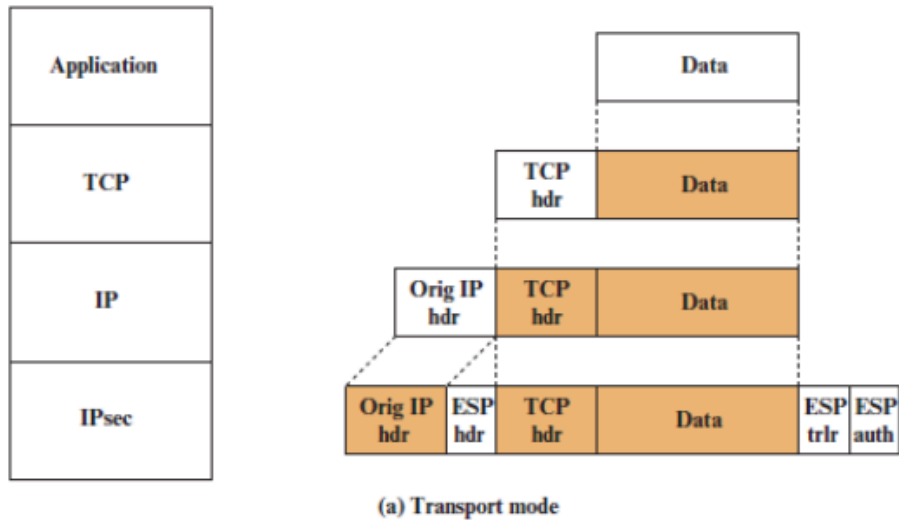


3. Modalità Operative: Transport Mode vs Tunnel Mode

Il raggio d'azione (Scope) della protezione offerta da ESP varia drasticamente a seconda della modalità implementata.

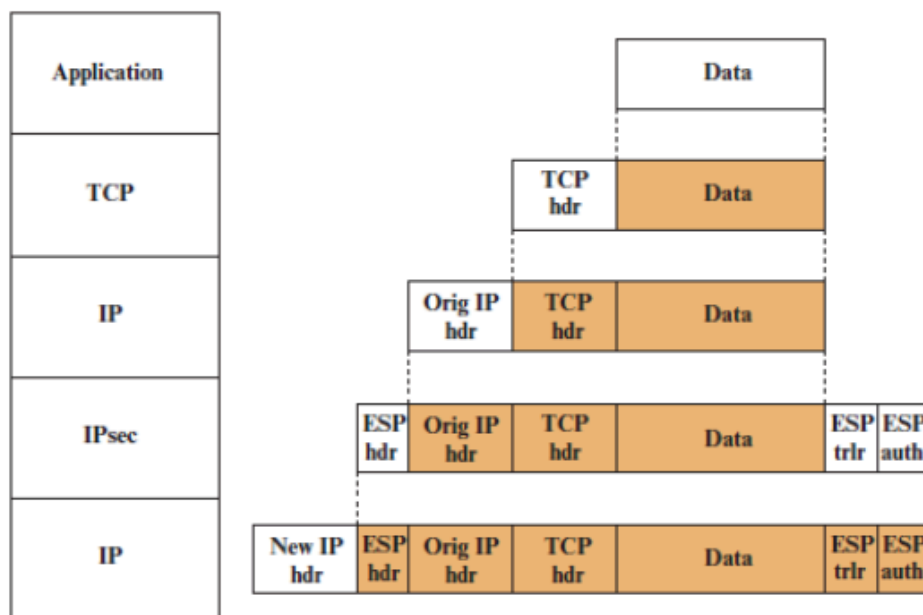
A. Transport Mode (Modalità di Trasporto)

- **Operatività:** Al nodo sorgente, viene cifrato unicamente il blocco composto dal livello di trasporto (es. TCP) e dall'ESP trailer. Il testo in chiaro originale viene sostituito da questo ciphertext e l'header IP originale viene mantenuto perfettamente intatto e in chiaro.
- **Routing e Raggio d'Azione:** I router lungo il percorso leggono l'header IP originale per l'instradamento, senza analizzare i dati cifrati. L'autenticazione (se presente) copre l'header ESP e tutto il ciphertext, ma **esclude totalmente l'header IP originale**, che rimane in chiaro e non protetto.
- **Vantaggi e Svantaggi:** Fornisce confidenzialità end-to-end in modo trasparente per le applicazioni. Il difetto principale è che, esponendo gli indirizzi IP originali, rende la comunicazione vulnerabile alla *traffic analysis*.
- **Gestione delle dimensioni del pacchetto:** Il Transport Mode funziona bene nelle reti in cui l'aumento delle dimensioni di un pacchetto potrebbe causare un problema. Questo è dovuto al fatto che non viene creato alcun pacchetto nuovo, ma l'header IPsec viene semplicemente inserito all'interno del pacchetto IP esistente.
- **Scenario d'uso tipico:** Questa modalità viene utilizzata frequentemente per le VPN ad accesso remoto (remote-access VPNs), in scenari in cui non vi è alcuna necessità di nascondere il traffico.

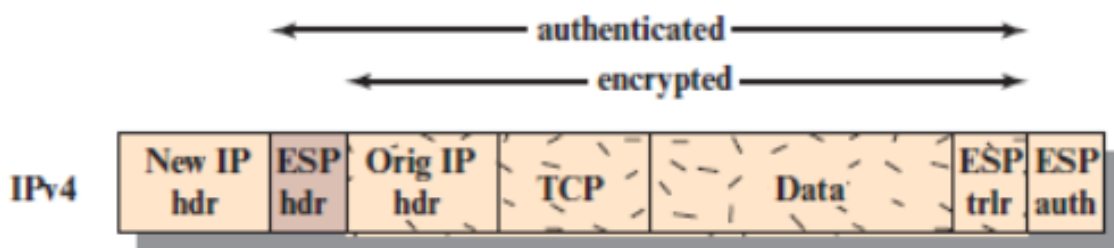


B. Tunnel Mode (Modalità Tunnel)

- **Operatività:** L'intero datagramma IP originale (Header IP originale + dati) viene trattato come un unico blocco dati e completamente cifrato.
- **Incapsulamento e Raggio d'Azione:** Questo blocco cifrato viene incapsulato all'interno di un **nuovo pacchetto IP**, dotato di un "New IP Header" in chiaro. L'autenticazione copre l'header ESP e l'intero pacchetto IP originale cifrato, ma **esclude il New IP Header**.
- **Routing e Sicurezza:** Il traffico viaggia attraverso reti esterne sfruttando gli indirizzi del nuovo header (spesso appartenenti a Security Gateway o Firewall di confine). Nessun router intermedio può leggere l'header IP originario, garantendo l'occultamento totale della topologia di rete interna e delle identità dei nodi comunicanti.



(b) Tunnel mode



Integrazioni per il Tunnel Mode

vantaggi dell'uso del Tunnel Mode quando viene implementato sui Security Gateway (firewall o router di confine):

- **Trasparenza per i dispositivi interni:** Con il Tunnel Mode, una moltitudine di host situati su reti interne protette da firewall può ingaggiare comunicazioni sicure senza dover implementare IPsec a bordo delle singole macchine.
- **Delega dell'elaborazione:** I pacchetti non protetti generati dagli host interni vengono instradati attraverso le reti esterne da Security Association (SA) in modalità tunnel; queste SA sono configurate e gestite esclusivamente dal software IPsec del firewall o del router di confine. La cifratura, infatti, avviene solo tra un host esterno e il security gateway, oppure tra due security gateway.

- **Riduzione del carico computazionale (Processing Burden):** Questo specifico setup architetturale solleva (relieves) gli host della rete interna dal pesante onere elaborativo richiesto dalle operazioni di cifratura.
 - **Semplificazione della gestione chiavi:** Demandando il lavoro ai gateway, si semplifica enormemente l'operazione di distribuzione delle chiavi (key distribution task), in quanto viene ridotto il numero di chiavi totali necessarie nel sistema.
 - **Mitigazione mirata:** Viene affermato esplicitamente che il Tunnel Mode sventa (thwarts) le attività di traffic analysis basate sulla destinazione finale (ultimate destination).
-

VPN (Virtual Private Network)

IPsec e VPN non sono la stessa cosa. **IPsec è il protocollo crittografico** (il lucchetto matematico), mentre la **VPN è l'intera infrastruttura di rete** che usa quel protocollo per comunicare in sicurezza.

Una VPN (Virtual Private Network) serve a creare una rete privata passando sopra una rete pubblica come Internet. L'obiettivo è sfruttare un'infrastruttura già esistente ed economica (economie di scala) mantenendo però la stessa sicurezza di una rete aziendale chiusa.

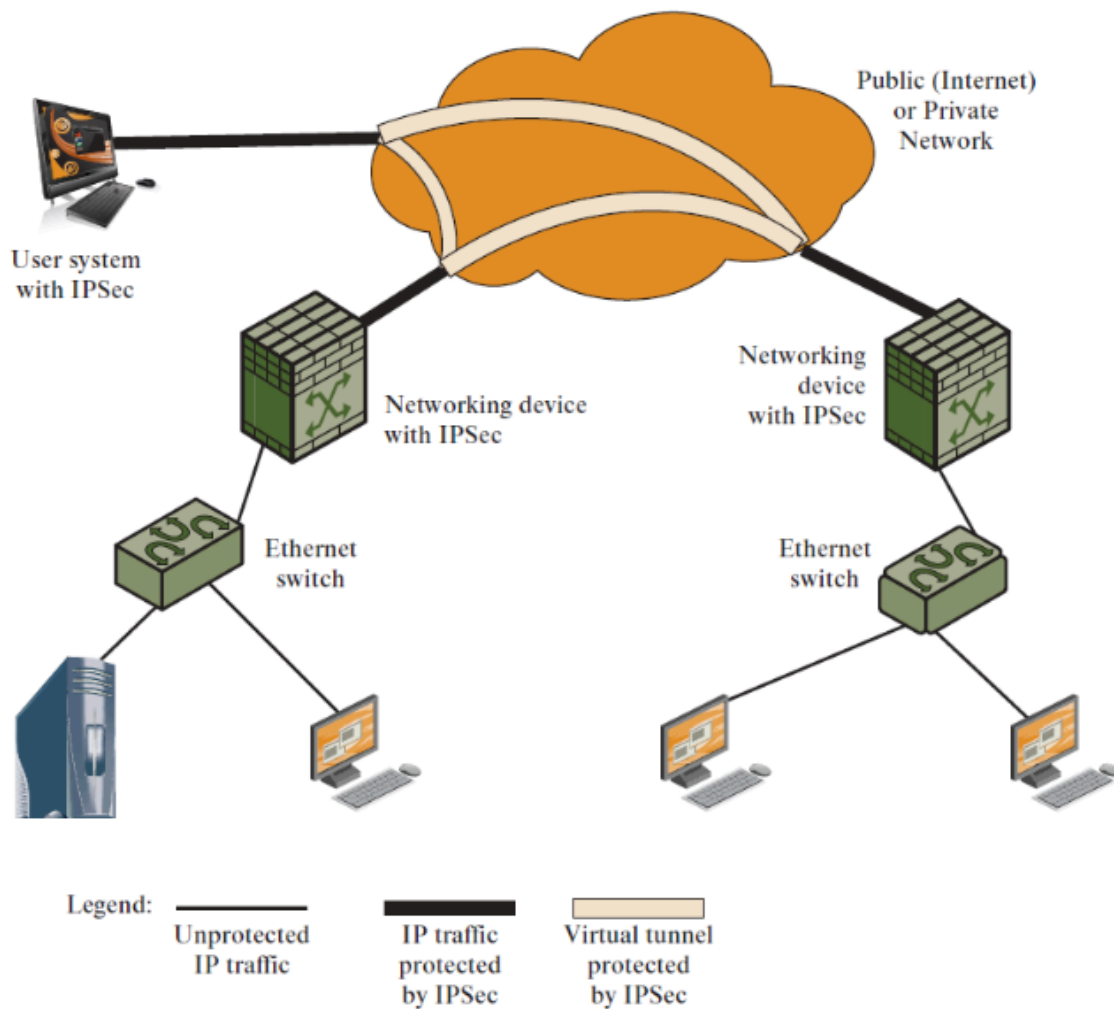
Oltre alla cifratura vera e propria (fatta da IPsec in Tunnel Mode), una VPN si occupa di tre cose pratiche:

1. Segregazione Logica del Traffico: Sebbene i dati viaggino fisicamente sull'infrastruttura pubblica condivisa, il traffico VPN è logicamente isolato. Il sistema impone un instradamento chiuso: i pacchetti transitano esclusivamente all'interno del tunnel protetto da una sorgente a una destinazione della medesima VPN, senza mai essere esposti in chiaro.

2. Gestione Infrastrutturale (Management Facilities): La VPN espande la crittografia di IPsec aggiungendo i servizi necessari per amministrare la rete (es. controllo accessi e IP privati). Questo permette di scalare l'architettura per connettere filiali distanti (site-to-site) o fornire accessi remoti (dial-up) agli utenti.

3. Ruolo del Security Gateway: Poiché la crittografia richiede elevate risorse computazionali, i tunnel IPsec vengono terminati da apparati hardware dedicati

posizionati ai confini della rete aziendale (Security Gateway). Questo accentramento solleva i normali host della rete interna dal pesante onere elaborativo (processing burden).



IKE (Internet Key Exchange)

Lo scopo di IKE è negoziare e distribuire le chiavi crittografiche necessarie per le Security Association (SA) di IPsec. Tipicamente servono 4 chiavi per una comunicazione bidirezionale (trasmissione/ricezione × integrità/confidenzialità).

IKE si basa su **ISAKMP/Oakley**:

- **Oakley**: protocollo di scambio chiavi basato su Diffie-Hellman, con sicurezza aggiuntiva
 - **ISAKMP**: fornisce il framework e i formati dei messaggi per la negoziazione
-

LE 2 FASI DI IKE

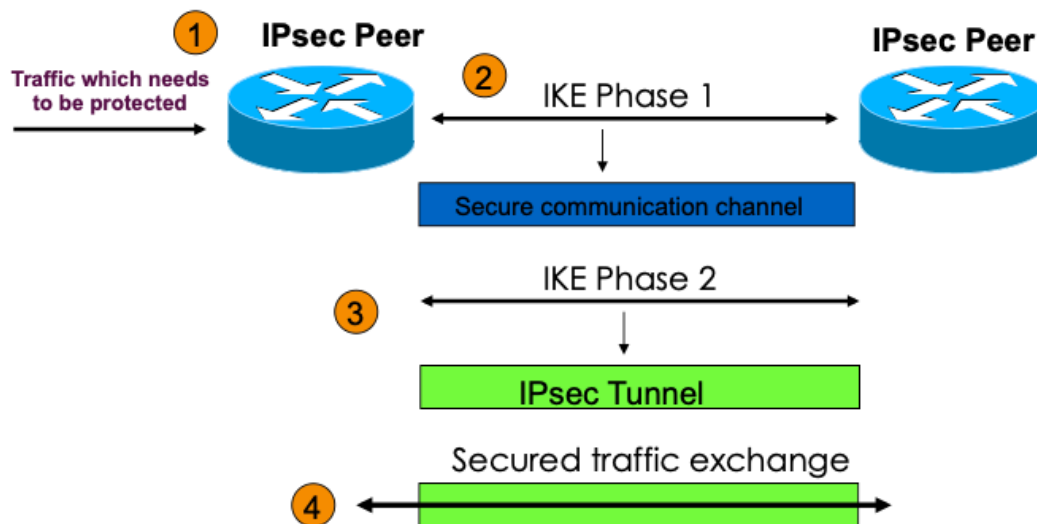
Fase 1 — Main Mode (crea il canale sicuro tra i peer):

1. Negoziazione della policy IKE (algoritmi da usare)
2. Scambio DH autenticato (scambio delle chiavi pubbliche + nonce)
3. Protezione dell'identità dei peer (messaggi 5 e 6 cifrati)

Risultato → **IKE SA**: canale sicuro su cui fare la fase 2.

Fase 2 — Quick Mode (crea il tunnel IPsec vero e proprio):

- Tutto il traffico è cifrato con l'IKE SA della fase 1
- Ogni negoziazione produce **due IPsec SA** (una inbound, una outbound)
- Crea/rinnova le chiavi per IPsec

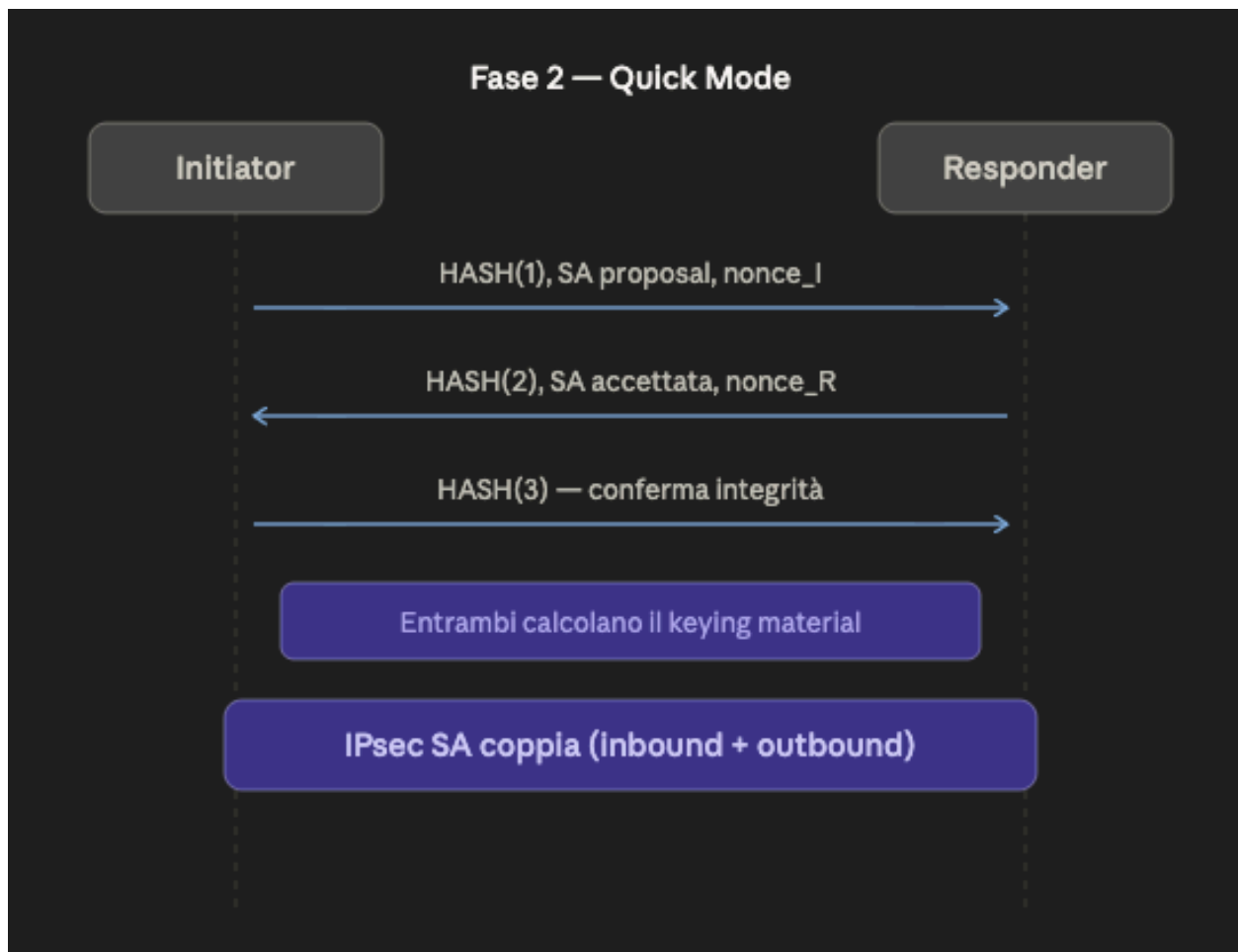


5 caratteristiche chiave dell'algoritmo IKE:

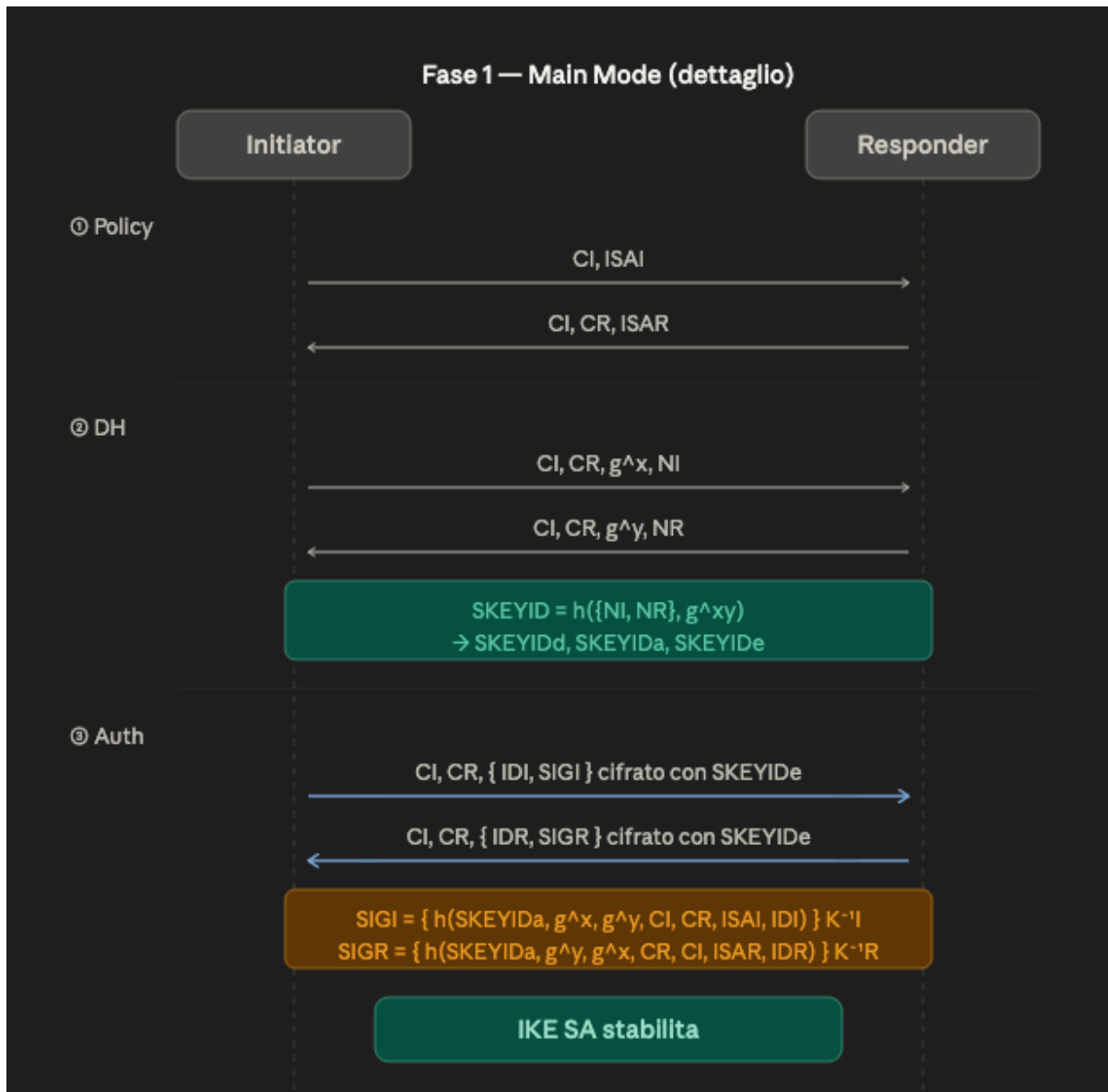
1. **Cookies** — per prevenire attacchi di clogging (Dos)
2. **Negoziazione del gruppo DH** (parametri globali)
3. **Nonce** — contro i replay attack
4. **Scambio valori pubblici** Diffie-Hellman → Perfect Forward Secrecy
5. **Autenticazione** dello scambio DH (contro man-in-the-middle)

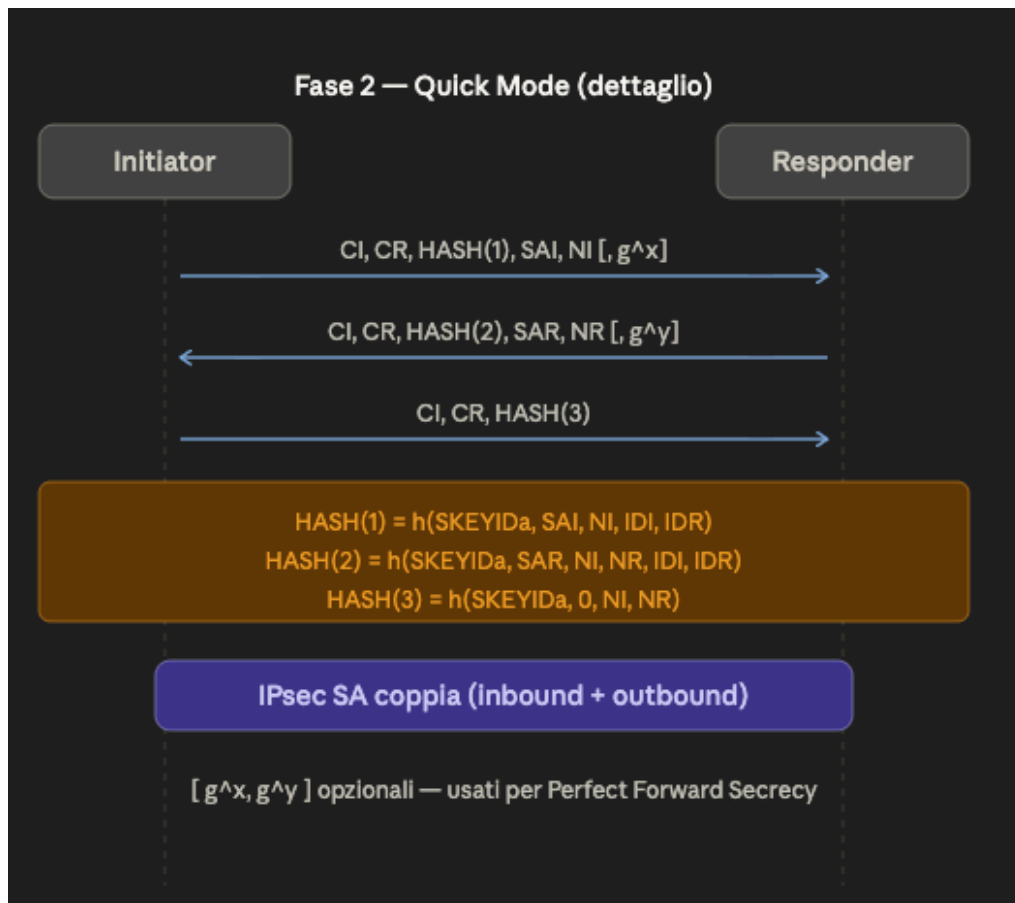
PROTOCOLLO IKE (semplice)



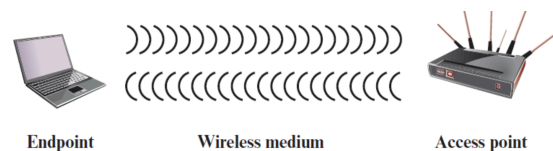


PROTOCOLLO IKE (completo)





WIRELESS LAN



Quattro fattori chiave rendono le reti wireless più vulnerabili di quelle cablate:

- **Canale** → le trasmissioni sono broadcast (come urlare in una piazza): chiunque nelle vicinanze può "ascoltare" o disturbare il segnale.

- **Mobilità** → i dispositivi si spostano, e questo crea nuovi rischi (connessioni indesiderate, perdita di controllo).
- **Risorse limitate** → smartphone e tablet hanno meno memoria/potenza per difendersi da malware e attacchi DoS.
- **Accessibilità fisica** → sensori e robot spesso sono incustoditi in luoghi remoti, vulnerabili ad attacchi fisici.

Tipologie di attacchi alle reti wireless

Attacco	Descrizione semplice
Accidental association	Ti connetti per sbaglio alla rete del vicino
Malicious association	Un attaccante finge di essere un access point legittimo (AP fasullo)
Ad hoc networks	Reti peer-to-peer senza controllo centrale = nessuna sicurezza centralizzata
MAC spoofing	L'attaccante ruba l'indirizzo MAC di un dispositivo autorizzato
Man-in-the-middle	L'attaccante si inserisce tra te e l'AP, intercettando tutto
DoS	L'attaccante bombarda l'AP di messaggi fino a renderlo inutilizzabile
Network injection	Iniezione di messaggi falsi (es. routing) in AP non filtrati

Come difendersi dalle minacce wireless

Contromisura	Descrizione semplice
Nascondere l'SSID	Non trasmettere il nome della rete rende più difficile individuarla
Ridurre potenza segnale	Meno segnale fuori dall'edificio = meno superficie esposta ad ascolti esterni
Posizionare AP lontano da finestre/pareti	Il segnale non "esce" facilmente dall'edificio
Cifrare il traffico	Anche se qualcuno ascolta, non capisce nulla senza la chiave

Contromisura	Descrizione semplice
IEEE 802.1X	Controlla ogni porta di accesso alla rete, blocca AP fasulli e dispositivi non autorizzati
Cambiare password di default	Le password di fabbrica sono pubblicamente note e facilmente sfruttabili
Firewall / Antivirus / Antispyware	Barriere software che filtrano il traffico e bloccano software malevolo
Whitelist MAC	Solo i dispositivi con indirizzo MAC registrato possono connettersi

Sicurezza dei dispositivi mobili (BYOD)

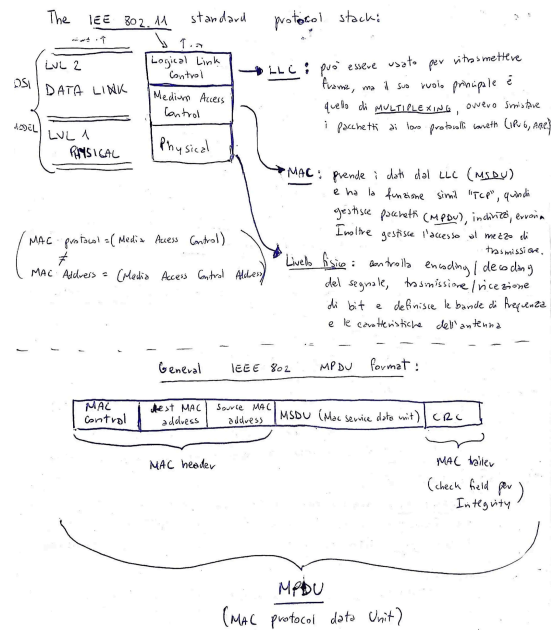
Prima degli smartphone la sicurezza era semplice: c'era un perimetro netto tra rete interna (fidata) e Internet (non fidato). Ora non funziona più così perché le organizzazioni devono gestire dispositivi personali, cloud, ospiti esterni.

tendenzialmente o si ha una white-list dei dispositivi che hanno il permesso di connettersi in rete, oppure si danno dei dispositivi (mobile, tablet, pc) controllati dell'azienda e limitati.

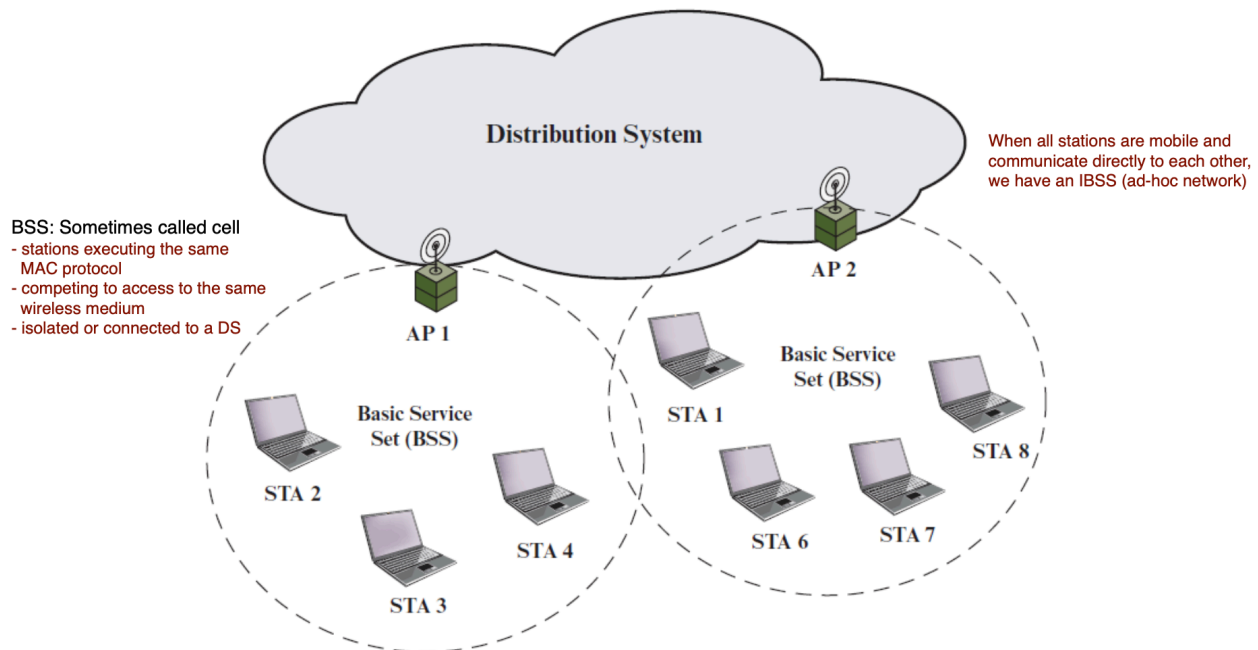
Standard IEEE 802.11 (WiFi)

L'IEEE 802.11 è lo standard che definisce le reti wireless (WiFi).

IEEE 802.11 Procol Stack:



I componenti fondamentali sono:



- **STA (Station)** → qualsiasi dispositivo con scheda WiFi
- **AP (Access Point)** → il "router" che gestisce le connessioni wireless
- **BSS (Basic Service Set)** → una cella = un AP + tutte le STA collegate

- **ESS (Extended Service Set)** → più BSS collegate tramite un Distribution System (DS)
- **DS (Distribution System)** → la rete backbone (tipicamente cablata) che collega gli AP tra loro

La mobilità delle STA si classifica in tre tipi: restano nella stessa BSS (no transition), si spostano tra BSS della stessa ESS (BSS transition), oppure cambiano ESS completamente con possibile perdita di connessione (ESS transition).

IEEE 802.11i – La sicurezza WiFi (WPA/WPA2/WPA3)

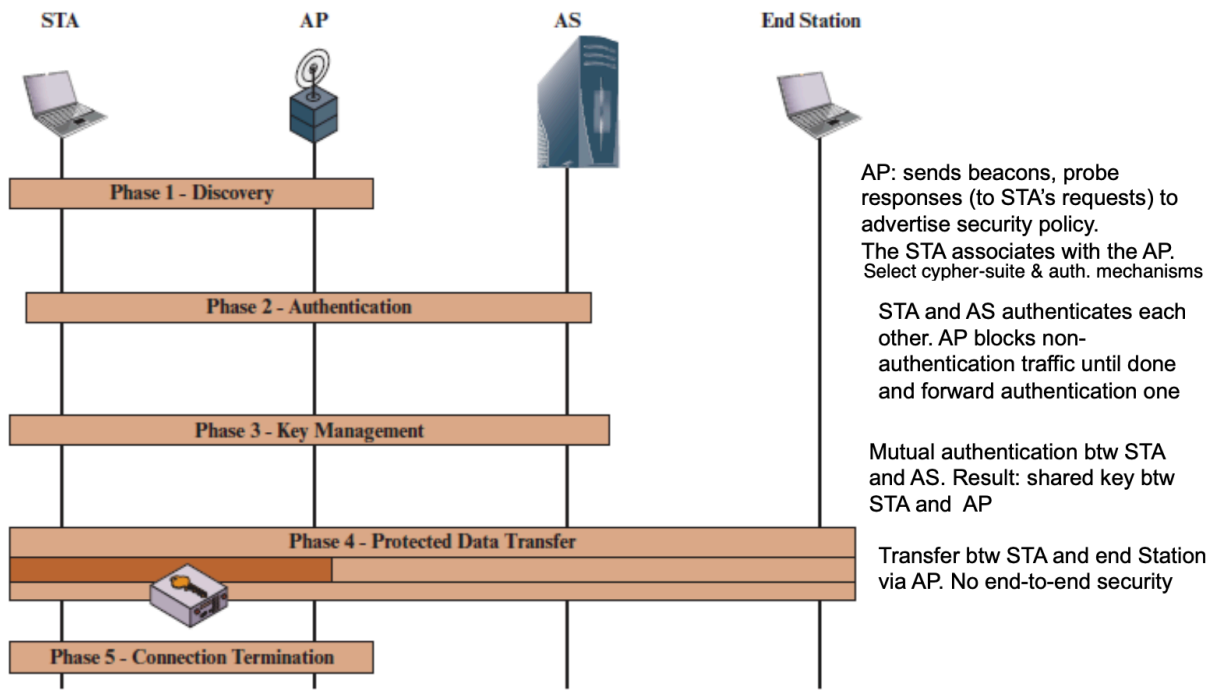
Evoluzione storica della sicurezza WiFi:

- **WEP** → primo tentativo, pieno di vulnerabilità critiche, abbandonato
- **WPA** → miglioramento intermedio basato su 802.11i
- **RSN / WPA2 / WPA3** → standard definitivo (802.11i del 2012), molto robusto

RSN offre controllo accesso, autenticazione/gen. chiavi, protezione dati, implementati da due protocolli alternativi: **TKIP** (vecchio, RC4) e **CCMP** (moderno, AES) — oggi si usa solo CCMP.

Le 5 fasi operative di 802.11i

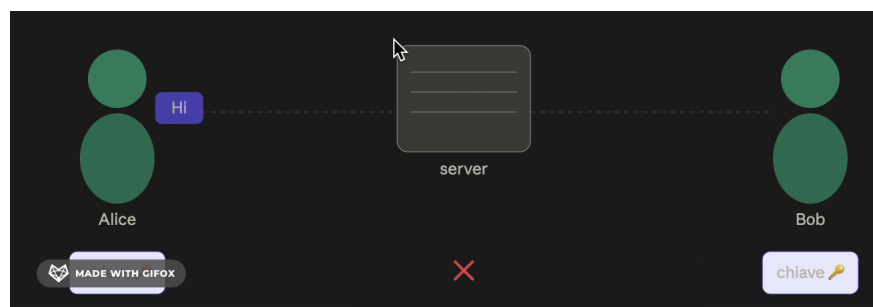
Queste fasi descrivono **cosa succede ogni volta che un dispositivo (STA) si connette a un AP protetto da WPA2**. È un processo sequenziale e obbligatorio.



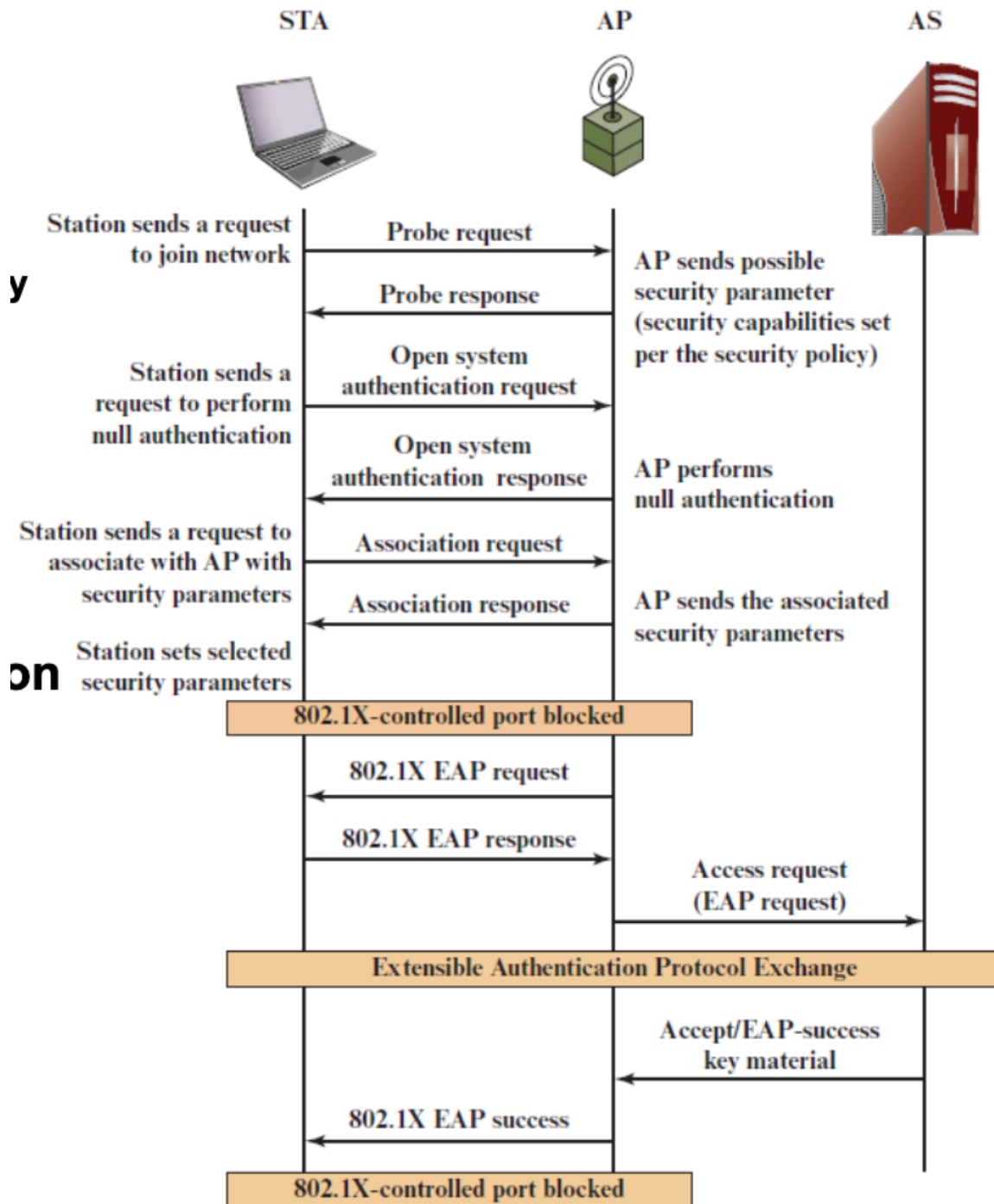
1. **Discovery** → l'AP annuncia le sue capacità di sicurezza; la STA si associa scegliendo i parametri condivisi
2. **Authentication** → autenticazione reciproca tra STA e Authentication Server (AS) tramite **EAP** (Extensible Authentication Protocol); l'AP fa da "ponte" e blocca tutto il traffico non-auth
3. **Key Management** → generazione e distribuzione delle chiavi crittografiche
4. **Protected Data Transfer** → trasmissione dati cifrata e autenticata
5. **Connection Termination** → chiusura sicura della sessione

Note

La crittografia NON è End-to-End



Passaggi più nel Dettaglio:



Fase 1 – Discovery

L'AP trasmette continuamente dei **beacon**, cioè messaggi broadcast che annunciano la sua presenza e le sue **capacità di sicurezza** (quali cifrari supporta, quale metodo di autenticazione usa). La STA può anche fare una **probe request** per cercare attivamente reti disponibili.

Una volta trovato l'AP, la STA esegue una **open system authentication** (sostanzialmente uno scambio di identità senza vera verifica — è un residuo di compatibilità con il vecchio 802.11) e poi invia una **association request** con i parametri di sicurezza scelti. Se AP e STA non hanno parametri compatibili, la connessione viene rifiutata. A questo punto la STA è associata ma non ancora autenticata: le porte controlled sono bloccate.

Fase 2 – Authentication

Questa è la fase più importante. L'obiettivo è che **STA e Authentication Server (AS) si autenticano reciprocamente**, cioè entrambi devono dimostrare di essere chi dicono di essere.

L'AS è tipicamente un server separato sulla rete cablata (es. un server RADIUS). L'AP in questa fase fa solo da "**ponte**" (relay): riceve i messaggi EAP dalla STA e li inoltra all'AS, senza partecipare all'autenticazione vera e propria. Blocca tutto il traffico non-EAP fino al completamento.

Il protocollo usato è **EAP (Extensible Authentication Protocol)**, che non è un singolo metodo ma una famiglia di metodi (EAP-TLS, EAP-TTLS, PEAP, ecc.) che permettono diversi meccanismi di autenticazione (certificati, password, token, ecc.).

Al termine, l'AS invia alla STA una **Master Session Key (MSK)**, da cui deriveranno tutte le chiavi successive. L'AP viene notificato del successo e sblocca le porte.

Fase 3 – Key Management

Dopo l'autenticazione, STA e AP devono **concordare le chiavi crittografiche** da usare per proteggere il traffico. Questo avviene tramite il cosiddetto **4-way handshake**, uno scambio di quattro messaggi tra STA e AP che serve a:

- Confermare che entrambi possiedono la stessa PMK
- Generare la PTK (Pairwise Transient Key) da usare per il traffico unicast
- Distribuire la GTK (Group Temporal Key) per il traffico multicast/broadcast

- Sincronizzare l'avvio della cifratura

È importante che le chiavi vengano generate **freshe ad ogni sessione** (grazie ai nonce scambiati nel handshake), così anche se una sessione venisse compromessa, le altre restano sicure — questo principio si chiama **forward secrecy**.

Fase 4 – Protected Data Transfer

Ora il traffico vero e proprio può fluire, protetto da uno dei due protocolli:

- **TKIP** (legacy, usa RC4 + MIC Michael)
- **CCMP** (moderno, usa AES-CTR + CBC-MAC)

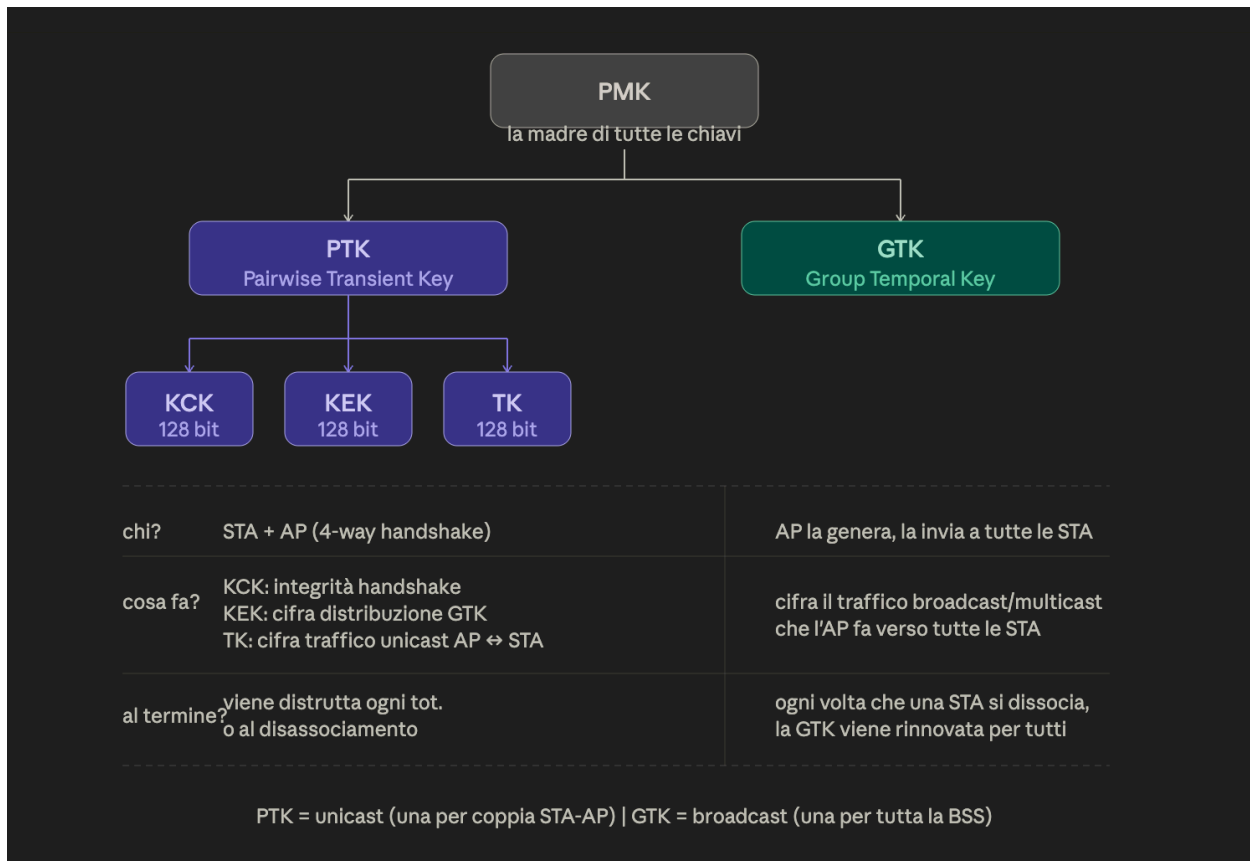
TKIP vs CCMP		
	TKIP	CCMP
Nato per	Aggiornare hardware WEP esistente via software	Hardware nuovo dedicato
Cifratura	RC4 con IV esteso (48 bit)	AES in modalità CTR
Integrità	Michael MIC (64 bit)	CBC-MAC
Sicurezza	Debole, attaccabile	Robusto, standard attuale
Usato in	WPA (legacy)	WPA2/WPA3

Ogni MPDU trasmesso è **cifrato e autenticato**: l'integrità garantisce che i dati non siano stati alterati, la cifratura che non siano leggibili da terzi, e la replay protection (tramite contatori di sequenza) che un attaccante non possa ritrasmettere pacchetti vecchi catturati.

Fase 5 – Connection Termination

Quando la sessione termina (disconnessione volontaria o timeout), STA e AP si scambiano messaggi di **disassociazione e deautenticazione**. Le chiavi temporanee vengono scartate. La GTK viene rinnovata se un dispositivo lascia il gruppo, per impedire che continui ad essere usabile da chi non è più nella rete.

La gerarchia delle chiavi



IMPORTANZA DI OGNI FASE

Fase 1

CI, ISAI — il cookie CI serve a prevenire clogging attack, ISAI contiene la lista di proposte crittografiche (cifrazione + hash) tra cui il responder deve scegliere.

CI, CR, ISAR — CR è il cookie del responder, ISAR contiene la singola proposta accettata; da qui entrambi sanno quali algoritmi useranno.

CI, CR, g^x , NI — g^x è la metà pubblica DH dell'initiator, NI è un nonce fresco che servirà a derivare le chiavi e a proteggersi dai replay.

CI, CR, g^y , NR — speculare al precedente; ora entrambi hanno i dati per calcolare g^{xy} e derivare SKEYID.

SKEYID / SKEYIDd / SKEYIDa / SKEYIDe — da g^{xy} e i nonce si ricava SKEYID master, da cui si derivano tre chiavi separate: d per derivare keying material futuro, a per

l'autenticazione dei messaggi, e per la cifratura.

{ IDI, SIGI } cifrato con SKEYIDe — la firma SIGI autentica tutto lo scambio precedente (hash firmato con la chiave privata dell'initiator), il fatto che sia cifrato protegge l'identità da osservatori esterni.

{ IDR, SIGR } cifrato con SKEYIDe — stesso meccanismo lato responder; dopo questi due messaggi entrambi si sono autenticati mutuamente e l'IKE SA è stabilita.

Fase 2

CI, CR, HASH(1), SAI, NI [, g^x] — HASH(1) autentica il messaggio usando SKEYIDa, SAI propone i parametri per la IPsec SA; g^x è opzionale e serve solo se si vuole Perfect Forward Secrecy.

CI, CR, HASH(2), SAR, NR [, g^y] — il responder accetta la SA e risponde con il proprio nonce; HASH(2) include NI del messaggio precedente per legare i due messaggi insieme.

CI, CR, HASH(3) — messaggio di conferma finale, $\text{HASH}(3) = h(\text{SKEYIDa}, 0, \text{NI}, \text{NR})$; dimostra che l'initiator ha ricevuto correttamente il msg 2 senza trasmettere altro materiale.

[g^x , g^y] opzionali — PFS — se inclusi, la chiave di sessione IPsec non dipende da g^{xy} della fase 1, quindi compromettere la chiave di fase 1 non espone le sessioni passate.

IKE VERSIONE 2

IKEv2 è la versione semplificata e più sicura di IKEv1. Passaggi:

- 1) Initiator manda i parametri DH (g^x), un nonce, e la lista di algoritmi che supporta.
- 2) Responder risponde con il suo valore DH (g^y), il suo nonce, e sceglie gli algoritmi. Opzionalmente chiede il certificato dell'initiator. A questo punto entrambi calcolano g^{xy} e derivano le chiavi di sessione — da qui tutto è cifrato.
- 3) Initiator manda la sua identità, il suo certificato, e la firma AUTH — ovvero firma con la sua chiave privata un hash di tutto lo scambio precedente, così il responder può verificare con la chiave pubblica che è davvero lui. Aggiunge anche la proposta per la prima IPsec SA e i traffic selectors, cioè quali indirizzi e porte deve proteggere questa SA.

4) Responder fa la stessa cosa — manda identità, certificato e firma AUTH. Conferma la IPsec SA proposta. A questo punto entrambi si sono autenticati mutuamente e la IPsec SA è operativa.



La differenza più grande è questa:

In **IKEv1** le due cose sono separate — prima stabilisci il canale sicuro (fase 1, 6 msg), poi negozia la IPsec SA (fase 2, 3 msg). Sono due processi distinti.

In **IKEv2** il canale sicuro e la prima IPsec SA vengono stabiliti **nello stesso scambio** — nei 4 messaggi iniziali fai tutto insieme. Non esiste più la distinzione fase 1 / fase 2.

IKEv1 = 9 messaggi minimi, IKEv2 = 4 messaggi minimi. Stesso risultato, metà del traffico.

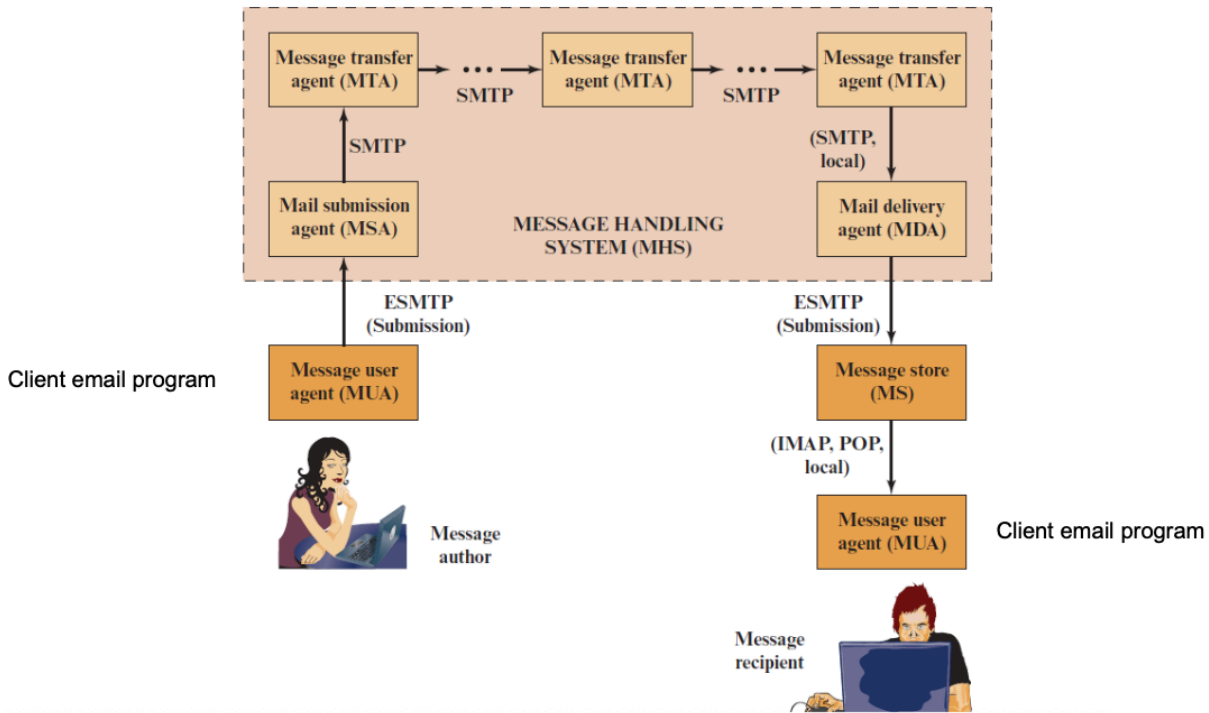
COMBINAZIONE SA DI IPSEC

IPsec permette di combinare più SA insieme — si chiama **SA bundle**. Ci sono due modi: **transport adjacency** (due SA sullo stesso pacchetto, senza tunnel) e **iterated tunneling** (SA annidate con tunnel dentro tunnel). Il risultato pratico sono 4 casi standard che coprono tutti gli scenari reali.

I 4 casi di combinazione SA		
Caso	Protocolli	Cosa fa
1	ESP transport + AH transport	Due host si parlano direttamente, senza tunnel. ESP cifra il payload, AH autentica tutto. → sicurezza massima host-to-host.
2	ESP tunnel	I gateway delle due reti cifrano il traffico tra loro. Gli host interni non fanno nulla. → VPN tra due sedi aziendali.
3	ESP tunnel + AH/ESP transport	Tunnel tra gateway più protezione diretta tra i due host. Doppio livello di sicurezza. → quando non ci si fida nemmeno del gateway.
4	ESP tunnel + ESP tunnel	L'host remoto crea il suo tunnel verso il gateway, dentro un secondo tunnel tra i due gateway. → dipendente in smart working.

EMAIL SECURITY

L'immagine mostra l'architettura standard di come viaggia un'email da chi la scrive a chi la riceve, dividendo il percorso in blocchi.



L'Architettura del Sistema Email

Il viaggio di un'email si divide principalmente in due parti: i programmi usati dagli utenti (**Client**) e la rete di server che sposta il messaggio (**Message Handling System - MHS**).

Ecco cosa fanno i singoli componenti nell'ordine in cui viene inviato il messaggio:

1. La Fase di Invio (A sinistra nel diagramma)

- **MUA (Message User Agent):** È il programma di posta elettronica usato da chi scrive (es. Outlook, Thunderbird, o l'app Gmail). Serve a comporre il messaggio.
- **MSA (Mail Submission Agent):** È il primo server che riceve la mail dal tuo programma. Controlla che non ci siano errori e che tu sia davvero autorizzato a inviare quella mail (autenticazione).

*Protocollo usato: **ESMTP** (una versione estesa e più moderna del classico SMTP, specifica per l'invio iniziale).*

2. Il Trasferimento in Rete (Il blocco centrale rosa: MHS)

- **MTA (Message Transfer Agent):** Sono i veri e propri "uffici postali" di Internet. Si passano la mail l'un l'altro attraverso la rete fino a raggiungere il server del destinatario.

Protocollo usato: SMTP (Simple Mail Transfer Protocol). È il protocollo standard usato esclusivamente per **spostare/inviare** le mail da un server all'altro.

3. La Fase di Ricezione (A destra nel diagramma)

- **MDA (Mail Delivery Agent):** È il server finale che accetta la mail in arrivo dagli MTA e ha il compito di "imbucarla" nella casella corretta del destinatario.
- **MS (Message Store):** È il database, la memoria fisica sul server dove le mail rimangono salvate in attesa che l'utente le legga.
- **MUA (Message User Agent):** È il programma del destinatario, che scarica o visualizza la mail finale.

Protocolli usati per leggere: IMAP o POP. Servono al client dell'utente per **recuperare e leggere** i messaggi dal *Message Store*. (IMAP lascia i messaggi sul server sincronizzati, POP li scarica sul PC).

Sintesi dei Protocolli

1. **SMTP / ESMTP:** Si usa solo per **INVIARE** e **TRASFERIRE** le mail (da client a server e tra server).
 2. **IMAP / POP:** Si usano solo per **RICEVERE** e **LEGGERE** le mail (dal server finale al client del destinatario).
-

I Limiti del vecchio sistema SMTP / RFC 5322

Il sistema originario basato solo su SMTP e RFC-5322(*formato*) aveva dei grossi limiti strutturali:

- **Solo testo semplice:** Non poteva trasmettere file eseguibili o oggetti binari.
- **Niente accenti o caratteri speciali:** Non supportava i caratteri delle lingue nazionali (funzionava solo in ASCII standard a 7 bit).

- **Limiti di dimensione:** I server potevano rifiutare mail oltre una certa grandezza.
-

La Soluzione: MIME (Multipurpose Internet Mail Extensions)

MIME è un'**estensione** nata proprio per superare questi limiti rimanendo compatibile con il vecchio standard. Grazie ad esso e a codifiche come Base64, ha permesso l'invio di allegati e caratteri speciali trasformando dati complessi in testo trasmissibile via SMTP.

Introduce tre elementi fondamentali per supportare la multimedialità:

1. **5 Nuovi Campi Header:** Inseriti nell'intestazione per dare informazioni sul corpo del messaggio.
 2. **Formati di Contenuto (Content Types):** Standardizzano la rappresentazione di elementi multimediali (immagini, audio, video).
 3. **Codifiche di Trasferimento (Transfer Encodings):** Trasformano i dati binari (es. un PDF o un'immagine) in testo sicuro, impedendo che il sistema di posta li alteri durante il viaggio
(es: Base64 per i file binari, Quoted-Printable per il testo..).
-

S/MIME (Secure/Multipurpose Internet Mail Extensions)

Fin'ora le email erano pensate senza sicurezza, mancavano completamente di:

- **Authenticity**: Fingere di essere qualcun'altro (spoofing)
- **Integrity**: Modificare il contenuto del messaggio
- **Confidentiality**: Mail in chiaro, leggibili
- **Availability**: Vulnerabile ad attacchi Dos, impedendo invio/ricezione posta.

La soluzione → **S/MIME**, è il protocollo standard usato per mettere in sicurezza il **corpo del messaggio** (il contenuto gestito da MIME). Si basa sulla crittografia a chiave pubblica (Asimmetrica) garantendo

- **Authenticity e Integrity** tramite **Firma Digitale**
 - **Confidentiality** tramite il solito scambio di **chiave simmetrica tramite crittografia asimmetrica** perchè più efficiente (tipicamente AES-128 CBC mod).
-

SICUREZZA del DNS (DNSSEC, DNS Security Extensions)

Il DNS classico serve a mappare i nomi di dominio (es. `google.com`) in indirizzi IP. Se un hacker riesce a manipolare le risposte DNS (avvelenamento della cache), può dirottare le mail verso server malevoli.

Invece di fidarsi alla cieca delle risposte, **DNSSEC aggiunge una firma digitale a ogni dato del DNS.**

La Triade Antispam/Antispoofing: SPF, DKIM, DMARC

Questi tre strumenti lavorano insieme per verificare che chi invia una mail sia davvero chi dice di essere.

SPF (Sender Policy Framework)

Consente al proprietario di un dominio di pubblicare nel DNS una lista di quali **indirizzi IP sono autorizzati** a inviare email a nome di quel dominio.

- **Come funziona:** Quando il server ricevente riceve la mail, controlla l'IP del mittente confrontandolo con il record SPF del dominio dichiarato nel comando `MAIL FROM` o `HELO`. Se l'IP non è in lista, la mail fallisce il controllo.
 - Si configura tramite un record DNS di tipo `TXT`.
-

DKIM (DomainKeys Identified Mail)

Permette al server di posta mittente (MTA) di **firmare crittograficamente** l'intestazione e il corpo del messaggio.

- **Come funziona:** Il server mittente firma la mail con la sua chiave privata. Il server ricevente scarica la chiave pubblica del mittente direttamente dal DNS e verifica la firma.
 - **Cosa garantisce:** Garantisce che la mail provenga davvero da quel dominio e che il corpo del messaggio non sia stato alterato in transito.
-

DMARC (Domain-based Message Authentication, Reporting, and Conformance)

È il "regista" che unifica SPF e DKIM. Richiede che l'indirizzo visibile nel campo `From:` sia allineato con i domini verificati da SPF o DKIM.

- **A cosa serve:** Permette al proprietario del dominio di scrivere una regola nel DNS per dire ai server riceventi cosa fare se una mail fallisce i controlli SPF/DKIM.

STARTTLS

Il protocollo utilizzato oggi, serve a **trasformare una connessione nata in chiaro (non sicura) in una connessione cifrata (sicura)**, usando lo stesso identico canale.

Funziona come un "aggiornamento" al volo della sicurezza:

1. **La partenza in chiaro:** Il tuo server email si collega a un altro server usando il normale protocollo SMTP (senza crittografia, quindi chiunque in rete potrebbe intercettare il testo).
2. **La richiesta:** Prima di iniziare a mandare la mail vera e propria, il tuo server invia il comando: *"Ehi, supporti la crittografia? Facciamo uno **STARTTLS**?"*.
3. **Il passaggio a TLS:** Se l'altro server risponde di sì, i due si scambiano le chiavi crittografiche e, da quel momento in poi, tutto il resto della conversazione (mittente, destinatario, testo e allegati) viene **cifrato tramite TLS**.

Note: Viene fatto un nuovo STARTTLS tra ogni server di passaggio, ma la sicurezza non è compromessa perchè è presente una crittografia END-to-END essendo che abbiamo cifrato il contenuto del messaggio con una chiave simmetrica protetta dalla chiave pubblica del destinatario. Solo lui può aprirla, quindi otteniamo protezione del contenuto e protezione del trasporto.

SICUREZZA FIREWALL

Il firewall è l'unico punto di passaggio obbligato tra il “**dentro**” (una rete sicura) e il “**fuori**” (Internet, insicuro). **Tutto il traffico passa da lì, filtrando quello non autorizzato.**

(limite: il firewall è quasi inutile se l'attacco parte da dentro)

I 4 TIPI DI FIREWALL

1. Packet Filtering (il buttafuori veloce)

è la forma più semplice, lavora ai livelli IP e TCP/UDP (lvl. 3 e 4). **Guarda ogni singolo pacchetto** e decide se farlo passare, controllando solo: **Indirizzo Mittente, Indirizzo Destinatario, Porta**.

- Pro: è velocissimo e trasparente agli utenti
- Contro: non ha memoria (STATELESS), se un pacchetto malevolo sembra tecnicamente corretto, lo fa passare.

Vulnerabilità:

- Per consentire il traffico di ritorno o protocolli dinamici, il packet filtering è spesso costretto a lasciare aperte tutte le porte sopra la 1024, aprendo il fianco agli attaccanti.

- **IP Address Spoofing:** L'hacker maschera i pacchetti esterni assegnandogli un IP finto appartenente alla tua rete interna. (Soluzione: il firewall deve scartare i pacchetti in ingresso se l'IP sorgente risulta interno).

- **Source Routing Attacks:** L'attaccante specifica nel pacchetto il percorso di rete esatto da fare, cercando di aggirare i controlli. (Soluzione: scartare i pacchetti che usano questa opzione).

- **Tiny Fragment Attacks:** L'attaccante spezza l'header TCP in frammenti IP minuscoli per nascondere i dati malevoli al firewall. (Soluzione: imporre una grandezza minima obbligatoria per il primo frammento).

2. Stateful Inspection (il buttafuori con memoria)

l'evoluzione del precedente ma molto più sicuro, **non guarda solo il singolo pacchetto, ma mantiene una tabella di stato delle connessioni**. Capisce se un pacchetto fa parte di una conversazione già aperta e autorizzata o se sta cercando di intrufolarsi.

Source Address	Source Port	Destination Address	Destination Port	Connection State
192.168.1.100	1030	210.9.88.29	80	Established
192.168.1.102	1031	216.32.42.123	80	Established
192.168.1.101	1033	173.66.32.122	25	Established
192.168.1.106	1035	177.231.32.12	79	Established
223.43.21.231	1990	192.168.1.6	80	Established

3. Application-Level Gateway (il proxy)

qui la sicurezza è massima, questo firewall non fa passare i pacchetti direttamente ma si mette in mezzo: l'utente si connette al proxy, e il proxy si connette al server esterno.

- Pro: altissima sicurezza, nasconde la rete interna
- Contro: è lento perchè deve processare tutto il contenuto ("overhead" elevato)

Come funziona:

In entrambe le direzioni l'utente invia un flusso di pacchetti al proxy, questo li raccoglie, ricostruisce il messaggio completo e lo controlla.

Se il messaggio è sicuro e autorizzato, il proxy crea nuovi pacchetti (con il proprio ip come mittente) e li spedisce verso il server di destinazione.

Quindi il proxy fa da punto intermedio tra interno ed esterno per entrambi i sensi di comunicazione. Questo lo rende molto sicuro in quanto interrompe la comunicazione diretta.

4. Circuit-Level Gateway (una via di mezzo)

come il proxy, spezza la connessione in 2, ma non controlla il contenuto approfondito dei pacchetti. Si limita a verificare che l'handshake sia valide. Una volta stabilita la connessione, lascia passare i pacchetti velocemente.

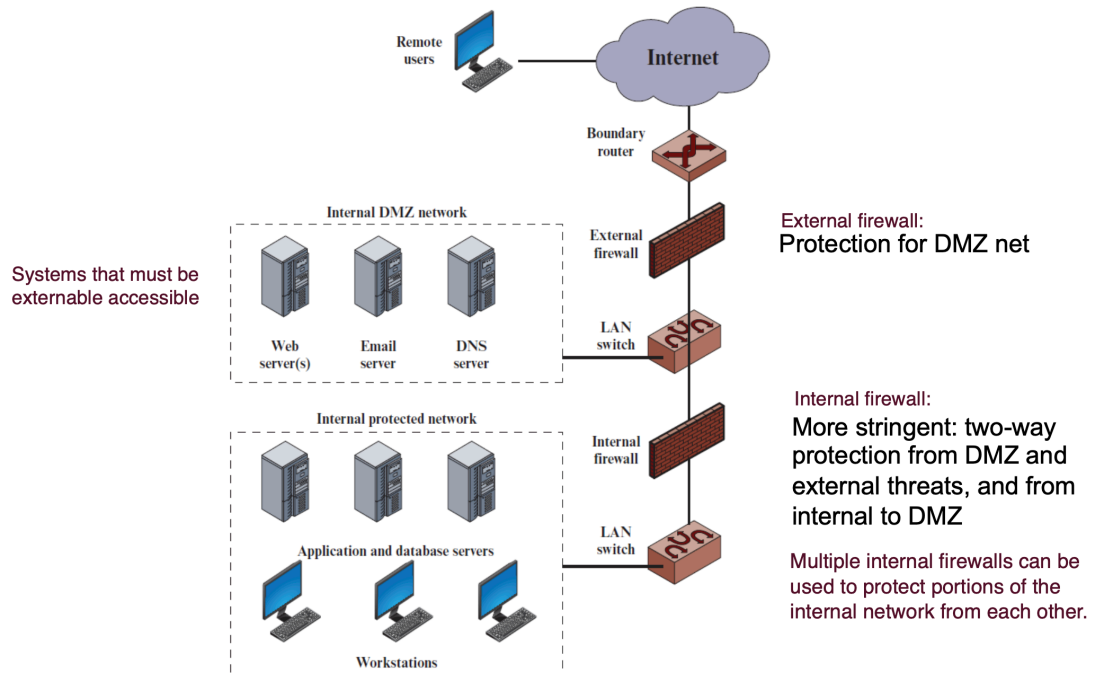
L'ARCHITETTURA

Non basta avere un firewall, bisogna saperlo posizionare, i modi:

- Bastion Host: è il computer che ospita il firewall. Essendo esposto, deve essere "blindato" (Hardening) togliendogli tutti i servizi non indispensabili.
- DMZ (Demilitarized Zone): è una zona cuscinetto dove vengono messe risorse e server (come un sito web) "sacrificabili".

Questa zona poi viene ulteriormente isolata da un altro firewall come una stanza intermedia tra la rete interna da proteggere e il mondo esterno. Se un Hacker buca il server web nella DMZ, è comunque bloccato dal firewall interno.

Configuration (DMZ)



Il firewall classico blocca accessi non autorizzati (chiude porte). Ma se l'attacco passa attraverso quelle aperte (es: sito web su porta 80)?

IPS (Intrusion Prevention System)

IPS (Intrusion Prevention System), che sta "in linea" ovvero il traffico ci passa attraverso, e analizza il comportamento del traffico per bloccare attacchi specifici.

- HIPS (Host-Based): quando IPS si trova sul pc dell'host a cui arrivano i pacchetti. Quando arriva un file/pacchetto, l'HIPS guarda se ne riconosce la firma (virus noto?), se SÌ la blocca, altrimenti lo esegue in una sandbox e se nota comportamenti anomali/strani lo uccide e ti avvisa.
- NIPS (Network-Based): è un hardware sulla rete che controlla pacchetti alla ricerca di firme di attacco note o anomali (es: DoS) e le scarta immediatamente.

IDS (Intrusion Detection System)

IDS (Intrusion Detection System) > È collegato ad una porta dello switch, non interrompe il traffico (è in parallelo). Si occupa di Ascoltare, Analizzare e Allertare

(quindi sostanzialmente "ti avvisa" se vede passare pacchetti sospetti o malevoli ma non li blocca).

- **HIDS (Host-Based):** È installato dentro un singolo computer o server. Non guarda i cavi di rete, ma controlla "da dentro" i file di sistema, i log e le attività di quella specifica macchina per scovare comportamenti sospetti.
- **NIDS (Network-Based):** È il sensore sulla rete che ascolta i pacchetti per scovare le intrusioni. Usa 3 tipi di firme: di Stringa (cerca testo pericoloso), di Porta (controlla tentativi verso porte note) e di Header (cerca intestazioni anomale).

L'Architettura NIDS (I 4 punti strategici in cui metterlo):

1. Fuori dal firewall esterno (per vedere quanti attacchi arrivano).
2. Nella DMZ (per proteggere i server pubblici sacrificabili).
3. Dietro il firewall interno (per bloccare chi riesce a bucarlo).
4. Sulle LAN dei dipendenti (per fermare minacce interne, es. un worm da chiavetta).

UTM (Unified Threat Management)

Oggi la soluzione più comoda (non sicura) è usare un UTM, ovvero un unico dispositivo che fa da Firewall, IPS, Antivirus e VPN tutto in uno.

note: Antivirus = analisi statica, HIPS (analisi dinamica)

MIXED NETWORKS

Il problema di base è che nonostante tutto quello che abbiamo detto, la crittografia non basta.

Internet è progettato come una rete pubblica, i pacchetti IP hanno gli header che mostrano chiaramente l'indirizzo sorgente e quello di destinazione.

L'errore da non fare è confondere confidenzialità e anonimato:

la crittografia (come IPsec) nasconde il *contenuto* (payload) del messaggio, ma **non nasconde le informazioni di routing**. Un osservatore passivo sa sempre chi sta parlando con chi (gateway).

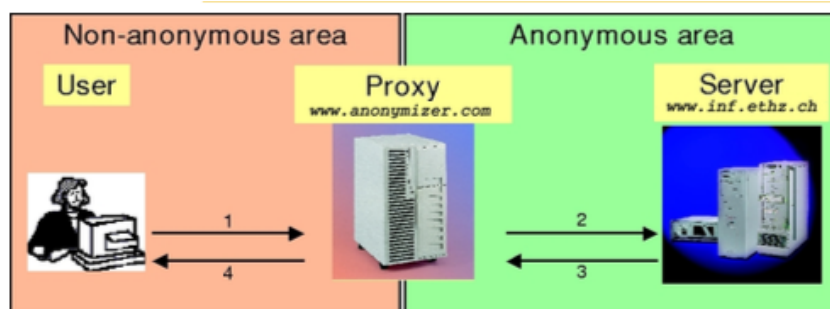
Definizioni Rigorose (da sapere a memoria)

- **Anonimato:** È lo stato di non essere identificabili all'interno di un gruppo di soggetti, chiamato *anonymity set*. Regola d'oro: non puoi essere anonimo da solo; più grande è il gruppo, migliore è l'anonimato.
- **Unlinkability (Non-collegabilità):** Impossibilità di collegare un'azione alla sua identità (es. mittente e messaggio inviato non possono essere associati dopo aver osservato la rete).
- **Unobservability (Inosservabilità):** Un livello ancora più alto, dove l'attaccante non riesce nemmeno a capire se una comunicazione ha avuto luogo o meno.

L'Evoluzione dell'Anonimato (Proxy vs. Mix Networks)

I Proxy

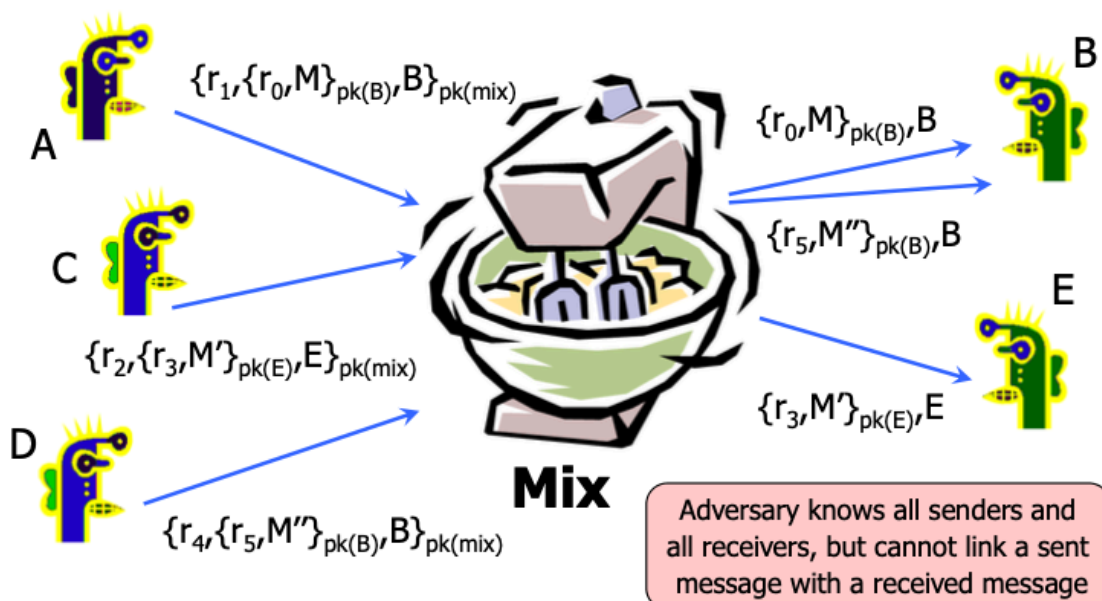
- Il traffico passa attraverso un proxy server che maschera l'IP del mittente.
- *Il dettaglio letale:* C'è un **Single Point of Failure**. Il proxy conosce tutto (sorgente e destinazione) ed è estremamente vulnerabile all'analisi del traffico. Anche concatenando più proxy e aggiungendo crittografia, l'analisi dei tempi e dei volumi di traffico permette ancora di tracciare i pacchetti. (un esempio: Firewall Proxy)



Mix Networks (Il modello di Chaum)

Sviluppato da David Chaum nel 1981, questo sistema è stato originariamente concepito per rendere non tracciabile la posta elettronica. L'obiettivo primario è difendere le comunicazioni da un attaccante in grado di osservare attivamente tutto il traffico di rete.

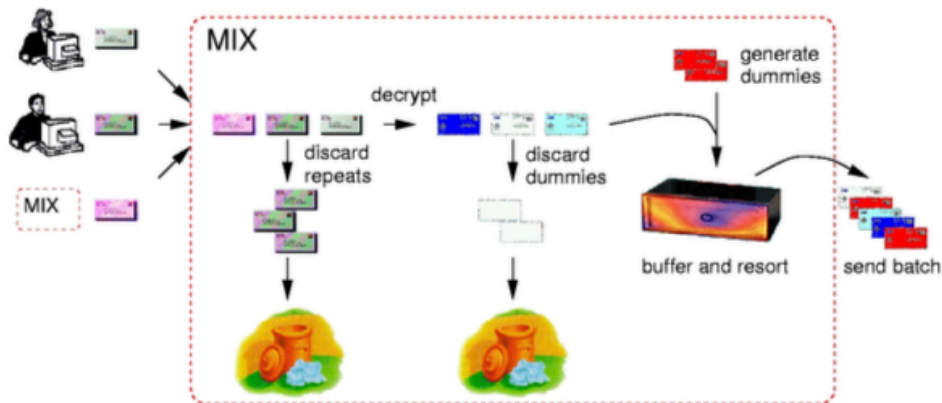
La meccanica del sistema: Un server Mix riceve messaggi criptati con la sua chiave pubblica e, per impedire a un osservatore esterno di collegare i pacchetti in entrata con quelli in uscita, esegue specifiche manipolazioni sui dati.



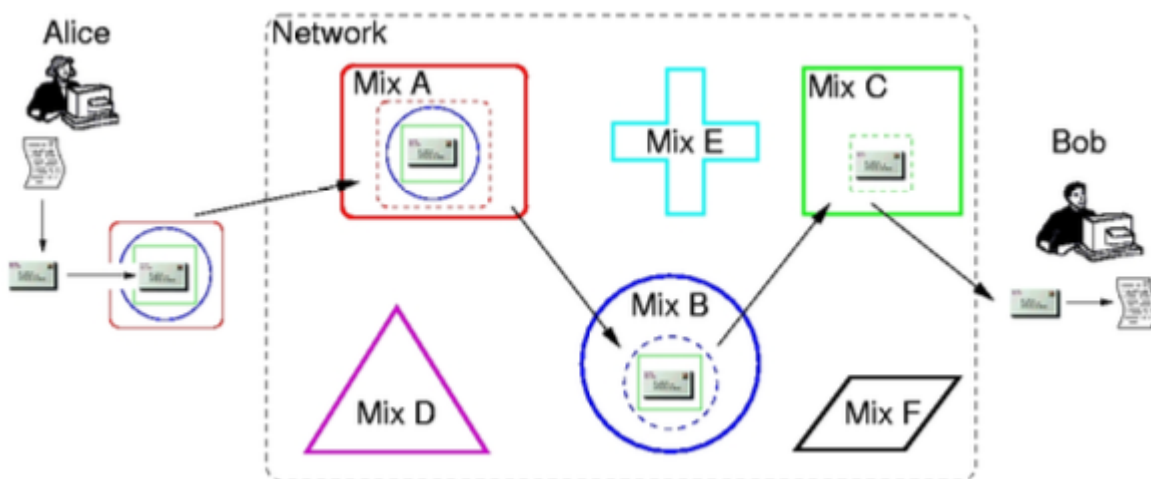
- **Uniformità delle dimensioni:** Tutti i pacchetti in transito vengono frammentati o riempiti con stringhe di bit casuali (padding) affinché assumano esattamente la medesima lunghezza fissa.
- **Rimozione delle anomalie:** Il sistema decifra i pacchetti e scarta i messaggi identici per prevenire attacchi "replay" (dove un attaccante reinvia un pacchetto intercettato per tracciarlo). Rimuove inoltre eventuali messaggi fittizi generati ai livelli precedenti.
- **Raggruppamento e riordino (Batching e Resorting):** Il Mix non inoltra i messaggi singolarmente al momento della ricezione. Li immagazzina in un buffer temporaneo, genera e inserisce nuovo traffico fittizio (dummy traffic) per

mascherare i volumi reali, e infine inoltra i pacchetti tutti insieme in blocchi (batch), secondo un ordine casuale o rigorosamente predefinito .

Functionality of a Mix



- **Architettura a cascata:** Poiché affidarsi a un singolo Mix significa creare un punto debole centrale, i messaggi vengono instradati attraverso una catena di server indipendenti . Il mittente cifra il messaggio in più strati concentrici; ogni server Mix rimuove uno strato e passa il risultato al nodo successivo . Il limite operativo risiede nel fatto che le continue operazioni crittografiche a chiave pubblica e l'attesa nei buffer generano un'elevata latenza, adatta alle email ma non alla navigazione Web



Il limite pratico: Tutto questo mescolare e ritardare crea un'alta latenza. Va benissimo per le email, ma è disastroso per la navigazione Web.

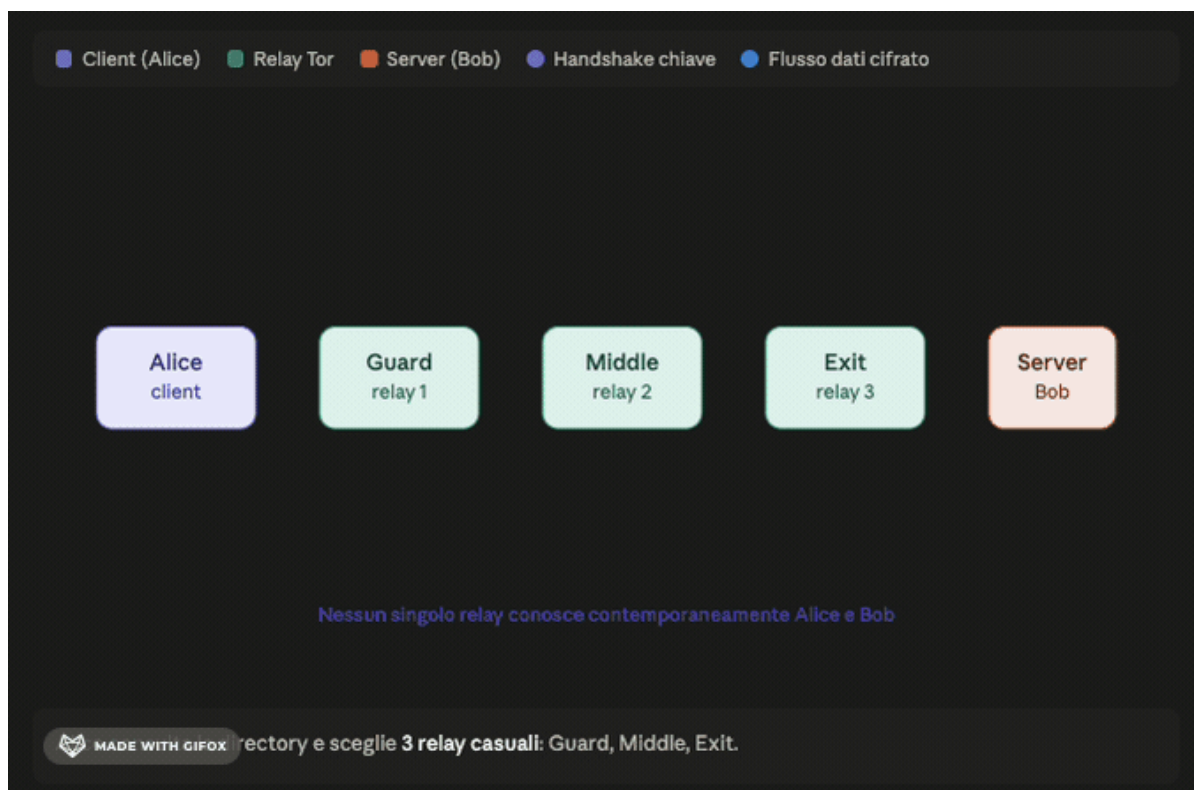
Onion Routing e TOR (La rete a bassa latenza)

L'Onion Routing viene implementato per superare le limitazioni di latenza delle reti Mix, rendendo possibile una navigazione Web anonima e interattiva.

La meccanica del sistema:

L'innovazione fondamentale risiede nel limitare l'uso della lenta crittografia a chiave pubblica esclusivamente alla fase di instaurazione del percorso, passando poi alla più rapida crittografia simmetrica per la trasmissione dei dati reali

- **Creazione del circuito:** Il software del client seleziona un percorso casuale di router all'interno della rete. Attraverso la crittografia a chiave pubblica, instaura una chiave di sessione simmetrica condivisa con il primo nodo . Mantenendo aperto questo collegamento, estende il circuito instaurando una nuova chiave simmetrica con il secondo nodo, e ripete il processo per il terzo .



- **La "Cipolla", Incapsulamento e trasmissione:** Completata la creazione del circuito, i pacchetti dati da inviare vengono crittografati in strati multipli, utilizzando in sequenza le chiavi simmetriche concordate con i nodi.



- **Hidden Services (Dark Web), Instradamento vincolato:** Durante il transito nella rete, ogni nodo possiede esclusivamente la chiave simmetrica necessaria per decifrare lo strato esterno di competenza. Questa operazione rivela unicamente le istruzioni di routing per raggiungere il nodo immediatamente successivo. Di conseguenza, nessun router dispone delle informazioni complete: il primo nodo dialoga con il mittente ma ignora la destinazione finale, mentre l'ultimo nodo invia il traffico alla destinazione ignorando chi lo abbia generato in origine.



Casi Studio: Come crolla l'anonimato

Tor mitiga gli attacchi, ma non li rende impossibili. I casi reali mostrano che la crittografia raramente viene "rotta"; a fallire è l'implementazione o l'utente.

- **Harvard Bomb Threat (Correlation Attack):** Uno studente ha inviato un allarme bomba usando Tor. È stato beccato perché i log della rete dell'università hanno mostrato che lui era *l'unica persona* connessa a Tor in quel preciso momento. L'anonymity set locale era troppo piccolo.
- **LulzSec (Traffic Analysis e OpSec):** Un hacker (Hammond) è stato incastrato incrociando gli orari in cui chattava su IRC tramite Tor con i grafici di utilizzo della

sua rete domestica.

- **Freedom Hosting (Browser Exploit):** L'FBI non ha attaccato la rete Tor in sé, ma ha sfruttato una vulnerabilità JavaScript (*CVE-2013-1690*) in una versione vecchia di Firefox (su cui si basa il Tor Browser). Il payload maligno ha fatto sì che i computer degli utenti aggirassero Tor, inviando direttamente all'FBI il loro vero IP pubblico e l'indirizzo MAC. La lezione? Se il software alle estremità (il browser) è bucato, l'anonimato della rete è inutile.
-

SCENARI DI ATTACCO SU TOR

Tor è vulnerabile a diverse tipologie di attacco. L'obiettivo della rete non è garantire un'invulnerabilità assoluta, ma cercare di rendere gli attacchi il più difficili possibile da eseguire. Di conseguenza, alcune minacce vengono completamente neutralizzate dal protocollo, mentre per altre è possibile solamente ridurne gli effetti.

L'elenco dei principali vettori di attacco comprende:

- **Cells tampering** (Manomissione delle celle dati)
 - **Traffic analysis** (Analisi del traffico)
 - **Sybil attack** (L'attaccante crea un vasto numero di router fittizi per compromettere la rete)
 - **Browser attack** (Sfruttamento di vulnerabilità del software client, come avvenuto nel caso Freedom Hosting)
 - **Bad Apple attack**
 - **Congestion attack** (Attacco di congestione della rete)
 - **Website Fingerprinting on Tor** (Analisi dei pattern di traffico per identificare il sito specifico visitato dall'utente)
-